KONZEPTIONELLES DATENDESIGN

Kapitel 2

ÜBERSICHT

Einführung Modellierung

- Phasenmodell, Begriffe: Entity, Attribut, Entity-Typ

Vom konventionellen Datendesign zum Konzept – Übersicht

- ERD – Übersicht, Kardinalität, (min, max) Notation

Vom konventionellen Datendesign zum Konzept – Details

- Beziehungen mit Attributen, Unterschiede zwischen Beziehungs-Typ und

VOM KONZEPTIONELLEN DATENDESIGN ZUM MODELL

Es wird ermittelt, welche Dinge (**Entities**) für eine Organisation / ein System bedeutend sind, welche Eigenschaften (**Attribute**) diese Dinge haben und wie sie zueinander in Beziehung (**Relations**) stehen.

Beziehungen zwischen Entity-Typen können folgendermaßen beschrieben werden:

- Beziehungen werden durch **Entity-Relationship-Modell** ermittelt
- Grafische Darstellung erfolgt durch das **Entity-Relationship-Diagramm (ERD)**

Re·la·ti·on

/relaˈtsioːn,Relatión/

Substantiv, feminin [die]

1a. **BILDUNGSSPRACHLICH; FACHSPRACHE**
Beziehung, in der sich [zwei] Dinge, Gegebenheiten, Begriffe vergleichen lassen oder [wechselseitig] bedingen; Verhältnis
"logische Relationen"

1b. **MATHEMATIK**
Beziehung zwischen den Elementen einer Menge (2)

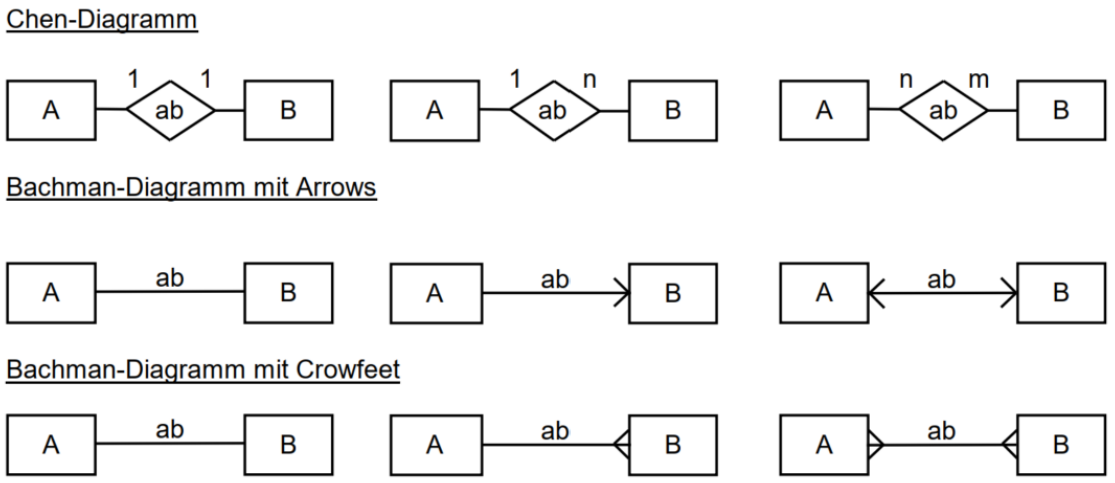
ENTITY-RELATIONSHIP-DIAGRAMM (ERD)

Darstellung der Beziehungen zwischen Entity-Typen

Notationsformen:

- Chen
- Bachmann

Implementierungs- und Performanceüberlegungen sind in diesem Stadium der Datenmodellierung **nicht** anzustellen

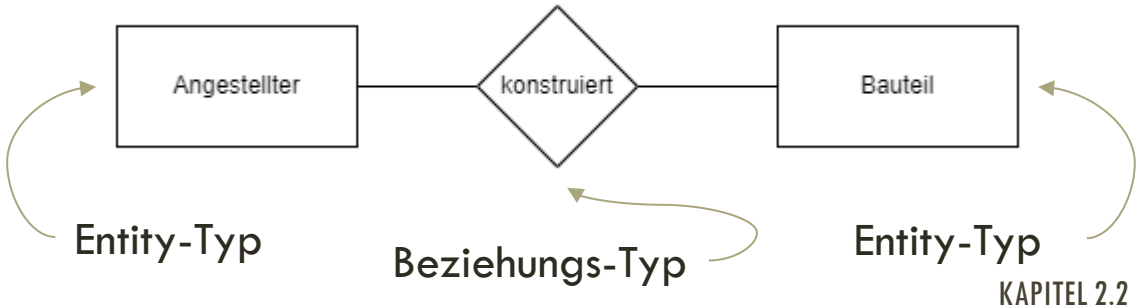


ERD NACH CHEN

Der Begriff **Relationship** ist ein Überbegriff für Relationship-Instance (Beziehung) und Relationship-Type (Beziehungs-Typ)

Entity-Relationship-Diagramm, ER-Diagramm (nach P. Chen, 1976):

- **Entity-Typ**: Rechteck
- **Beziehungs-Typ**: Raute mit Linien zu den, am Beziehungs-Typ beteiligten, Entity-Typen
- **Attribute von Entity-Typen** werden in ER-Diagramme, wegen der Übersichtlichkeit, üblicherweise nicht eingezeichnet (sondern in textlicher Darstellung als Legende angegeben)



ERD – ÜBERSICHT

Der Entity-Typ „Bauteil“ beschreibt **alle Möglichkeiten** eines Bauteils und keine konkrete Möglichkeit

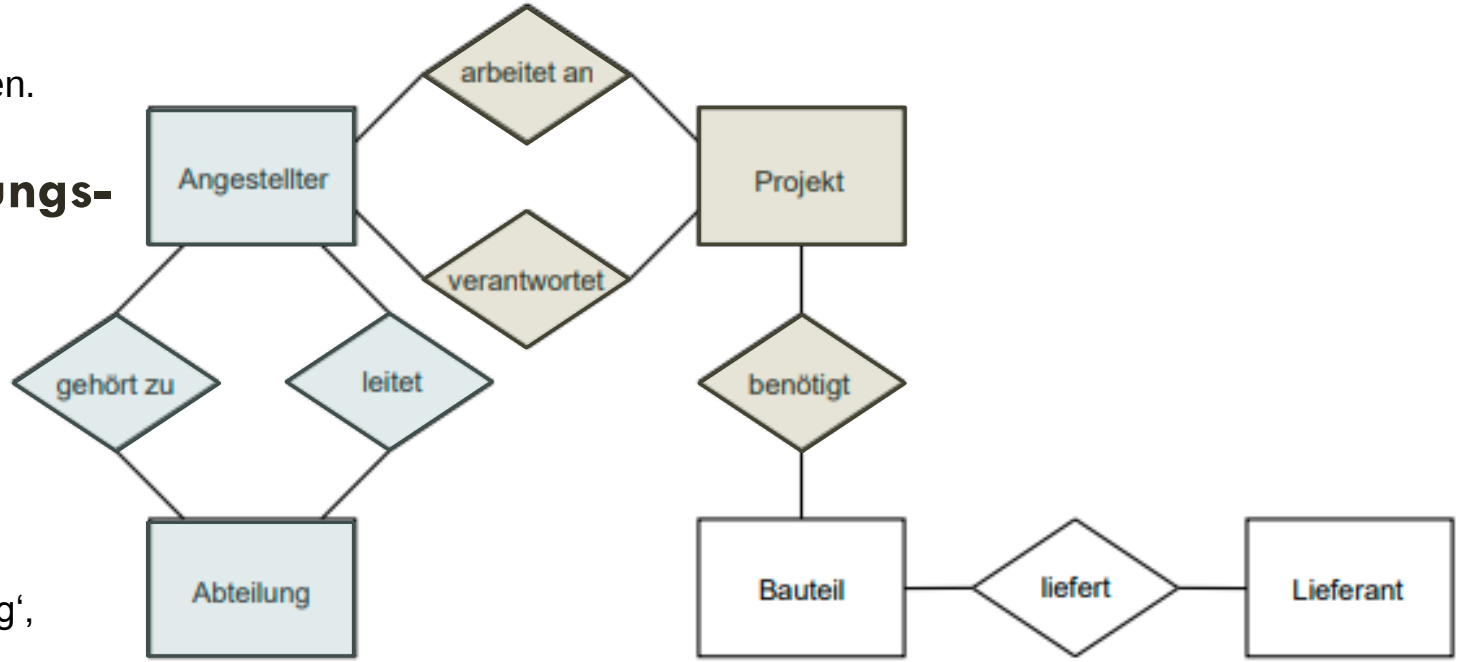
- Beispiel: M5 Schraube, sondern alle Möglichkeiten.

Ein Entity-Typ kann **mehrere Beziehungs-Typen** haben

- Beispiel: Entity-Typ 'Projekt', Beziehungs-Typen 'verantwortet‘, 'arbeitet an' und 'benötigt‘

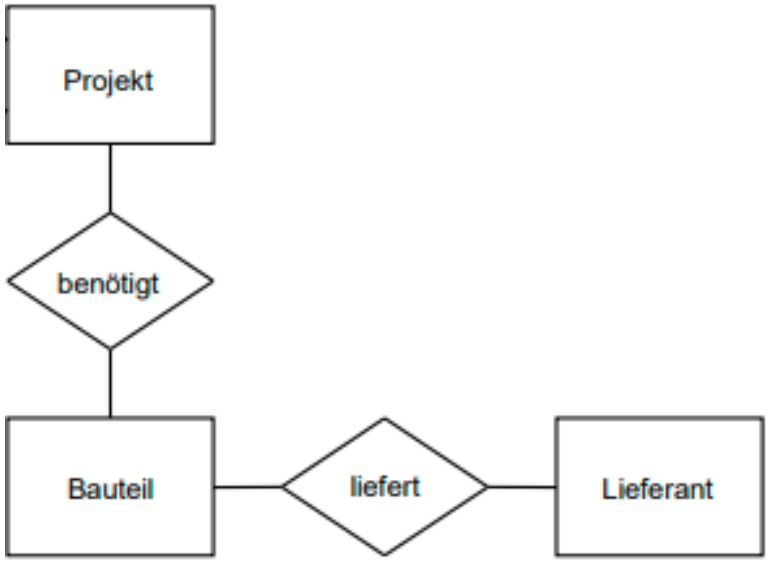
Zwei Entity-Typen können **mehrere Beziehungs-Typen** haben

- Beispiel: Entity-Typen 'Angestellter' und 'Abteilung‘, Beziehungs-Typen 'gehört zu' und 'leitet'



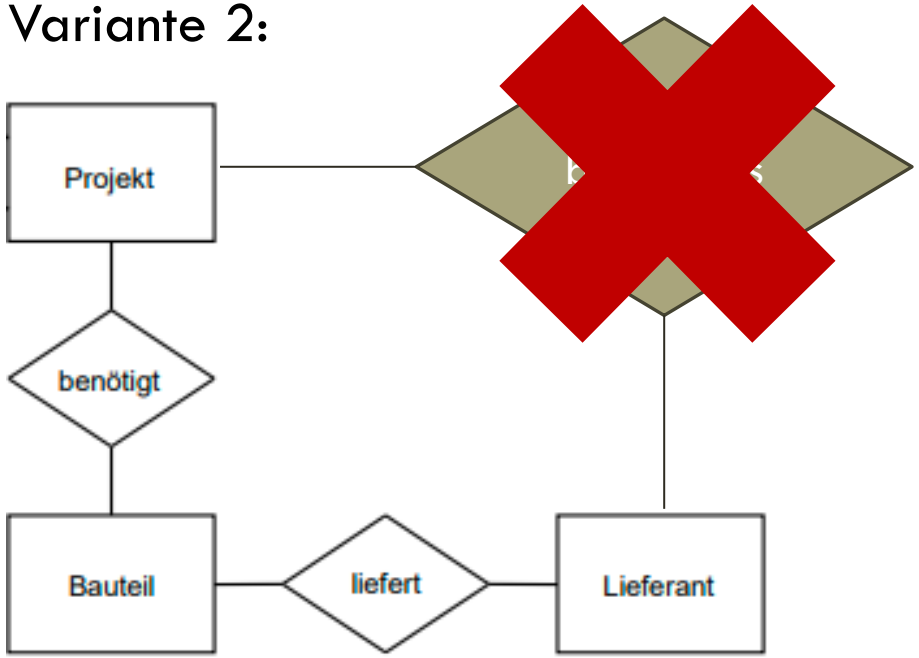
ERD – ÜBERBESTIMMTHEIT

Variante 1:



Variante 2 überbestimmt:
Informationen, die aus bestehenden Beziehungen abgeleitet werden können, dürfen nicht als eigene Beziehungen nochmals definiert werden

Variante 2:

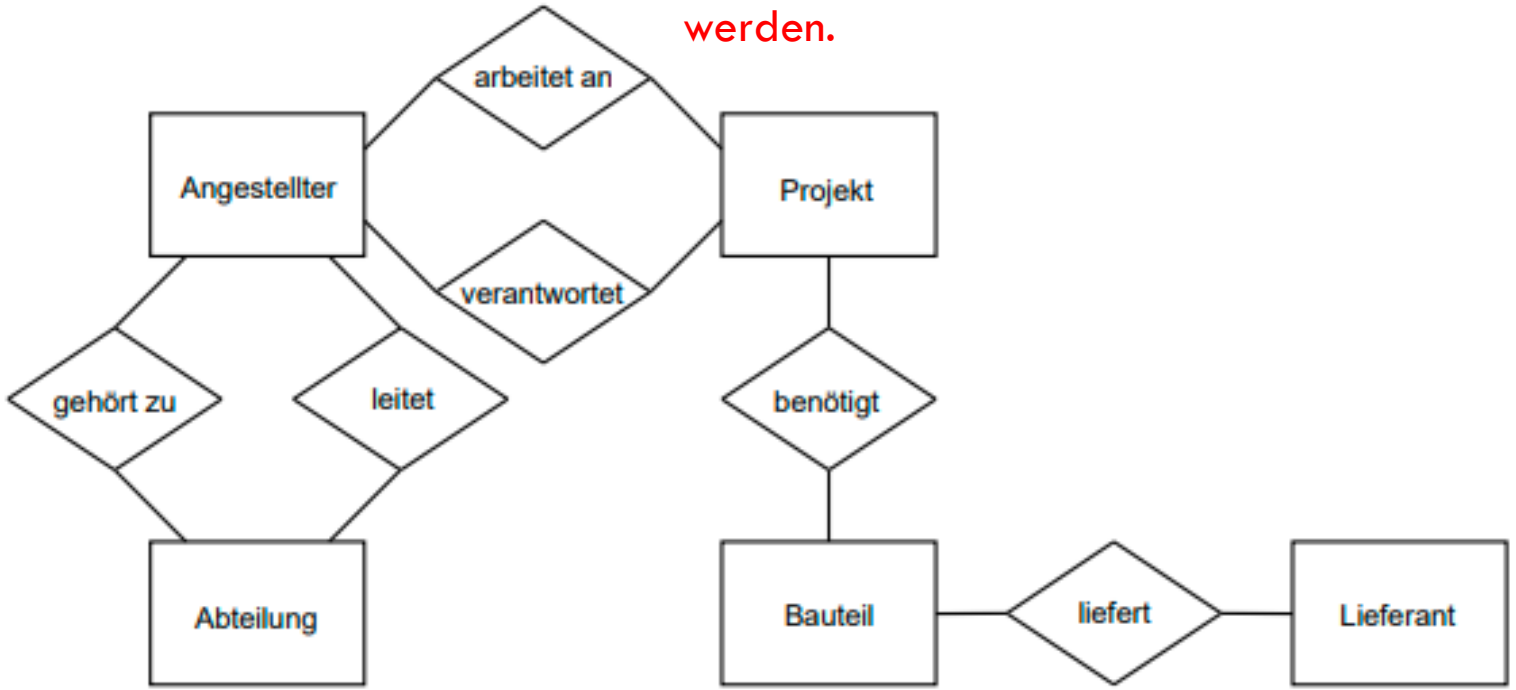


DISKUSSION: ÜBERBESTIMMTHEIT

Frage: Ist die direkte und eindeutige Ableitung der Information 'Welche Lieferanten beliefern welche Projekte' möglich?

- Beziehungs-Typ 'benötigt' und Beziehungs-Typ 'liefert‘ haben keine Informationen über welche Lieferanten welche Projekte beliefert.
- Selbst durch JOIN nicht eindeutig möglich, da mehrere Lieferanten das Projekt beliefern könnten

Auch wenn zwischen zwei Entity-Typen eine Beziehung besteht, können nicht automatisch alle Beziehungsinformationen abgeleitet werden.



ÜBUNG: ERD & ÜBERBESTIMMUNG

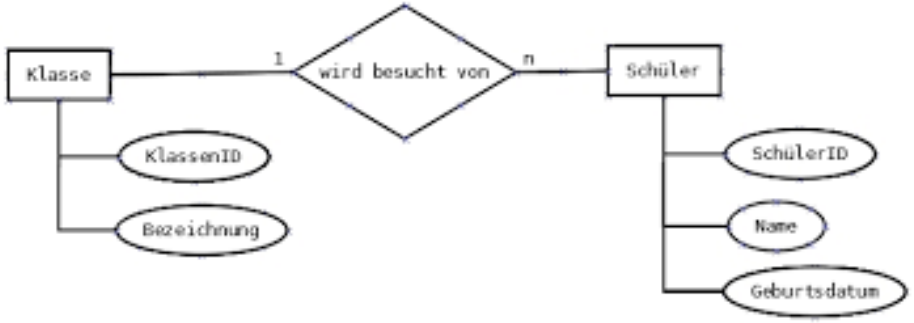
Sie werden beauftragt ein Bestellsystem zu entwickeln. Überlegen Sie sich ein mögliches ERD wie Bestellungen sowie die dazu gehörigen Artikel dargestellt werden.

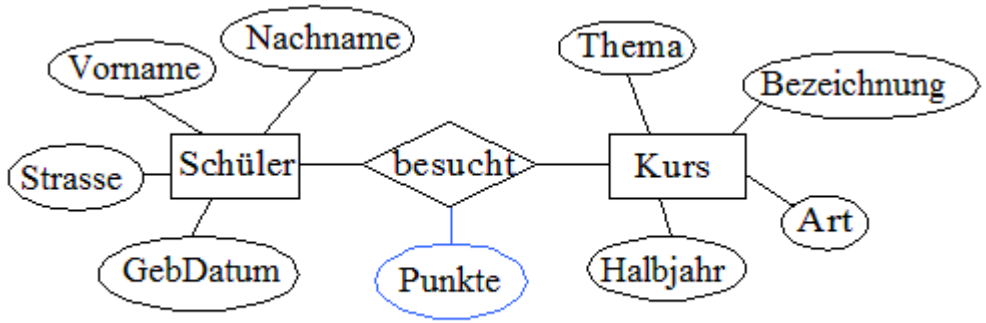
- Wie handhaben Sie folgende Dinge: Preise der einzelnen Artikel sowie die Gesamtsumme, Lieferadresse des Kunden, Rechnung, …
- Erarbeiten Sie ein ERD, das Ihrer Meinung nach **nicht überbestimmt** ist.
- Erarbeiten Sie ein ERD, das Ihrer Meinung nach **überbestimmt** ist

ERD – KOMPONENTEN

Mögliche Komponenten eines ERDs (welche davon kennen Sie bereits):

- Beziehung vs. Beziehungs-Typ
- Kardinalität
- (min,max)-Notation
- Attribute
- Beziehungs-Typ vs. Entity-Typ
- Attribute vs. Entity-Typ
- Rekursive Beziehungstypen
- Mehrwertige Beziehungs-Typen





BEZIEHUNG

Beziehung (Relation, Relationship): Zwischen zwei (in der Folge auch mehr als zwei) **Entities** kann **eine Beziehung** (ein Zusammenhang) einer **bestimmten Bedeutung** bestehen.

Beispiele:

- Lisa Bonn konstruiert eine Schraube
- Bill Yard konstruiert einen Hammer

Verallgemeinerung? Ein Angestellter konstruiert ein Bauteil

BEZIEHUNGS-TYP

- Klara Fall tätigt eine Buchung von 199.99
- Paul Panther tätigt eine Buchung von 249.99

Verallgemeinerung? Ein Angestellter tätigt eine Buchung

BEZIEHUNGS-TYP

Beziehungs-Typ (Relation-Type, Beziehungs-Menge, Relation-Set): Beziehungen, an denen Entities des **gleichen Entity-Typs beteiligt sind** und die **dieselbe Bedeutung** haben, werden zu einem Beziehungs-Typ zusammengefasst.

Jeder Beziehungs-Typ bekommt einen **Namen** (meist ein **Zeitwort**).

Unterscheidung zwischen Beziehung und Beziehungs-Typ ist wichtig

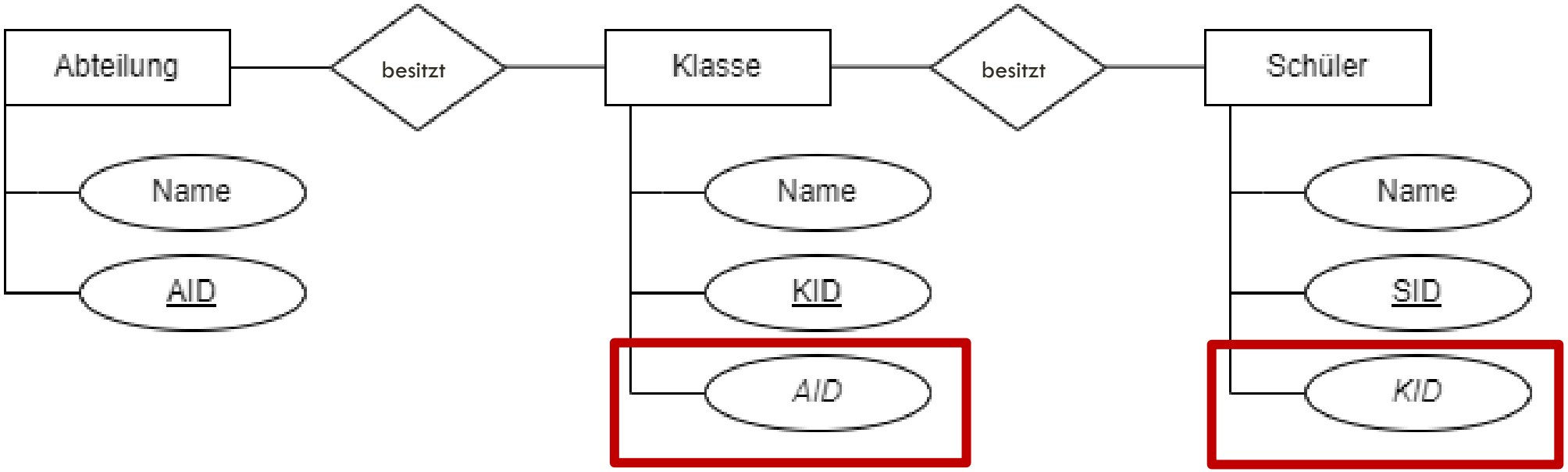
Beispiele:

- Angestellte **konstruieren** Bauteile
- Angestellte **tätigen** Buchungen

ÜBUNG: SCHULE MODELLIEREN

Wie würden Sie mittels eines ERD die Abteilungen in der HTL modellieren?

- Was sind mögliche Entity-Typen, Attribute sowie Relationen?



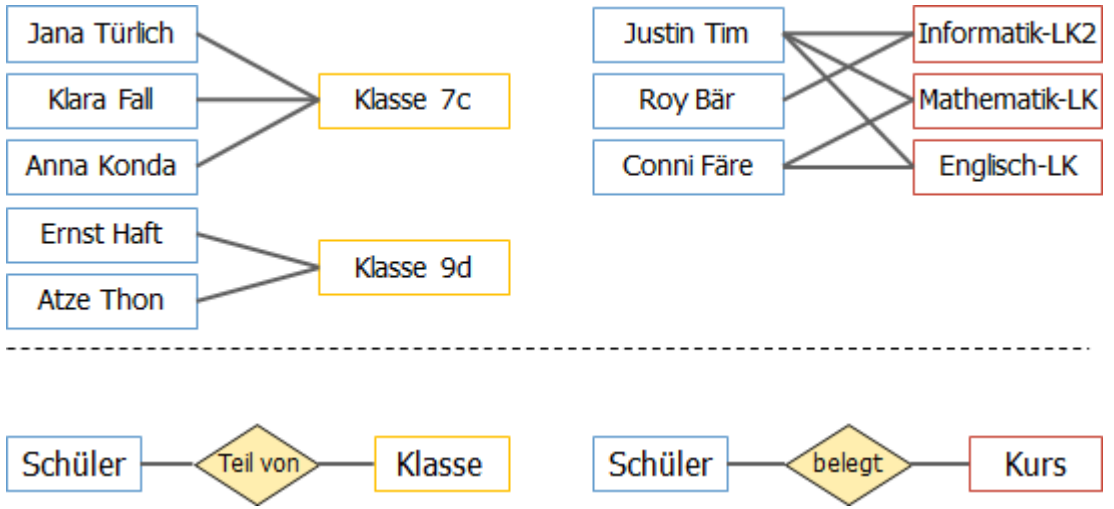
Keine Foreign Keys im ERD hinzufügen, diese werden durch den Beziehungstyp ermittelt

KARDINALITÄT EINER BEZIEHUNG

Die Art des Beziehungs-Typs wird auch **Beziehungskardinalität** (Cardinality), Konnektivität oder Komplexität eines Beziehungs-Typs genannt.

Verfügbare Beziehungstypen:

- 1:1 Beziehungstyp
- 1:n Beziehungstyp
- n:m Beziehungstyp



Interaktiv: https://www.inf-schule.de/datenbanksysteme/ermodelle/kardinalitaet

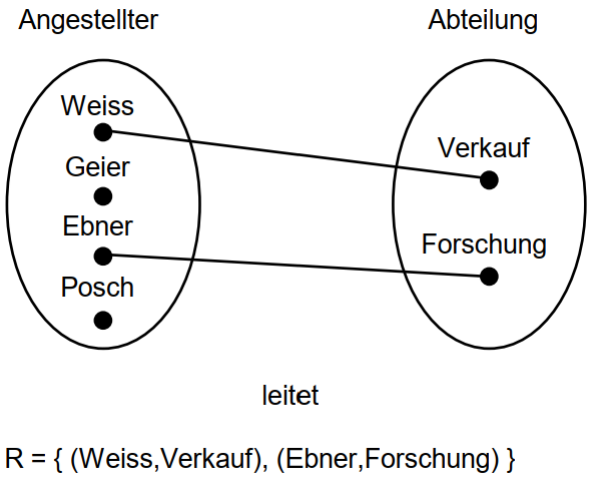
KARDINALITÄT – 1 :1 BEZIEHUNGSTYP

Jedem Entity des einen Entity-Typ ist **höchstens <u>ein</u> Entity** des anderen Entity-Typs zugeordnet und umgekehrt.

- `Beispiel:[Angestellter] ---1--- <leitet> ---1--- [Abteilung]`
- `Beispiel: [Mann] ---1--- <ist verheiratet mit> ---1--- [Frau]`

In einem 1:1-Beziehungs-Typ muss die mathematische Relation eine Funktion mit **<u>existierender Umkehrfunktion</u>** sein.

Die Entities der **beiden** Entity-Typen kommen in den Paaren der Relation höchstens **<u>einmal</u> vor.**



R = { (Weiss,Verkauf), (Ebner,Forschung) }

ÜBUNG: KARDINALITÄT – 1:1 BEZIEHUNGSTYP

Beispiel: Beziehung zwischen Schüler und Schülerausweis darstellen?

- 1 Schüler besitzt genau 1 Schülerausweis



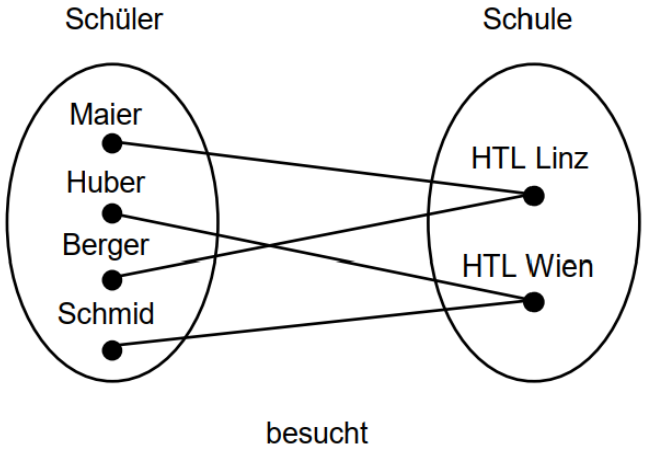
KARDINALITÄT – 1:N BEZIEHUNGSTYP

Jedem Entity des einen Entity-Typs ist **höchstens <u>ein</u> Entitiy** des anderen Entity-Typs zugeordnet, **<u>nicht</u> aber umgekehrt** → hierarchischer Beziehungstyp

- `Beispiele: [Bezirk] ---1--- <bewohnt von> ---n--- [Niederösterreicher]`
- `Beispiele: [Schüler] ---n--- <besucht> ---1--- [Schule]`

In einer 1:n-Beziehung muss die mathematische Relation **<u>eine Funktion</u>** sein.

Die Entities des **einen Entity-Typs** kommen in den Paaren der Relation höchstens **<u>einmal</u>** vor, die des **anderen Entity-Typs** kommen **<u>mehrmals</u>** vor.

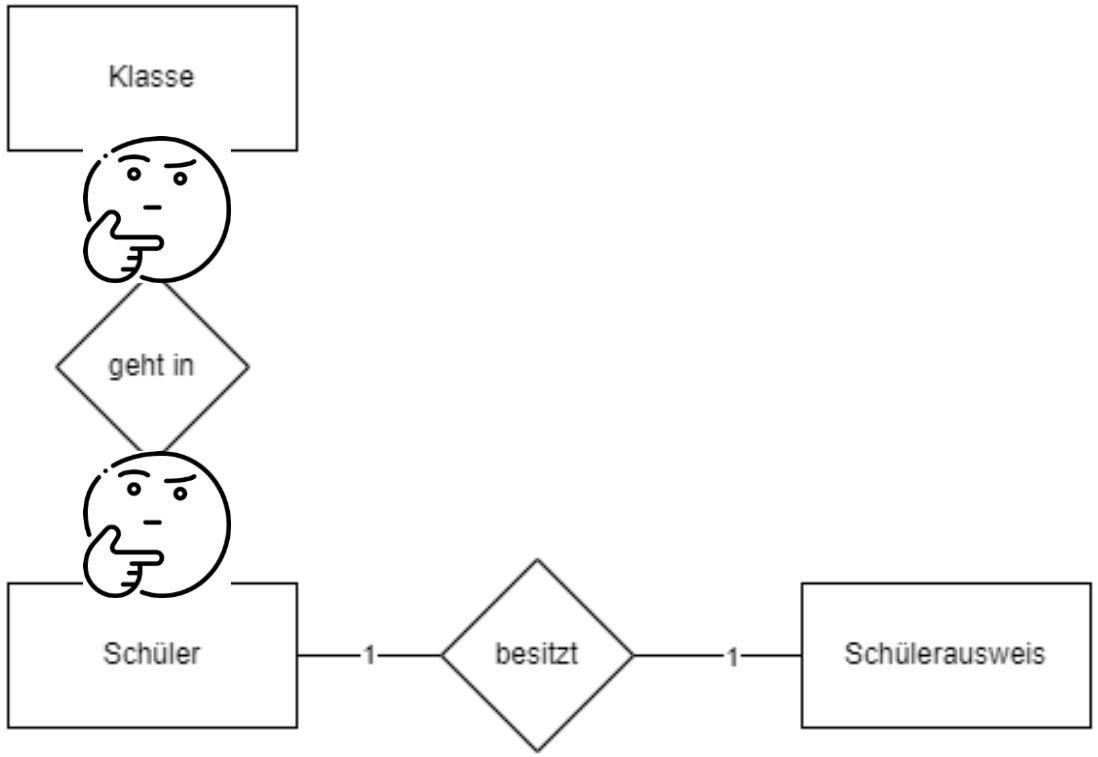


R = { (Maier,HTL Linz), (Huber,HTL Wien), (Berger,HTL Linz), (Schmid,HTL Wien) }

ÜBUNG: KARDINALITÄT – 1:N BEZIEHUNGSTYP

Beispiel: Die Beziehung zwischen Schüler und Klasse

- 1 Schüler geht in 1 Klasse
- 1 Klasse hat mehrere (n) Schüler
- Wichtig auf welcher Seite das „n“ steht



ÜBUNG: KARDINALITÄT – 1:N BEZIEHUNGSTYP

Welche Aussage zu der Beziehung zwischen einem Museum und dessen Kunstwerke ist korrekt?

☐ [Kunstwerk]-1-<ausgestellt in>-n-[Museum]

☒ [Kunstwerk]-n-<ausgestellt in>-1-[Museum]

Antwort:

- 1 [Kunstwerk] ist ausgestellt in 1 [Museum]
- 1 [Museum] kann mehrere [Kunstwerke] ausstellen

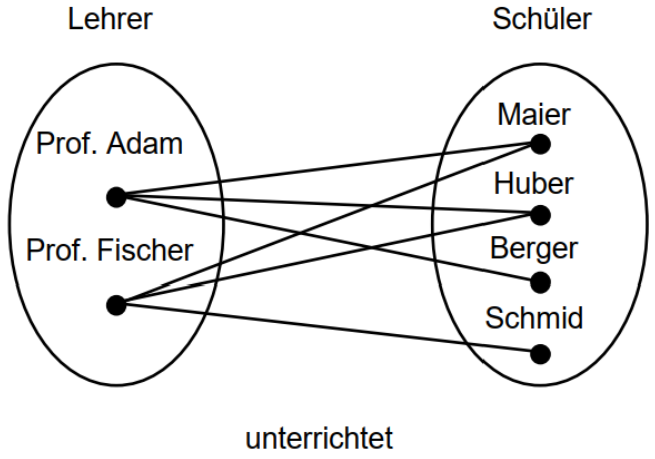
KARDINALITÄT – N:M BEZIEHUNGSTYP

Keine Einschränkungen in der Zuordnung zwischen den Entities der beiden Entity-Typen.

- `Beispiel:[Lehrer] ---m--- <unterrichtet> ---n--- [Schüler]`
- `Beispiel: [Bauteil] ---m--- <wird geliefert von> ---n--- [Lieferant]`

In einer m:n-Beziehung muss die mathematische Relation **keinen Bedingungen** genügen.

Die Entities der **beiden Entity-Typen** kommen in den Paaren der Relation **mehrmals** vor.

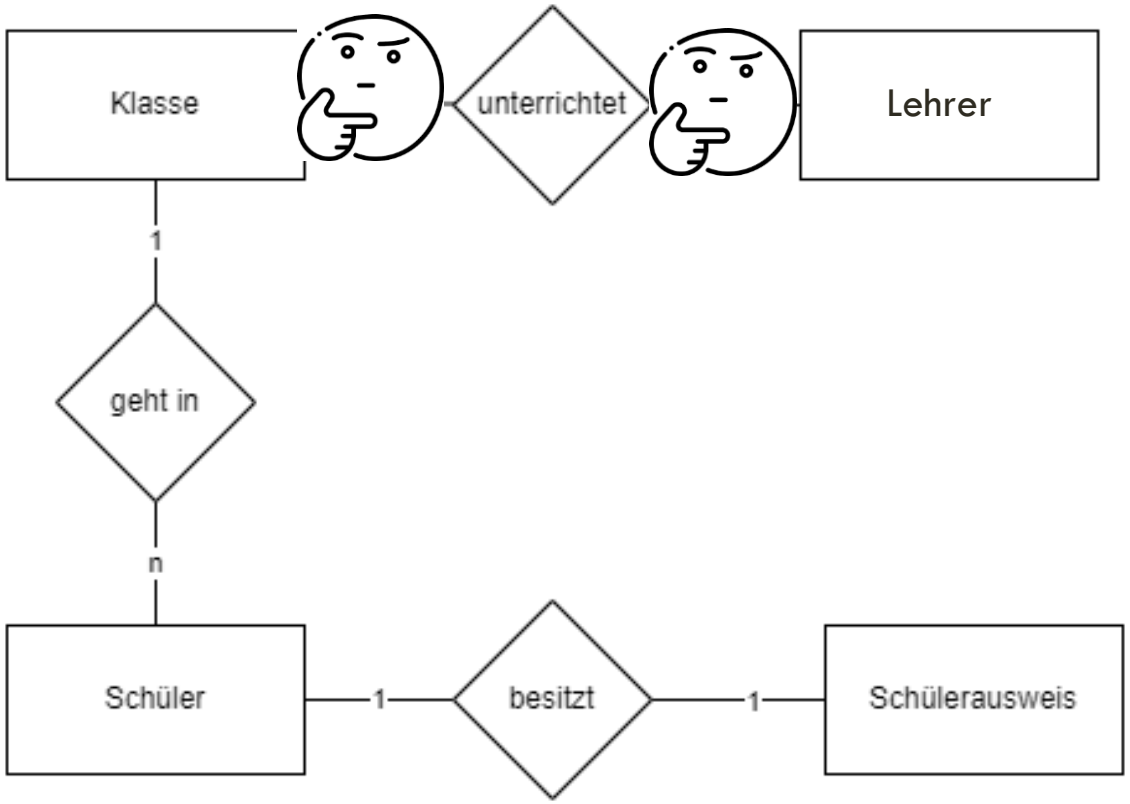


R = { (Prof. Adam,Maier), (Prof. Adam,Huber), (Prof. Adam,Berger),
(Prof. Fischer,Maier), (Prof. Fischer,Huber), (Prof. Fischer,Schmid) }

ÜBUNG: KARDINALITÄT – N:M BEZIEHUNGSTYP

Beispiel: Die Beziehung zwischen Lehrer und Klasse

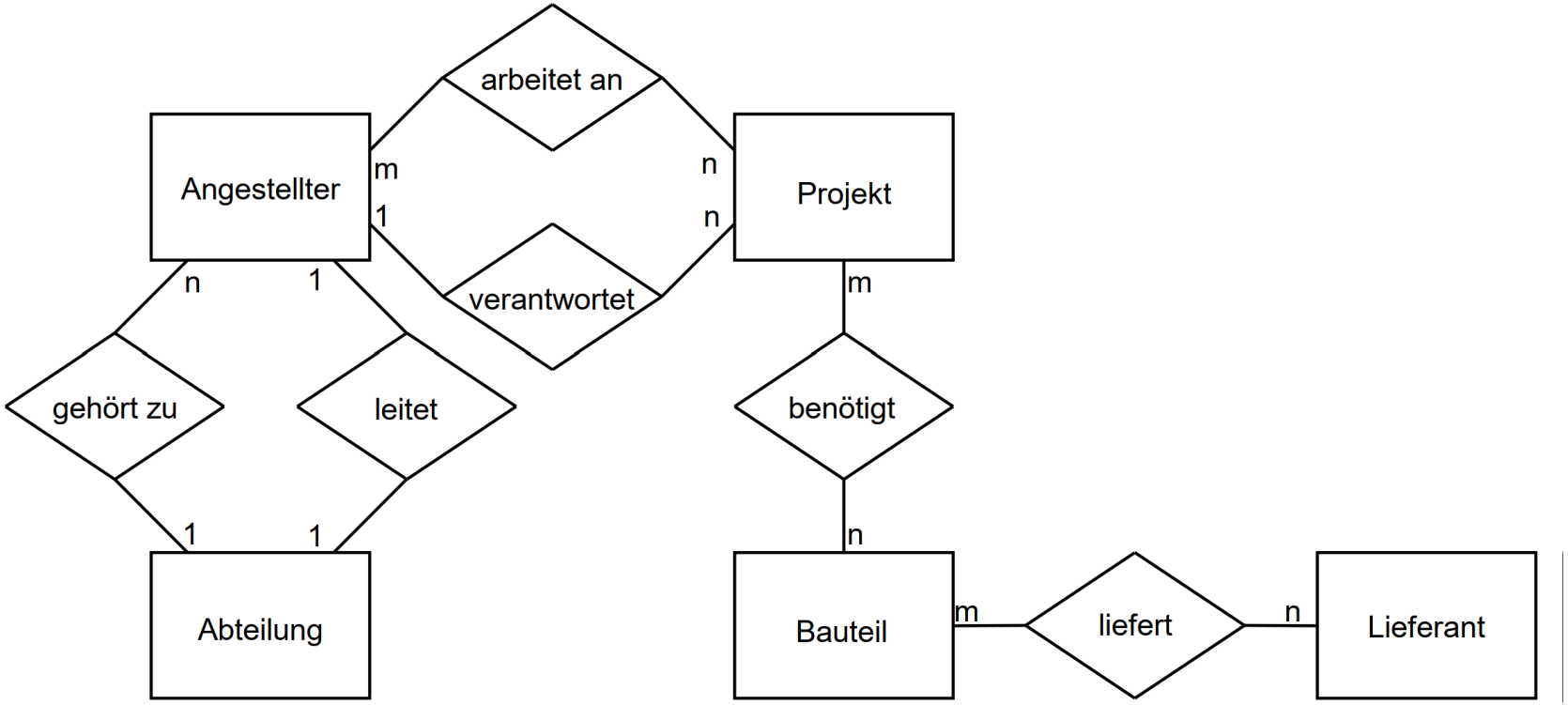
- Mehrere Lehrer unterrichten mehrere Klassen
- Mehrere Klassen werden von mehreren Lehrern unterrichtet



ÜBUNG 1: KARDINALITÄT

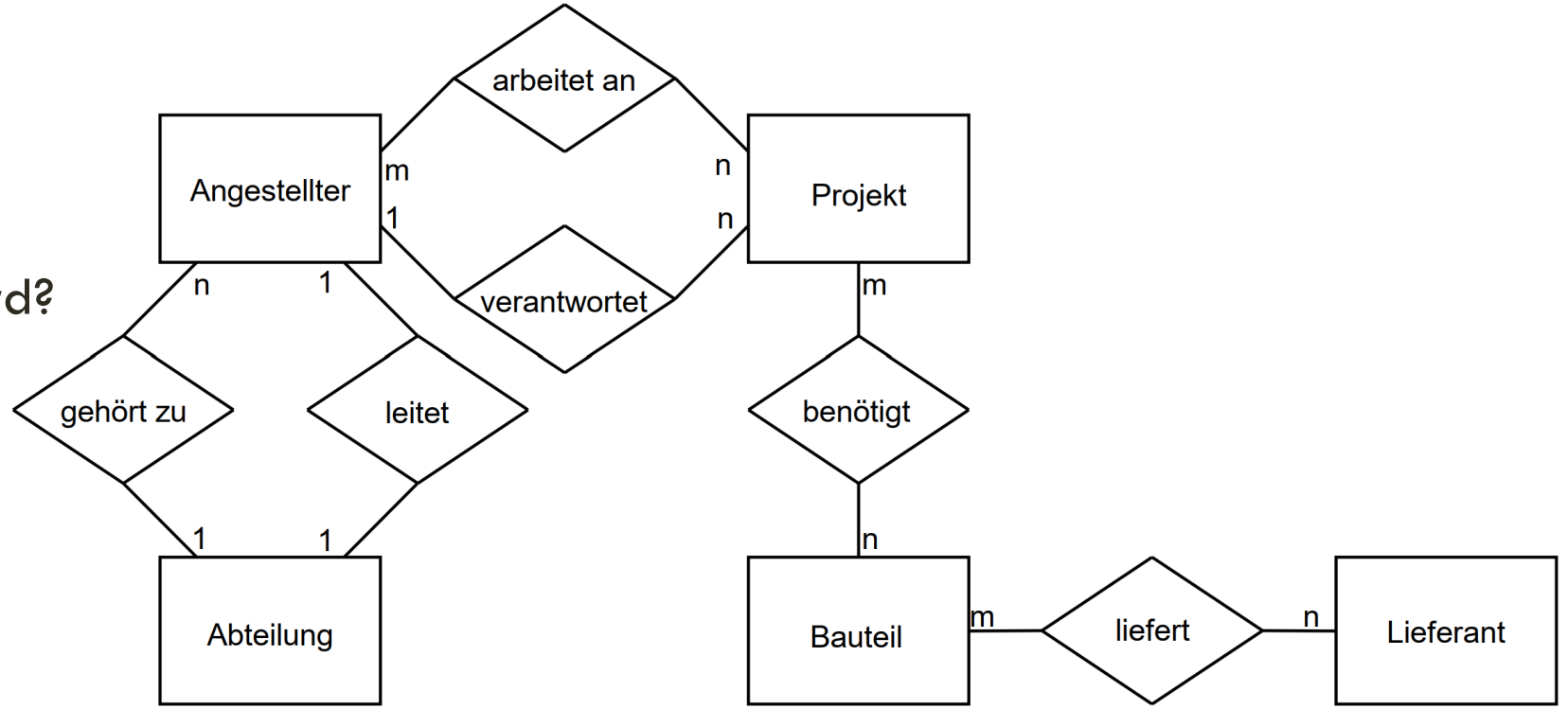
Welche Komplexitätsgrade sind Ihrer Meinung nach für das folgende ERD sinnvoll?

- Angestellter wird einer Abteilung zugewiesen
- Angestellter kann an mehreren Projekten arbeiten



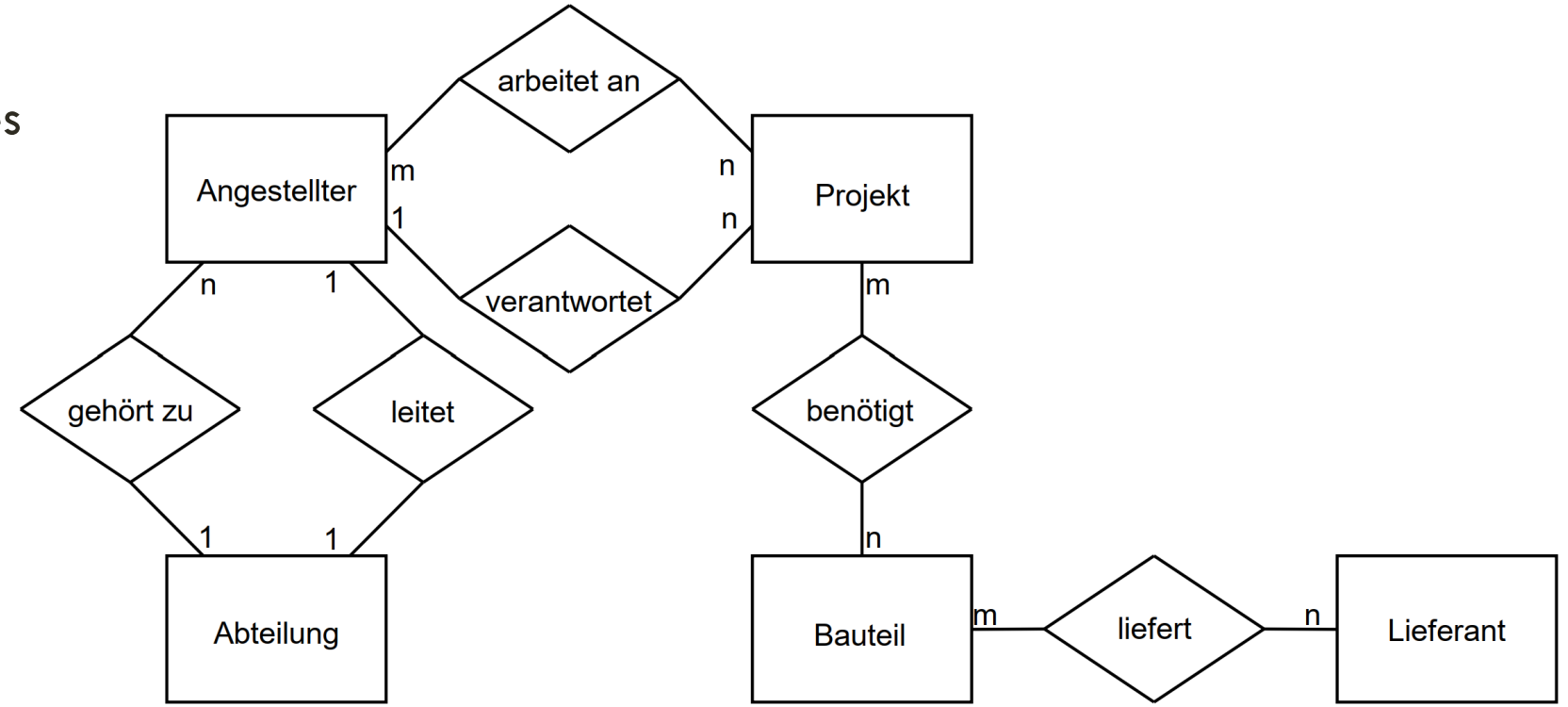
ÜBUNG 2: KARDINALITÄT

Wie ändern sich die Komplexitätsgrade, wenn Bauteil nicht im Sinne von Bauteil-Art, sondern als Einzelbauteil aufgefasst wird?



ÜBUNG 3: KARDINALITÄT

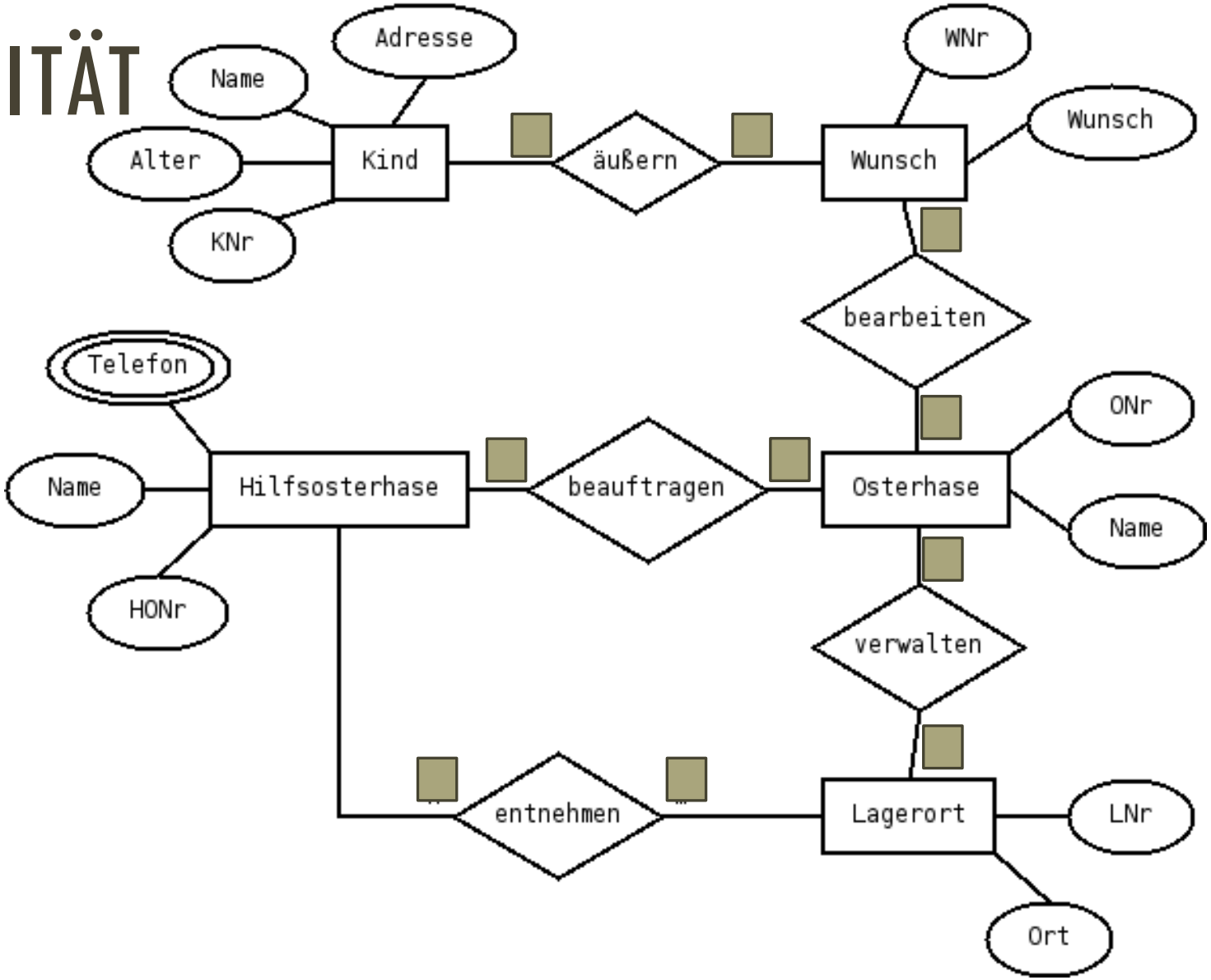
Wie ändern sich die Komplexitätsgrade, wenn es zu jedem Bauteil nur einen (Stamm-)Lieferant gibt?



ÜBUNG 4: KARDINALITÄT

Überlegen und begründen Sie Ihre Wahl der Kardinalitäten für das gegebene Beispiel

- Ein Wunsch beschreibt ein konkretes physisches Geschenk
- Es existiert ein Osterhase, der mehrere Hilfsosterhasen beauftragen kann



(MIN, MAX)-NOTATION

Genauere Charakterisierung des Beziehungs-Typs durch **ein Zahlenpaar** (min,max):

- Jedes Entity des Entity-Typs ist **mindestens an min** und **höchstens an max** Beziehungen des Beziehungs-Typs beteiligt.
- Es muss gelten: $0 \leq min \leq max$
- Es muss gelten: $max \geq 1$
- Ein beliebig großes max wird mit **n oder *** symbolisiert. (n ist zu bevorzugen!)
- Bei einigen Notationsformen sind nur die Werte 0,1 und n für min und max erlaubt.

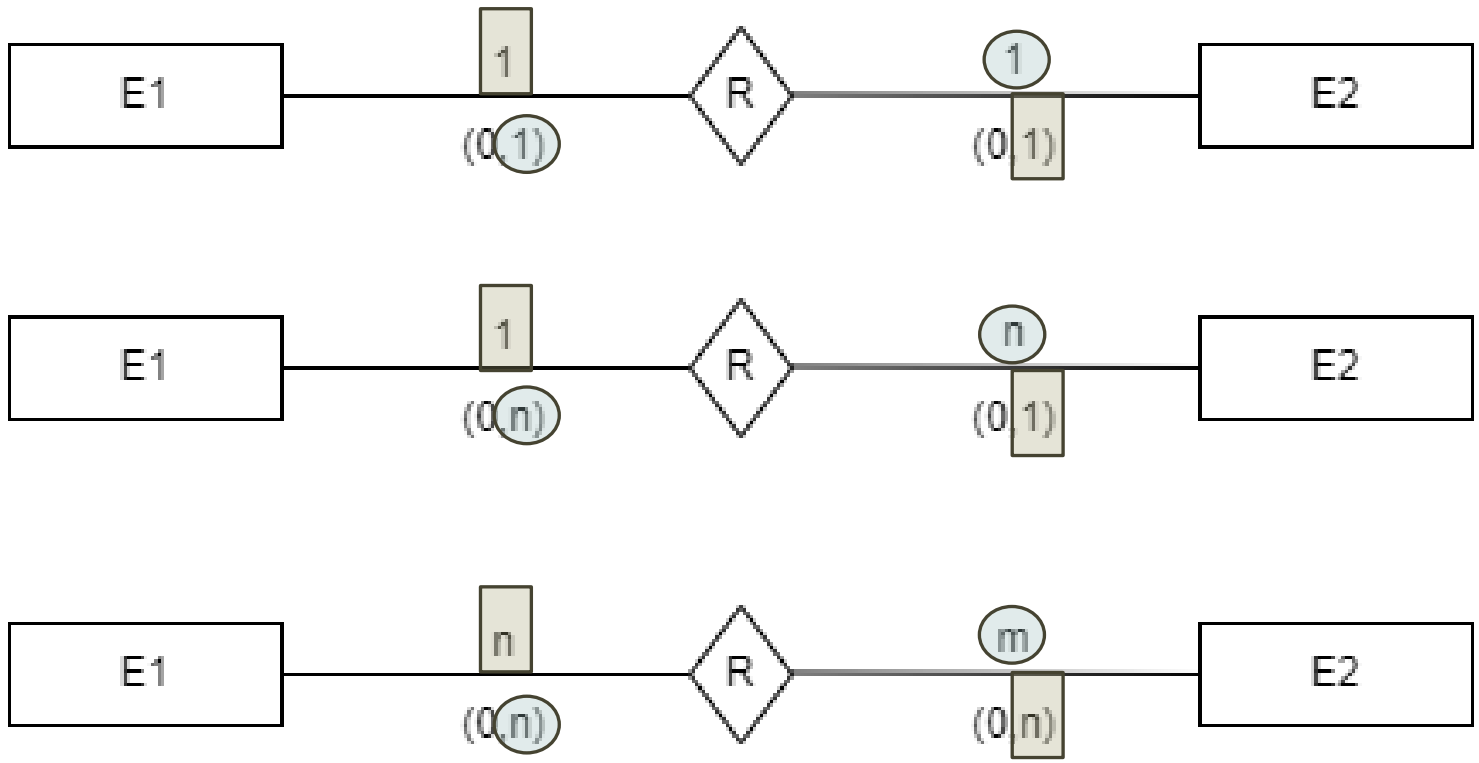


(MIN, MAX)-NOTATION VS. CHEN-NOTATION

Kardinalität der Chen Notation ist entspricht dem max-Wert der (min,max)-Notation

Leserichtung:

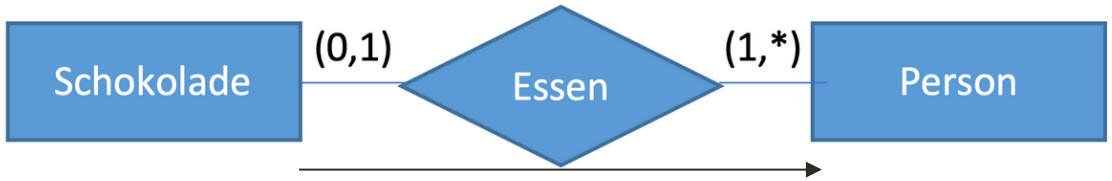
- (min, max)-Notation liest von E1 weg zu E2
- Chen-Notation liest von E2 hin zu E1



https://www.youtube.com/watch?v=oxcaqMQdGyw

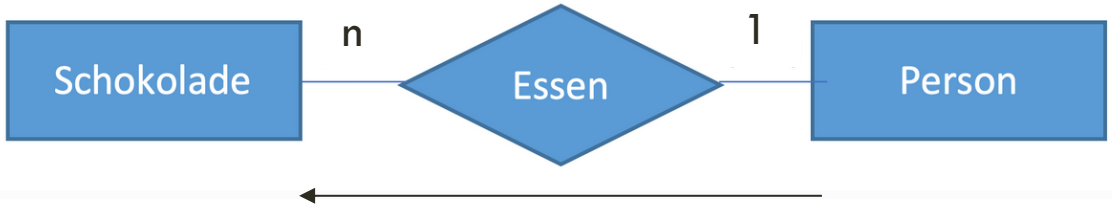
(MIN, MAX)-NOTATION – LESERICHTUNG

Interpretation ausgehend dem Entity-Typ Schokolade



Leserichtung: wird vom Entity-Typ zu den anderen Entity-Typen gelesen

- Schokolade kann von keiner Person gegessen werden
- Schokolade kann maximal von einer Person gegessen werden



Leserichtung: wird vom anderen Entity-Typ zum konkreten Entity-Typ gelesen

- Schokolade wird von einer Person gegessen

(MIN, MAX)-NOTATION

Welche Aussage ist für das gegebene ERD korrekt?

- [x] Eine Person muss mindestens eine Schokolade essen
- [ ] Eine Schokolade kann von beliebig vielen Personen gegessen werden
- [x] Eine Schokolade kann von maximal einer Person gegessen werden
- [x] Eine Schokolade muss nicht gegessen werden
- [ ] Eine Person kann keine oder beliebig viele Schokoladen essen

Welche Aussagen treffen auf das folgende ER-Modell zu?



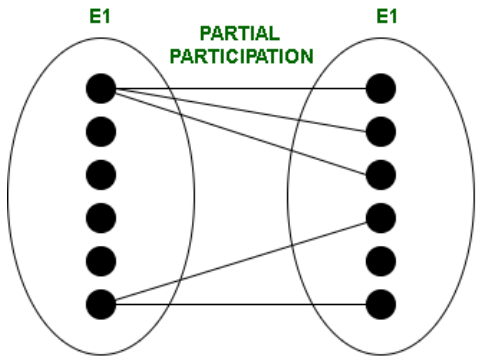
(MIN, MAX)-NOTATION

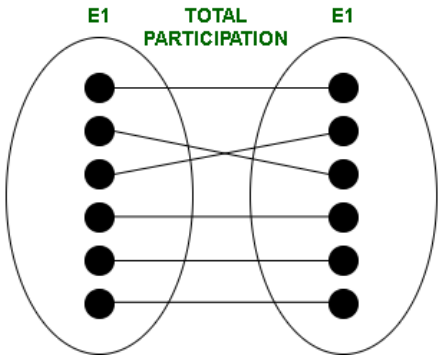
Konditionelle Beziehungs-Typen (Kann-Beziehungs-Typen, Partial Relation-Type)

- enthalten als min=0

Teilnahme an Beziehungs-Typ:

- **Partial oder Optional Participation**: Ein Entity-Typ mit Beziehungs-Typ **min=0** nimmt an diesem Beziehungs-Typ **teilweise** teil
- **Total oder Mandatory Participation**: Ein Entity-Typ mit Beziehungs-Typ **min>0** nimmt an diesem Beziehungs-Typ **vollständig** teil





ÜBUNG: (MIN, MAX)-NOTATION

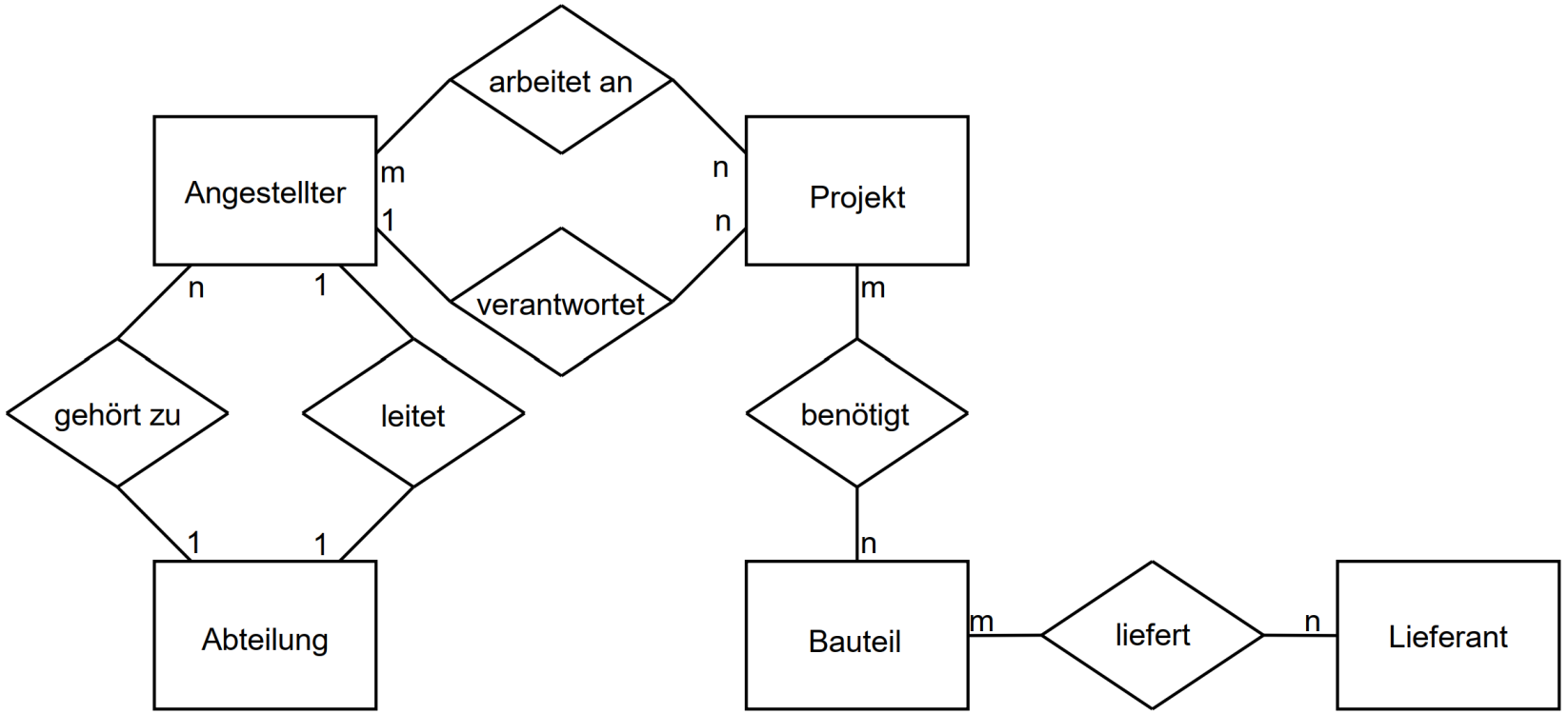
Beispiele:

- [Angestellter] ---(min,max)--- <leitet> ---(min,max)--- [Abteilung]
- [Mann] ---(min,max)--- <ist verheiratet mit> ---(min,max)--- [Frau]
- [Person] ---(min,max)--- <besitzt> ---(min,max)--- [Österreichischer Führerschein]
- [Mitarbeiter] ---(min,max)--- <hat> ---(min,max)--- [Firmenauto]
- [Bezirk] ---(min,max)--- <bewohnt von> ---(min,max)--- [Niederösterreicher]
- [Schüler] ---(min,max)--- <besucht> ---(min,max)--- [Schule]
- [Lehrer] ---(min,max)--- <unterrichtet> ---(min,max)--- [Schüler]
- [Bauteil] ---(min,max)--- <wird geliefert von> ---(min,max)--- [Lieferant]

  (es gibt auch eigengefertigte Bauteile)

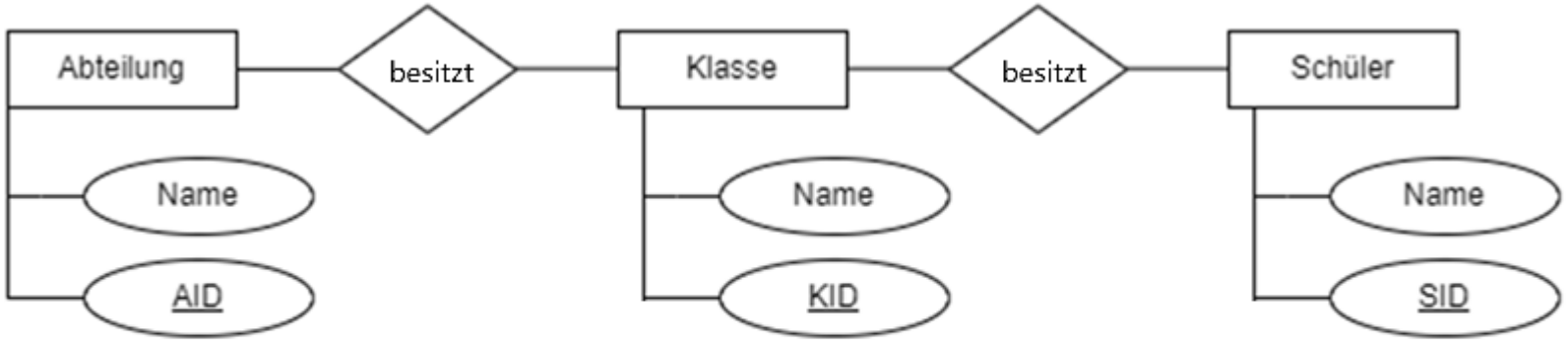
ÜBUNG 1: (MIN, MAX)-NOTATION

Erweitern Sie das folgende ERD um die (min, max)-Notation.



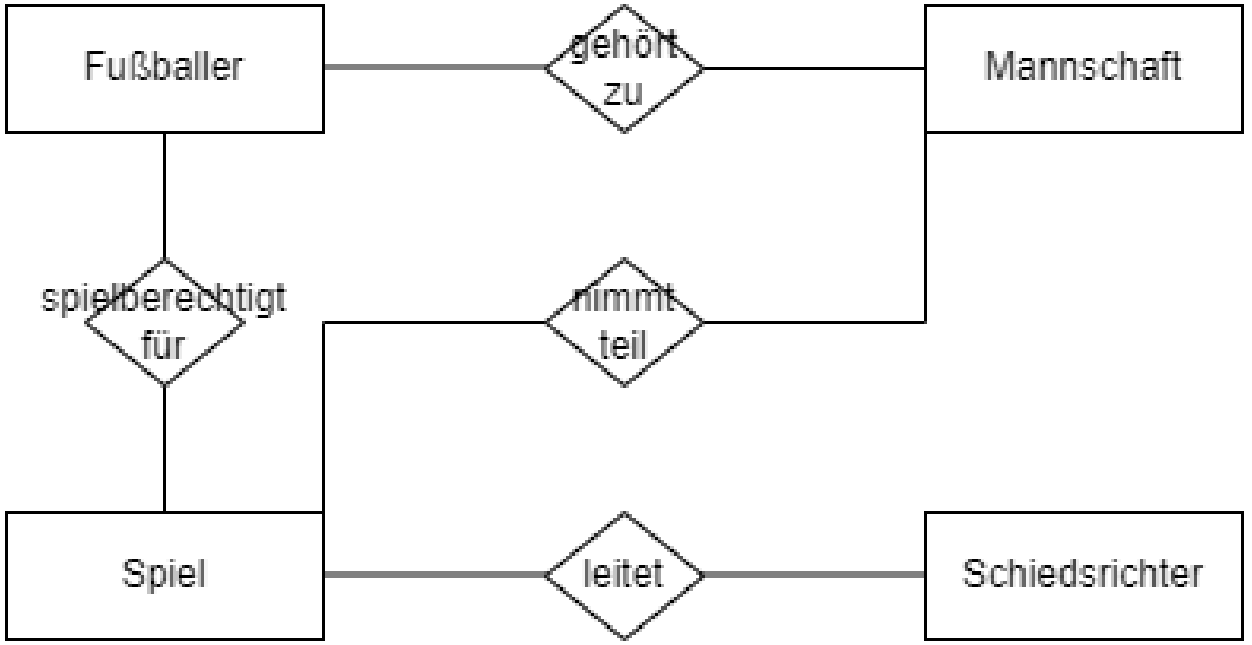
ÜBUNG 2: SCHULE MODELLIEREN

Wie definieren Sie die (min,max) Notation in dem nachfolgenden ERD?



ÜBUNG 3: (MIN, MAX)-NOTATION

Erweitern Sie das folgende ERD um die (min, max)- Notation.



EXKURS: ABLEITEN IN TABELLEN

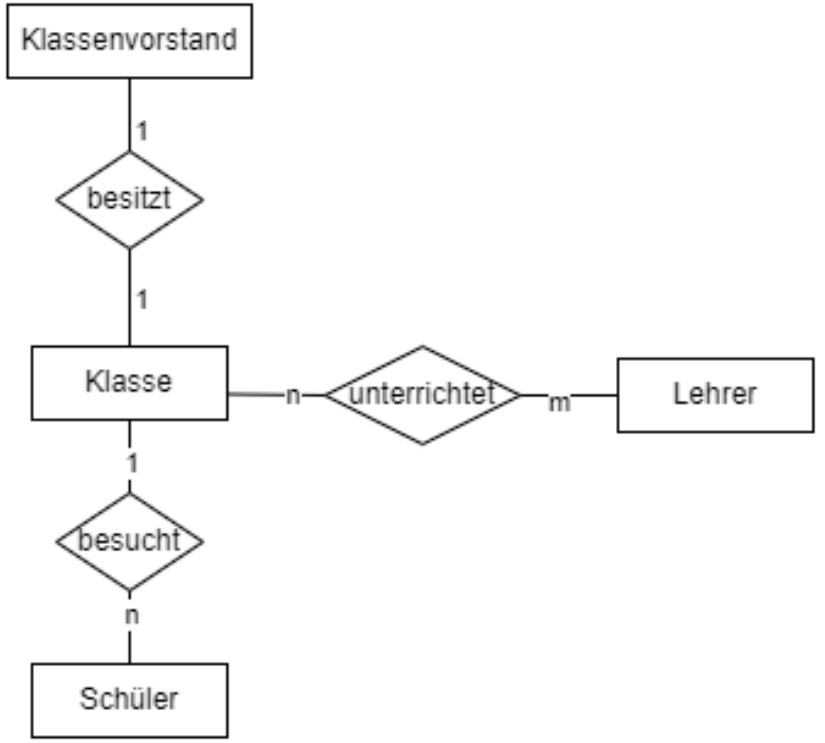
Folgende Informationen müssen transformiert werden (Details folgen später):

- Entity-Typ mit Attributen
- Beziehungstypen (1:1, 1:n, n:m)
- Definition von Primary Key und Foreign Key

Aufpassen auf Reihenfolge!

Ableitung:

- Klassenvorstand(<u>KlassenvorstandID</u>, Name)
- Lehrer(<u>LehrerID</u>, Name)
- Klasse(<u>KlasseID</u>, Name, *KlassenvorstandID*)
- Schüler(<u>SchülerID</u>, Name, *KlasseID*)
- Unterricht(<u>*LehrerID, KlasseID*</u>)



KONZEPTIONELLES DATENDESIGN

Kapitel 2

ÜBERSICHT

Einführung Modellierung

- Phasenmodell, Begriffe: Entity, Attribut, Entity-Typ

Vom konventionellen Datendesign zum Konzept – Übersicht

- ERD – Übersicht, Kardinalität, (min, max) Notation

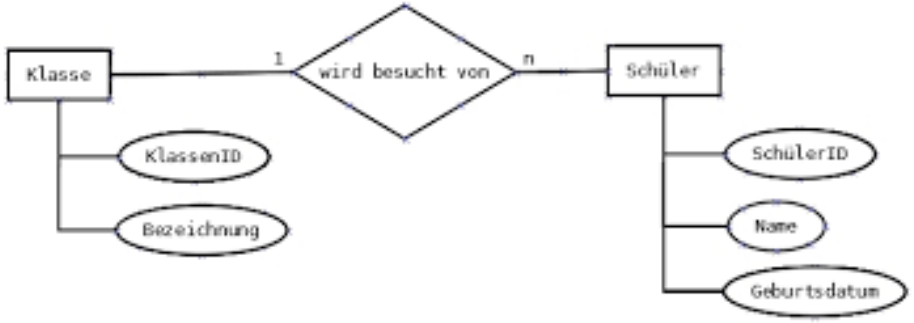
Vom konventionellen Datendesign zum Konzept – Details

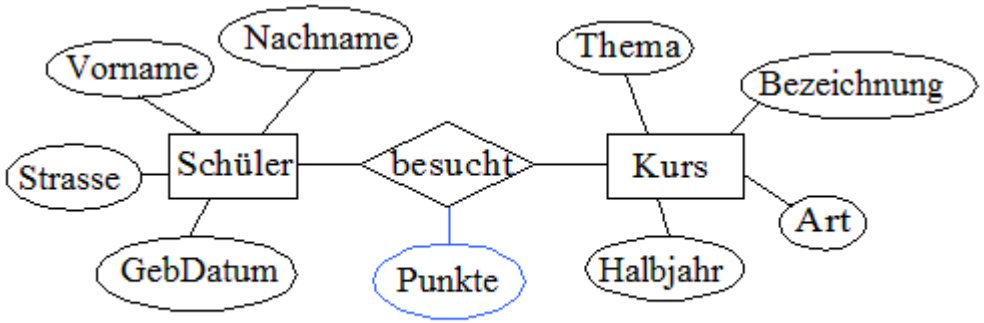
- Beziehungen mit Attributen, Unterschiede zwischen Beziehungs-Typ und

ERD – KOMPONENTEN

Mögliche Komponenten eines ERDs (welche davon kennen Sie bereits):

- Beziehung vs. Beziehungs-Typ
- Kardinalität
- (min,max)-Notation
- Attribute
- Beziehungs-Typ vs. Entity-Typ
- Attribute vs. Entity-Typ
- Rekursive Beziehungstypen
- Mehrwertige Beziehungs-Typen



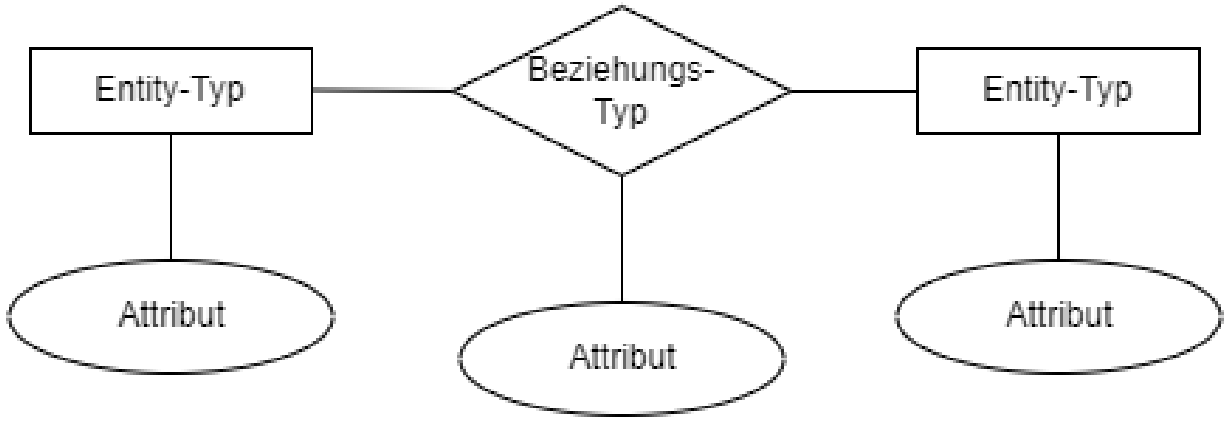


BEZIEHUNGEN MIT ATTRIBUTEN

Nicht nur Entity-Typen können Attribute enthalten auch Beziehungs-Typen können Attribute enthalten

- Wie die beste Wahl für das Datenmodell wählen?

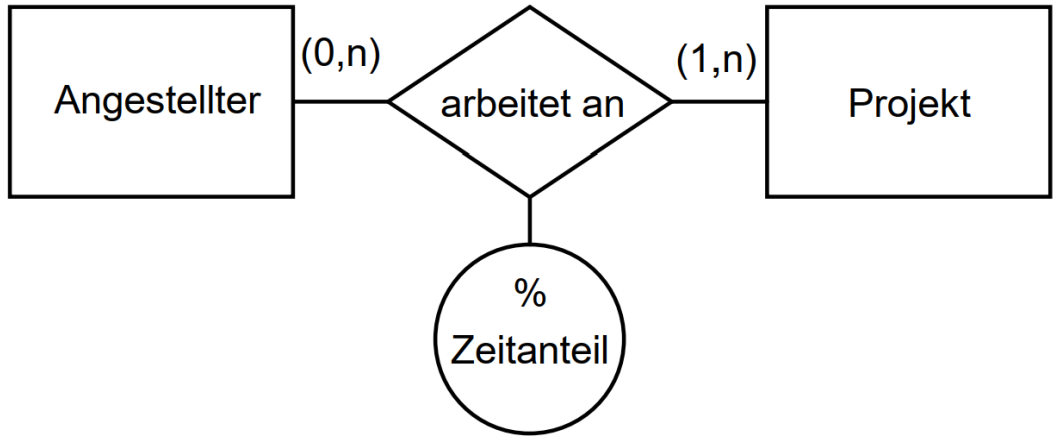
Diskussion: Fällt Ihnen ein Beispiel ein bei dem ein Beziehungs-Typ ein Attribut besitzt?



ÜBUNG 1: BEZIEHUNGEN MIT ATTRIBUTEN

Angestellte arbeiten einen bestimmten Zeitanteil an diesem Projekt mit (Randbedingung: Summe der Zeitanteile eines Angestellten ≤ 100)

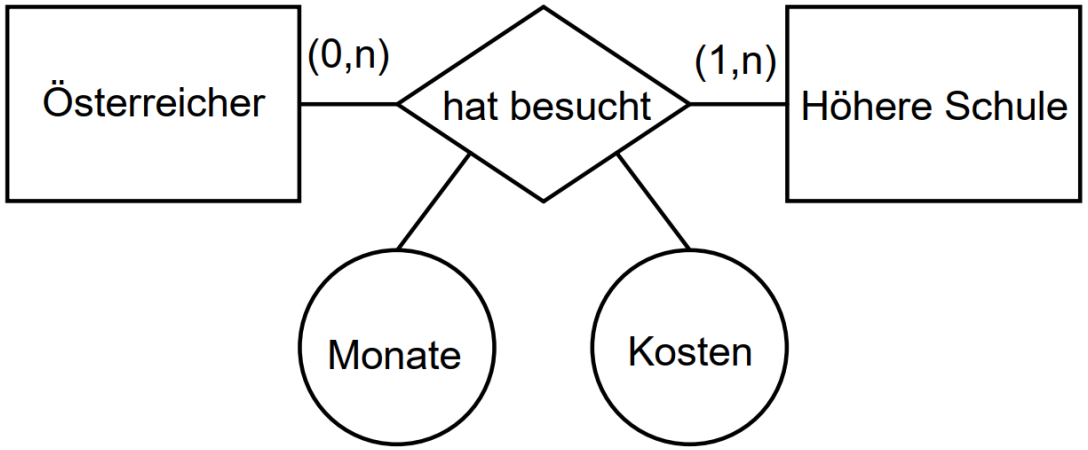
- Welche Entity-Typen sind sinnvoll?
- Welche Beziehungs-Typen sind sinnvoll?
- Welche Attribute sind sinnvoll?
- Definieren Sie die (min, max) Notation



ÜBUNG 2: BEZIEHUNGEN MIT ATTRIBUTEN

Österreicher besuchen die Höhere Schule eine bestimmte Anzahl von Monaten und verursachen dabei bestimmte Kosten.

- Welche Entity-Typen sind sinnvoll?
- Welche Beziehungs-Typen sind sinnvoll?
- Welche Attribute sind sinnvoll?
- Definieren Sie die (min, max) Notation

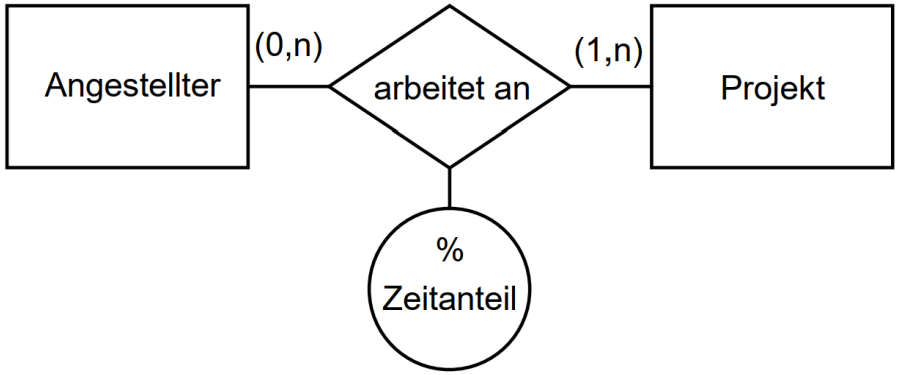


ATTRIBUT BEI BEZIEHUNGS-TYP ODER ENTITY-TYP?

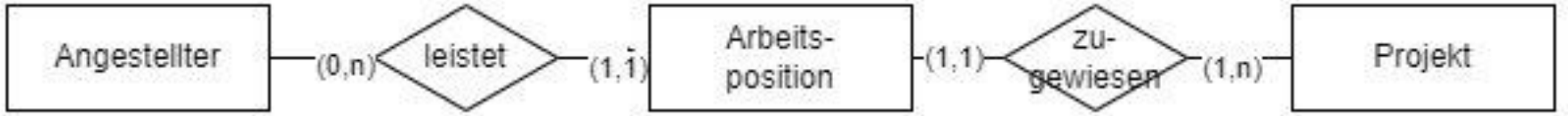
Diskussion: Was halten Sie von den beiden Varianten zu Übung 1?

- Beschreibe Sie denselben Sachverhalt?
- Kann man frei wählen, was man nimmt?

Variante 1:



Variante 2:



ATTRIBUT BEI BEZIEHUNGS-TYP ODER ENTITY-TYP?

Ein Sachverhalt kann

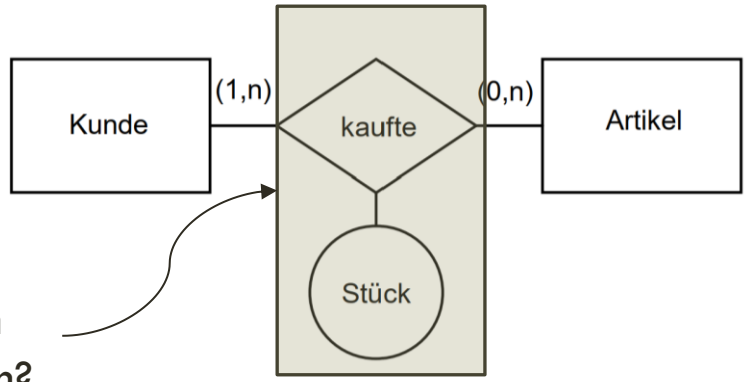
- durch **eine Beziehung** beschrieben werden
- durch **ein Entity** beschrieben werden → besonders bei Beziehungen mit Attributen

Umwandlung einer **Beziehung** zu einem **Entity-Typ** ist möglich

- Die verbleibenden Beziehungs-Typen haben keine Attribute mehr.

Wie die „richtige" Variante auswählen?

- Aufgrund der Aufgabenstellung
- Teilweise willkürliche Festlegung



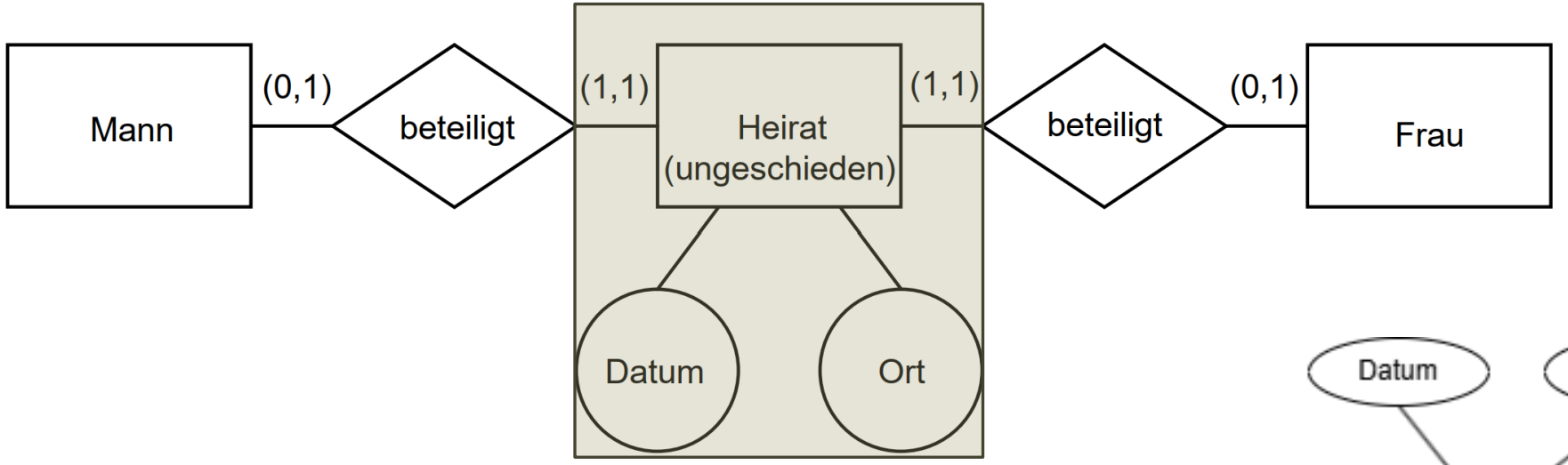
Als Beziehungs-Typ bei belassen oder eigenen Entity-Typ erstellen?



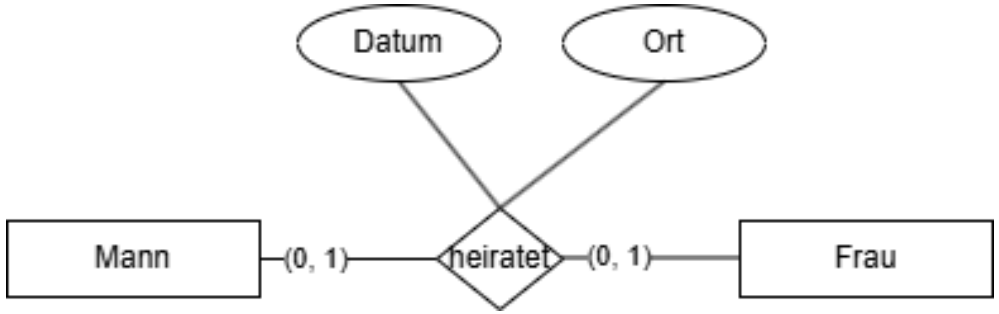
ÜBUNG 1: BEZIEHUNGS-TYP ODER ENTITY-TYP?

Mann und Frau heiraten an einem bestimmten Datum, an einem bestimmten Ort.

- Wandeln Sie das Beispiel in einen Beziehungs-Typ mit Attributen um



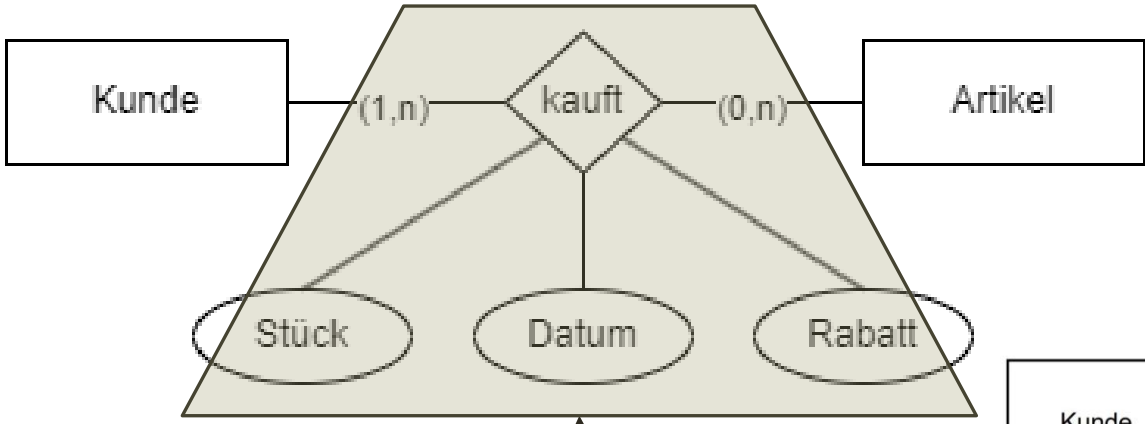
Als Entity-Typ oder Beziehungs-Typ?



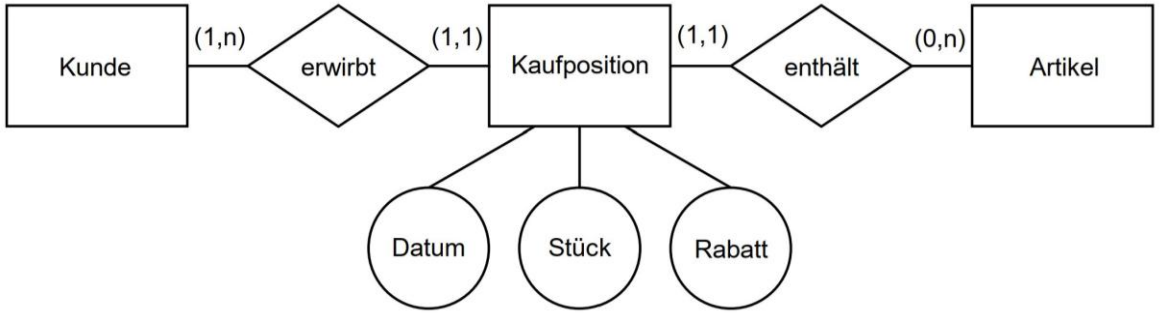
ÜBUNG 2: BEZIEHUNGS-TYP ODER ENTITY-TYP?

Der Kunde hat an einem bestimmten Datum so viel Stück des Artikels mit einem Rabatt gekauft.

- Wandeln Sie das Beispiel in einen Entity-Typ mit Attributen um



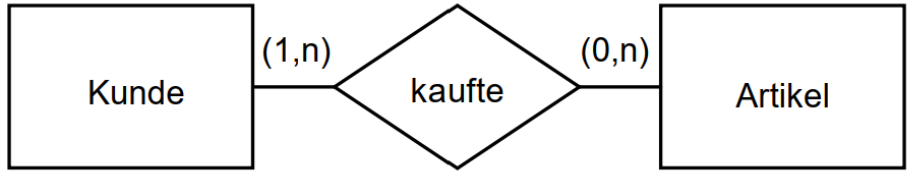
Als Entity-Typ oder Beziehungs-Typ?

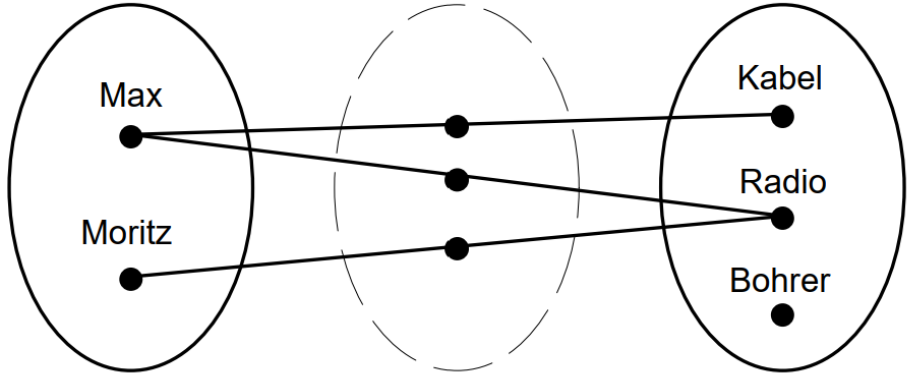


BEZIEHUNGS-TYP ODER ENTITY-TYP?

m:n Beziehungs-Typen können in **zwei 1:n Beziehungs-Typen** aufgelöst werden

- Sinnvolle Bezeichnungen für Entity-Typ und für beide 1:n-Beziehungs-Typen
- Solche Entity-Typen werden auch als **Assoziativer Entity-Typ** (Associative Entity-Type, Intersection Entity-Type) bezeichnet.





(min,max)-Werte

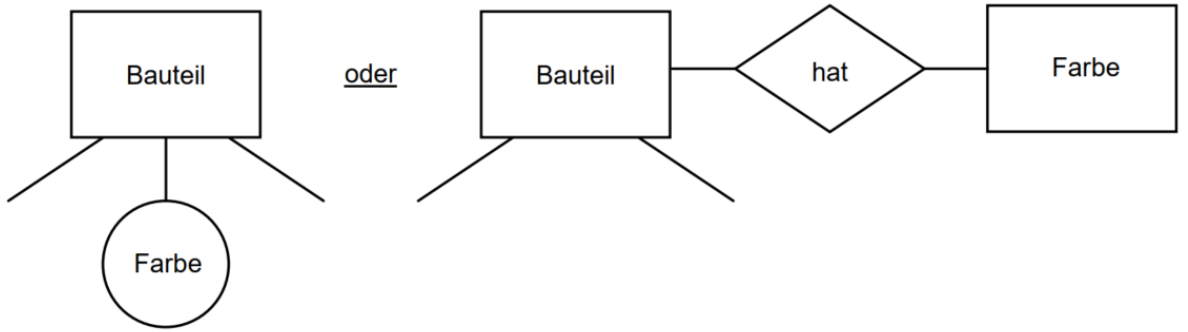
- der **ursprünglichen** Entity-Typen **bleiben gleich**
- die zwei **neuen** sind immer **(1,1)**



ATTRIBUT ODER ENTITY-TYP?

Wenn möglich ist ein **Entity-Typ** einem Attribut **vorziehen**

Beispiel: Die Farbe eines Bauteils kann als Attribut oder als Entity-Typ angeben



In welchen Fällen ist ein Entity-Typ vorzuziehen?

- Bauteile haben mehrere Farben
- Farben werden noch zusätzlich beschrieben (weitere Eigenschaften)
- Mögliche Werte (des Attributs) sollen auf bestimmte Begriffe eingeschränkt werden (z.B. Schlagworte, Wissensgebiete)



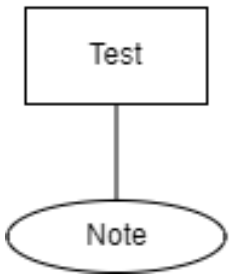
ÜBUNG 1: ATTRIBUT ODER ENTITY-TYP?

Sie möchten die Noten von Tests abspeichern

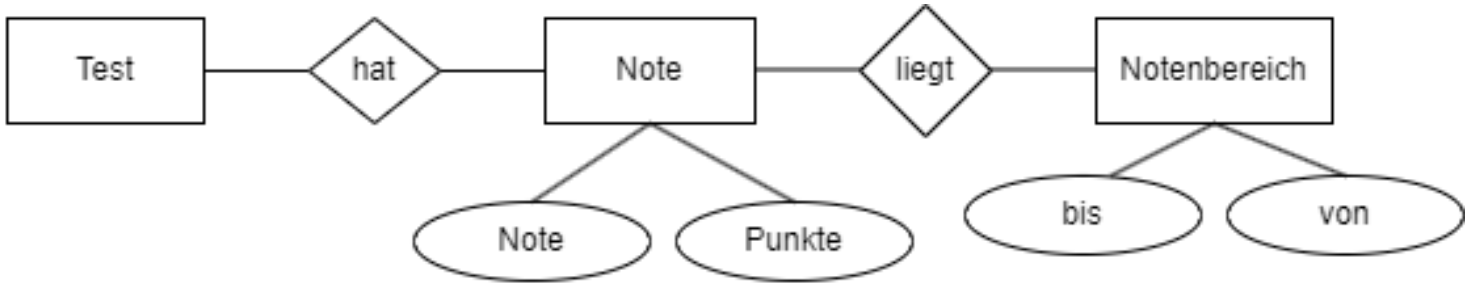
- Frage 1: Welche Entity-Typen bzw. Attribute benötigen Sie dafür?
- Frage 2: Begründen Sie, warum Sie sich für Ihr Modell entschieden haben!

Was fällt Ihnen bei den nachfolgenden Varianten auf?

Variante 1:



Variante 2:

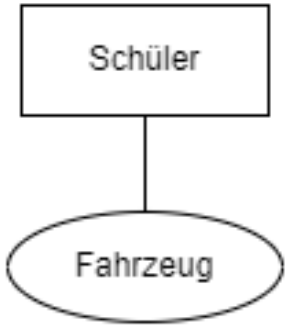


ÜBUNG 2: ATTRIBUT ODER ENTITY-TYP?

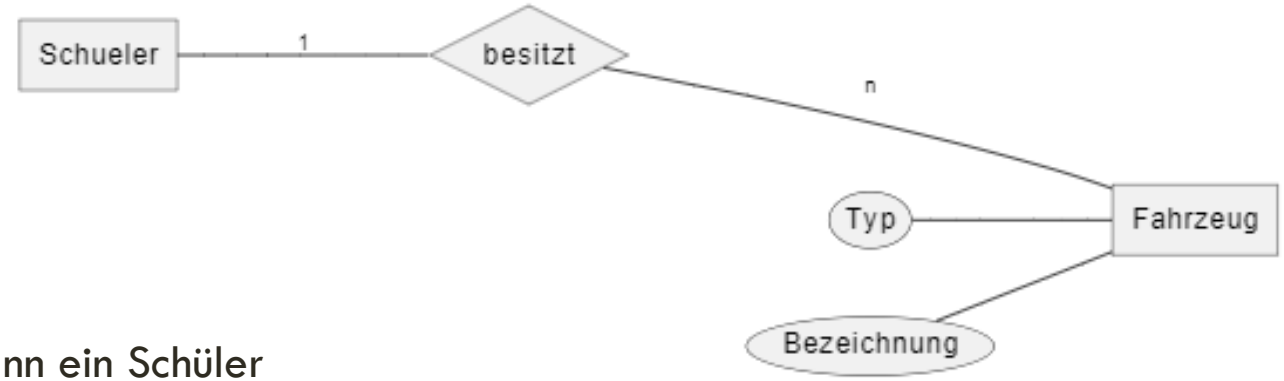
Sie möchten die Fahrzeuge von Schüler modellieren

- Frage: Welche Variante ist Ihrer Meinung nach sinnvoller bzw. was sind die Vor- bzw. Nachteile

Variante 1:



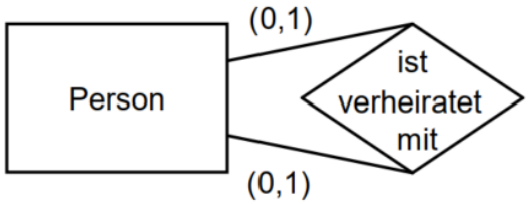
Variante 2:

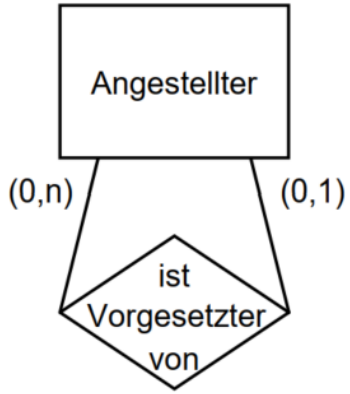


Bei Variante 2 kann ein Schüler mehrere Autos besitzen bei Variante 1 nur ein Auto

REKURSIVE BEZIEHUNGEN

Rekursive oder Reflexive Beziehungen **beschreiben Beziehungen zwischen Entities <u>desselben</u> Typs**





- 0 in (0,n) weil nicht jeder Angestellte Vorgesetzter ist ('Die Hackler')
- 0 in (0,1) weil es zumindest einen Angestellten geben wird, der keinen Vorgesetzten hat ('Der Big Boss')

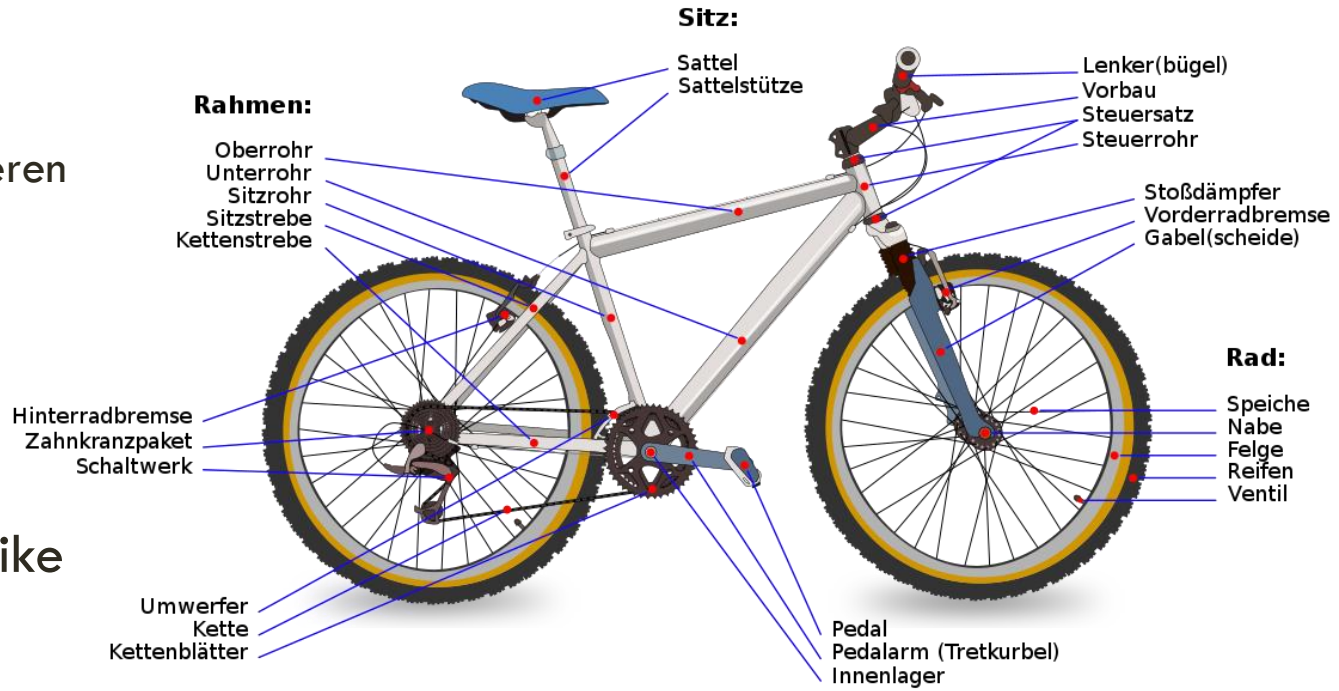
ÜBUNG 1: REKURSIVE BEZIEHUNGEN MIT ATTRIBUTEN

Aufgabe: Bauteile eines Fahrrads sollen in einer Tabelle abgebildet werden:

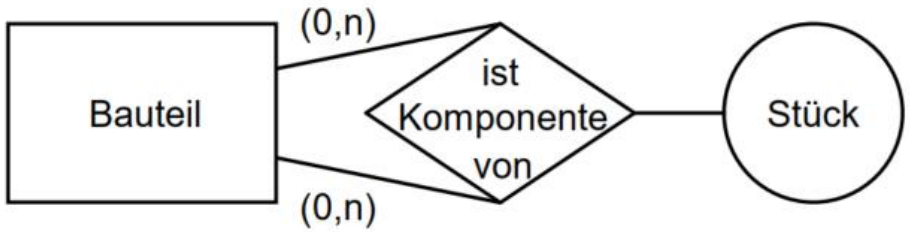
- Alle Bauteile können, müssen aber nicht, von anderen Bauteilen Komponenten darstellen
- Die Anzahl der Bauteile, die in dem gesamten Fahrrad verbaut sind, sollen ebenfalls angeführt werden

Frage 1: Was sind mögliche Beispiele eines Bauteiles, welches für ein Mountainbike verbaut werden muss?

Frage 2: Welche Tabellen werden für dieses Beispiel benötigt?



ÜBUNG 1: REKURSIVE BEZIEHUNGEN MIT ATTRIBUTEN



Konkretes Beispiel:

- { (B3,B1,1), (B4,B1,1), (B4,B2,1), (B5,B1,2), (B6,B5,1), (B7,B5,1), (B8,B5,1), (B9,B5,24), (B10,B5,1), ...}
- 0 in (0,n) oben: Bauteil kommt in keinem anderen Bauteil vor ('Fahrrad Type 1')
- n in (0,n) oben: Bauteil kommt in mehreren anderen Bauteilen vor ('Lenker')
- 0 in (0,n) unten: Bauteil besteht aus keinen weiteren Bauteilen mehr ('Speiche')
- n in (0,n) unten: Bauteil besteht aus mehreren anderen Bauteilen ('Rad')

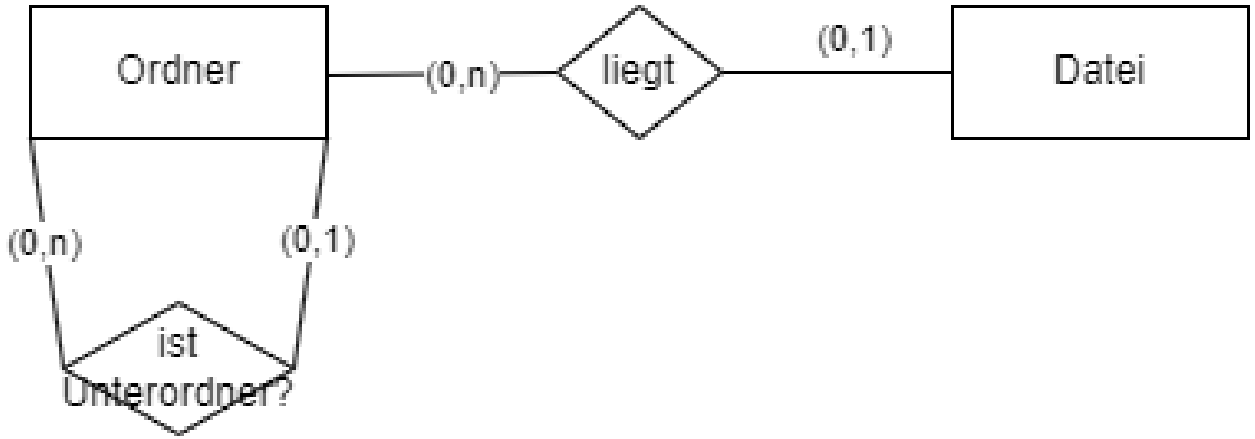
Komponenten:

- B1=Fahrrad Type 1,
- B2= Fahrrad Type 2,
- B3=Rahmen,
- B4=Lenker,
- B5=Rad,
- B6=Mantel,
- B7=Schlauch,
- B8= Felge,
- B9=Speiche,
- B10=Nabe,...

ÜBUNG 2: REKURSIVE BEZIEHUNGEN MIT ATTRIBUTEN

Bilden Sie Ordner und Dateien in einem ERD sinnvoll ab

- Welche Entity-Typen und Attribute werden benötigt?
- Definieren Sie die (min,max) Notation



MEHRWERTIGE BEZIEHUNGS-TYPEN

Grad, Wertigkeit oder Beziehung: Die Anzahl der beteiligten Entities an einer Beziehung

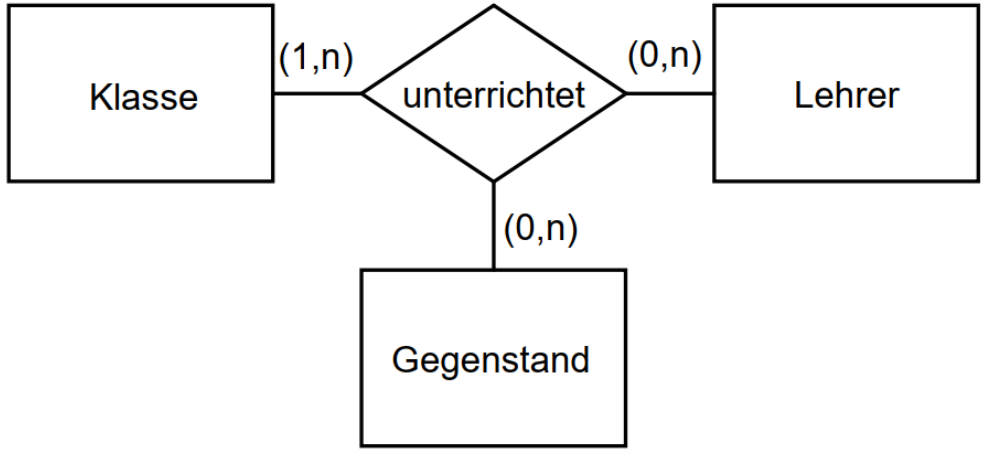
- Grad = 2 – zweiwertige oder binäre Beziehungs-Typen
- Grad > 2 – mehrwertige Beziehungs-Typen

Attribute bei mehrwertigen Beziehungs-Typen sind **möglich**

- Beispiel: Stundenanzahl

Frage: Was sagt die (min, max) Notation für das Beispiel aus?

- Max Werte müssen alle n sein, sonst ist kein mehrwertiger Beziehungs-Typ notwendig!

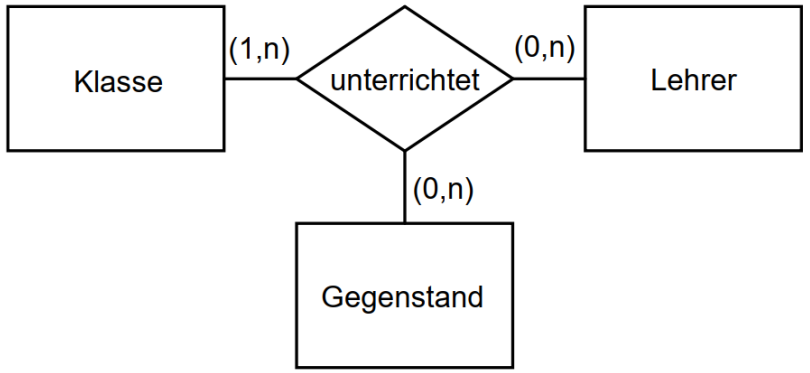


unterrichet = { (3A,Eva,D), (3A,Eva,E), (3A,Max,E), (3B,Eva,D), (3B,Eva,E), (3B,Max,E),... }

MEHRWERTIGE BEZIEHUNGS-TYPEN – BEISPIEL

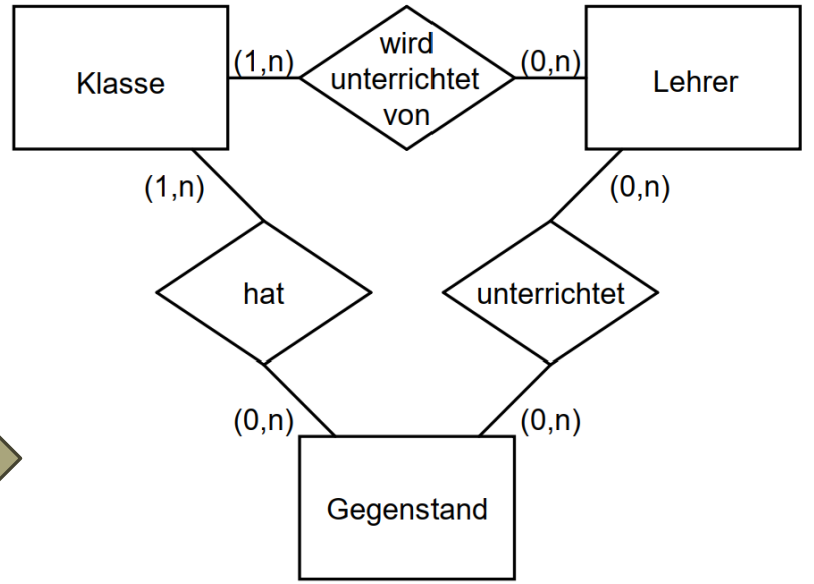
Frage 1: Geben Sie plausible Erklärungen für die min-Werte des mehrwertigen Beziehungs-Typs

Frage 2: Haben die beiden Beispiele denselben Informationsgehalt? Begründen Sie Ihre Antwort



unterrichet = { (3A,Eva,D), (3A,Eva,E), (3A,Max,E), (3B,Eva,D), (3B,Eva,E), (3B,Max,E),... }

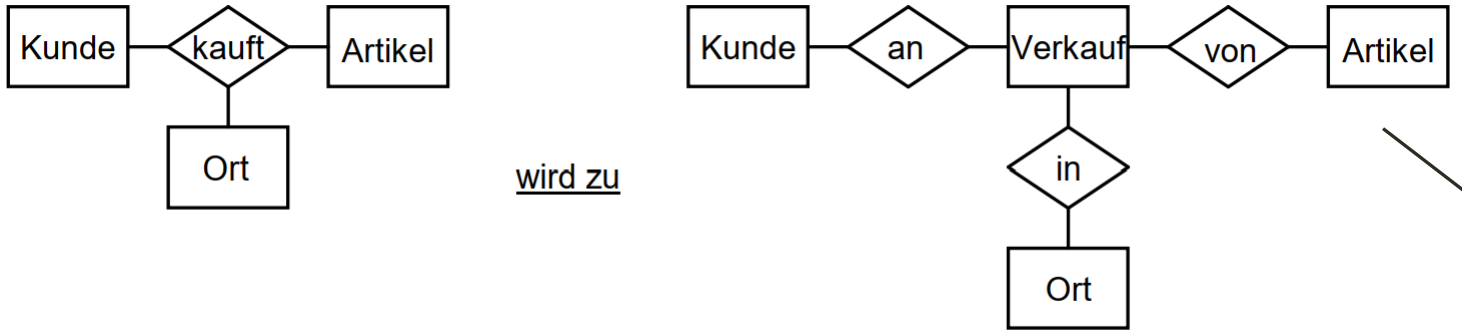


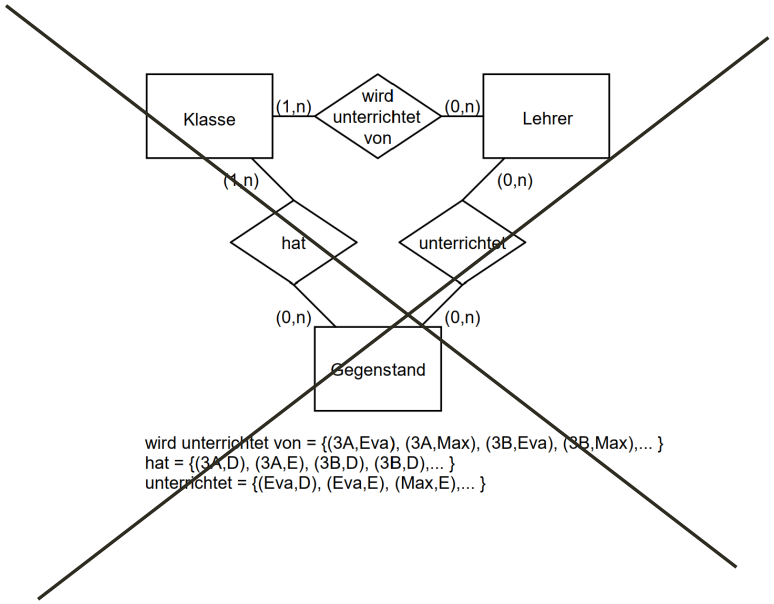


wird unterrichtet von = {(3A,Eva), (3A,Max), (3B,Eva), (3B,Max),... }
hat = {(3A,D), (3A,E), (3B,D), (3B,D),... }
unterrichtet = {(Eva,D), (Eva,E), (Max,E),... }

AUFLÖSUNG MEHRWERTIGE BEZIEHUNGS-TYPEN

Mehrwertige Beziehungs-Typen können auch als **binäre Beziehungs-Typen dargestellt** werden:





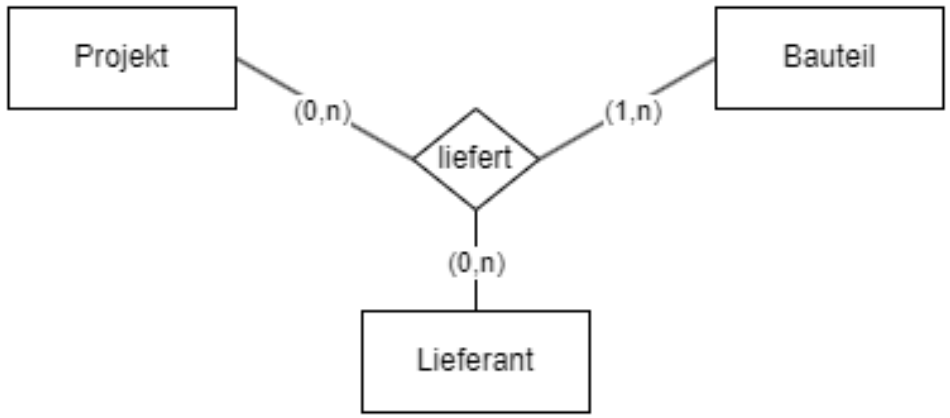
Die (min, max)-Werte müssen adaptiert werden:

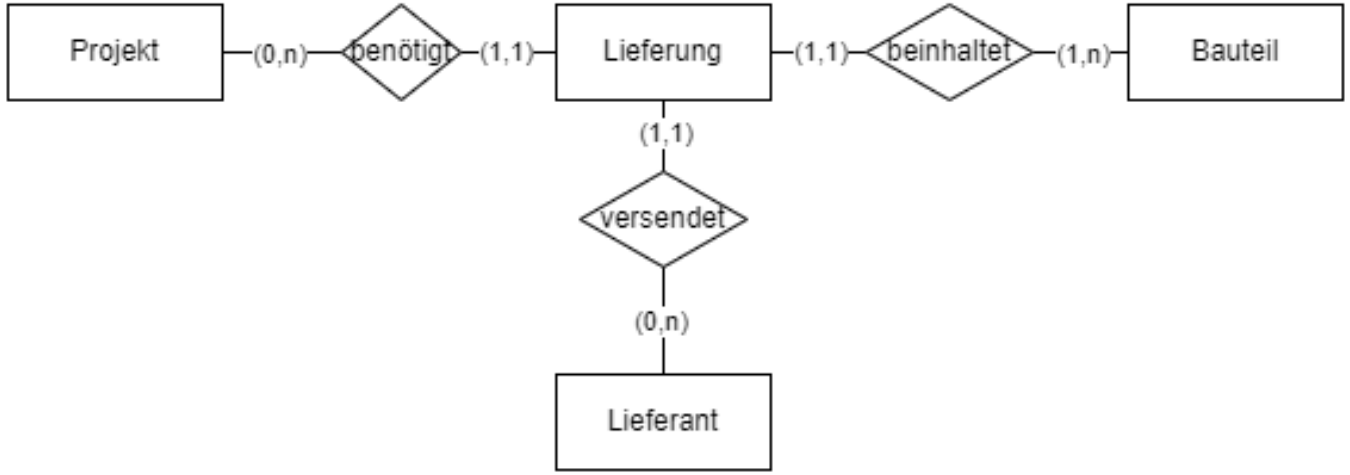
- **Ursprüngliche** Entity-Typen **bleiben gleich**
- **Neue** Entity-Typen sind immer **(1, 1)**

ÜBUNG 1: MEHRWERTIGE BEZIEHUNGS-TYPEN

Erstellen Sie ein ERD mit Angabe der (min,max)-Werte für die folgenden Situation. Nutzen Sie mehrwertige Beziehungs-Typen

- Projekt, Bauteil, Lieferant: Lieferant liefert Bauteil für Projekt
- Erstellen Sie 1x mit einem mehrwertigen Beziehungstyp und 1x mit der Auflösung des mehrwertigen Beziehunstyps





ÜBUNG: ERD BEWERTEN

Was ist Ihre Meinung zu diesem ERD?

- Überbestimmt!!!
- Sponsoren/Mitglieder/ Medaillien eigener Entity-Typ
- Einheitliche Benennung des PK (ID vs. Name vs. Nummer)
- Attributname Größe vs. GrößeInCM vs. Anzahl Personen → vorhandene Einheit beschreiben



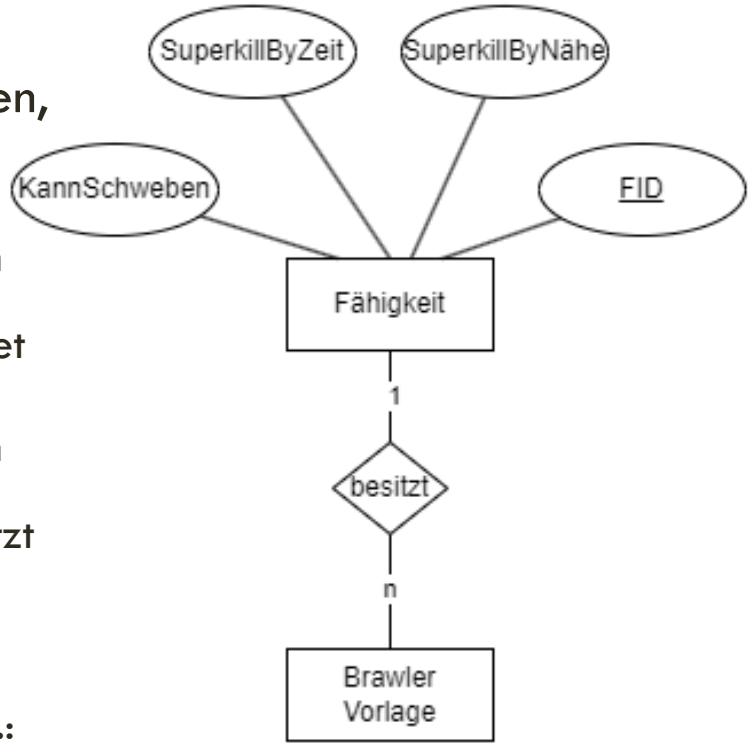
ÜBUNG: ERD BEWERTEN

Wie bewerten Sie die beiden Varianten, um Fähigkeiten einer Brawler-Vorlage darzustellen?

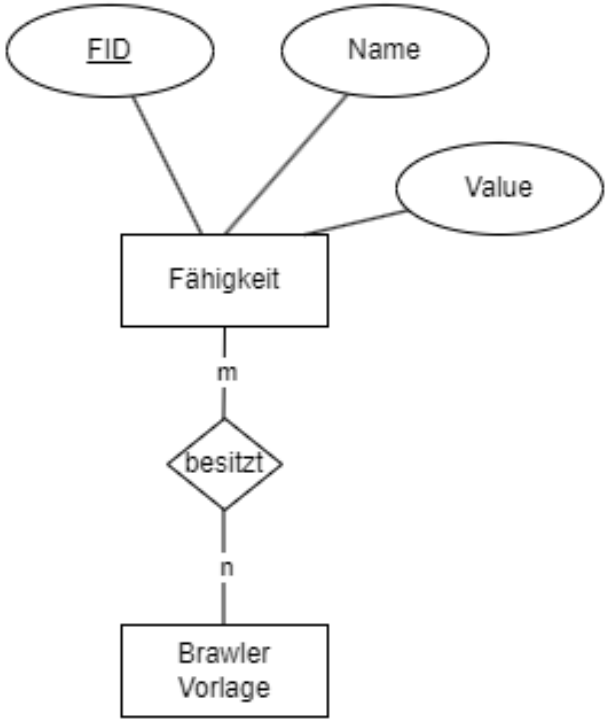
- Variante 1: Für das Hinzufügen einer neuen Fähigkeit muss das Datenbankmodell erweitert werden → neues Attribut bedeutet eine neue Spalte
- Variante 2: Für das Hinzufügen einer neuen Fähigkeit muss lediglich ein neuer Eintrag (INSERT) in die Tabellen Fähigkeit und besitzt durchgeführt werden

Benennung der Attribute

- Bei bool Namen als Frage formulieren: Bsp.: IsActive, KannSchweben
- Größenangaben mit in den Namen eines Attributs aufnehmen Bsp.: GrößeInCM



Variante 1



Variante 2

WIEDERHOLUNG: NOTATIONSFORMEN

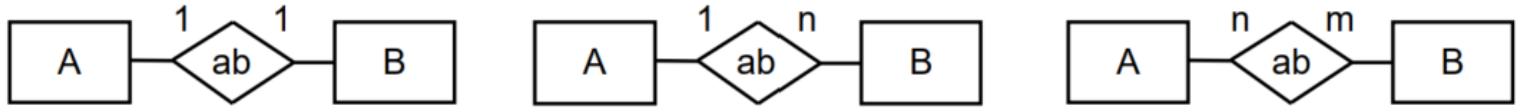
Notationsformen:

- Chen
- Bachmann

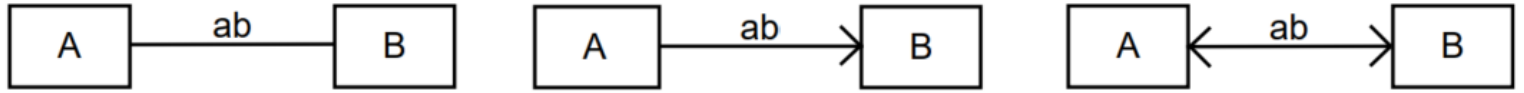
Bachman-Diagramme werden häufig verwendet

- **Nur binäre** Beziehungs-Typen sind möglich
- **Beziehungstypen mit Attributen** sind **nicht** möglich

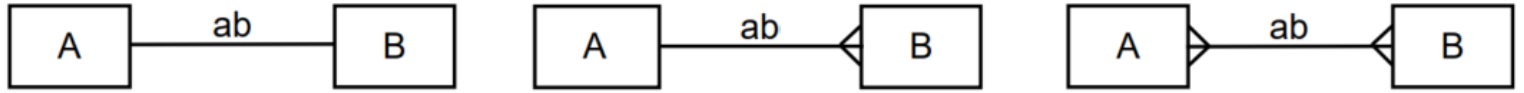
Chen-Diagramm



Bachman-Diagramm mit Arrows

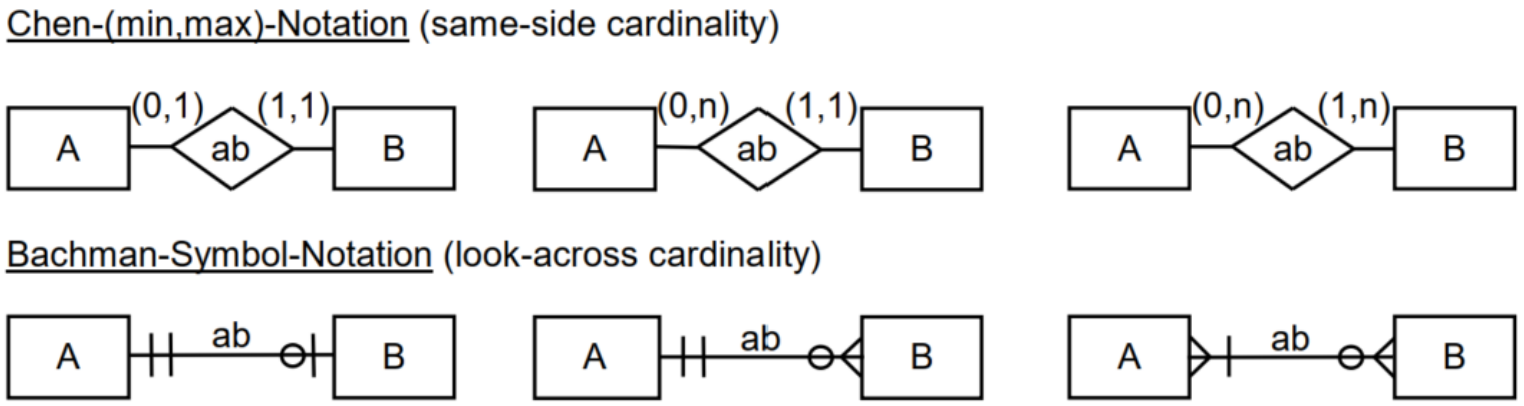


Bachman-Diagramm mit Crowfeet

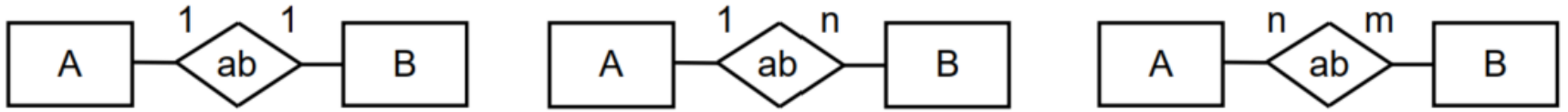


WEITERE NOTATIONSFORMEN

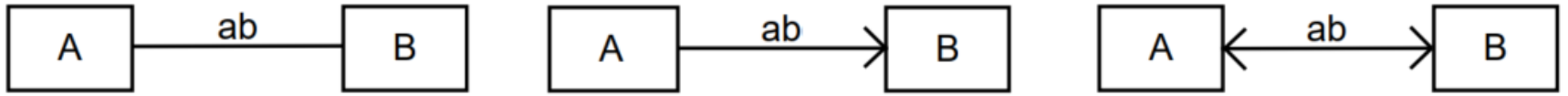
Bachman-Diagramme mit (min, max)-Notation



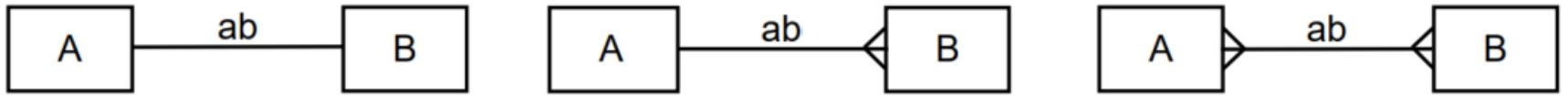
Chen-Diagramm



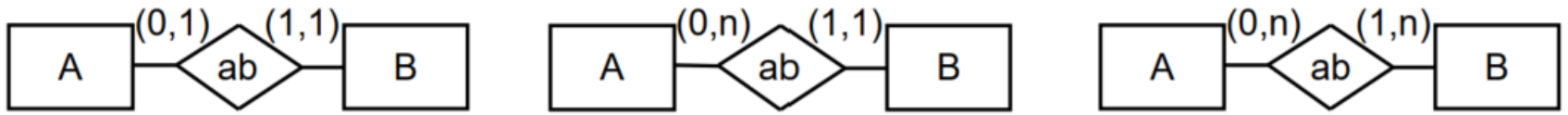
Bachman-Diagramm mit Arrows



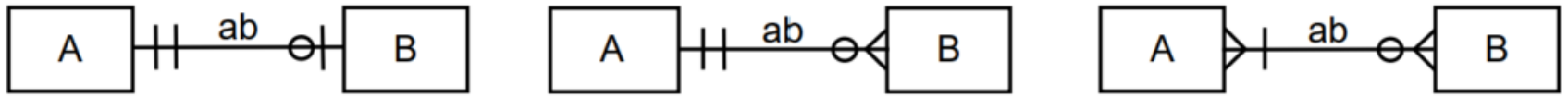
Bachman-Diagramm mit Crowfeet



Chen-(min,max)-Notation (same-side cardinality)

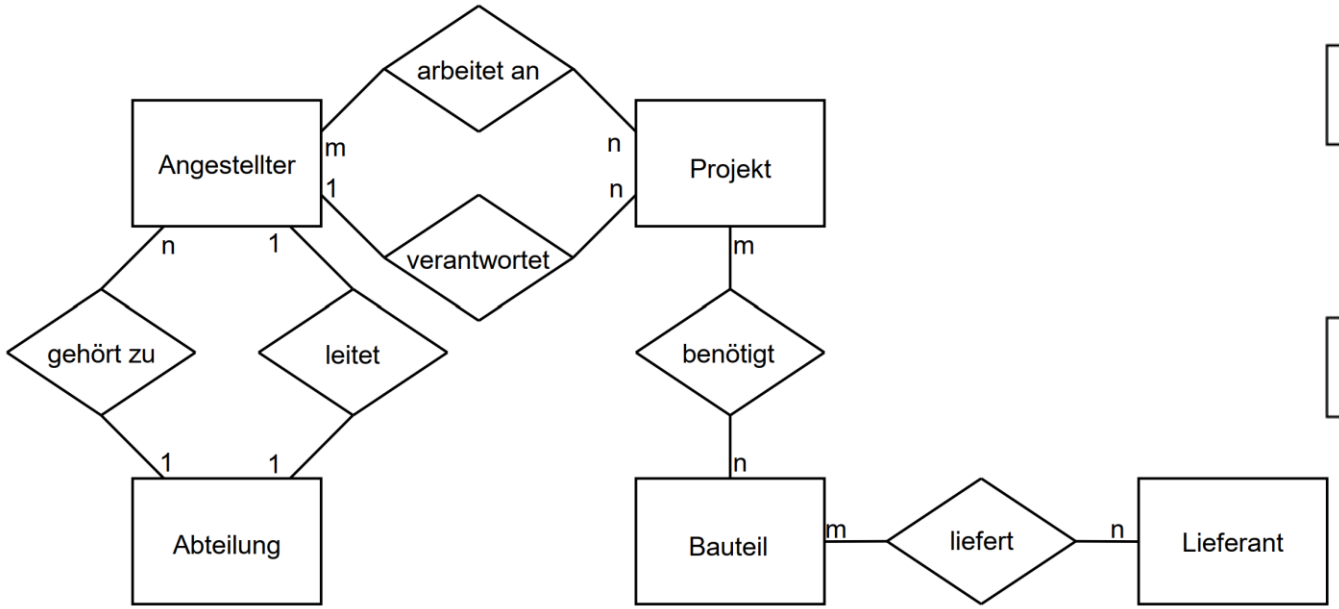


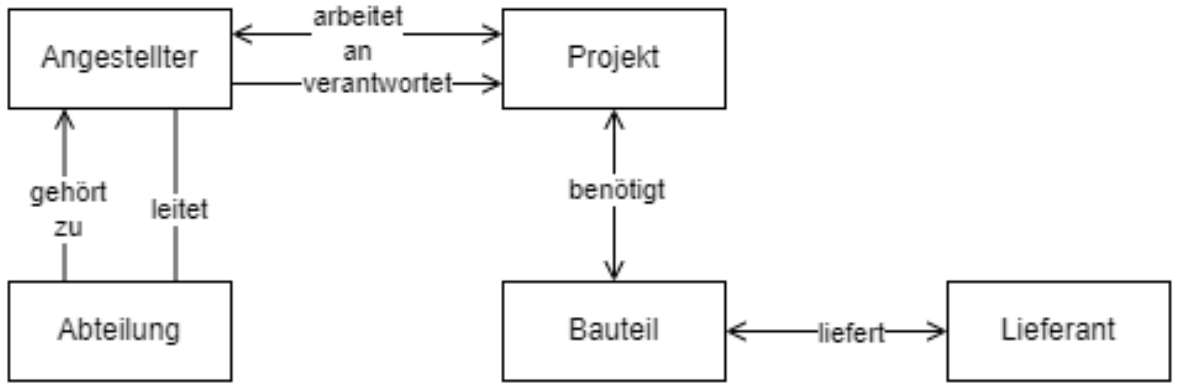
Bachman-Symbol-Notation (look-across cardinality)



ÜBUNG 1: CHEN → BACHMAN

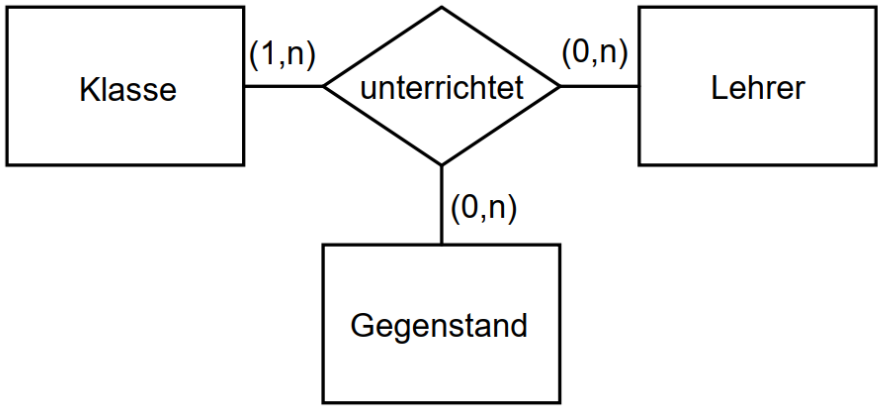
Transformieren Sie folgendes ERD von Chen nach Bachman





ÜBUNG 2: CHEN → BACHMAN

Transformieren Sie folgendes ERD von Chen nach Bachman

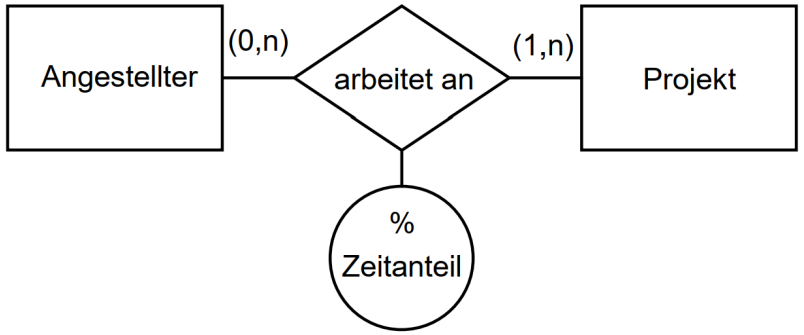




Keine mehrwertigen Beziehungen möglich

ÜBUNG 4: CHEN → BACHMAN

Transformieren Sie folgendes ERD von Chen nach Bachman



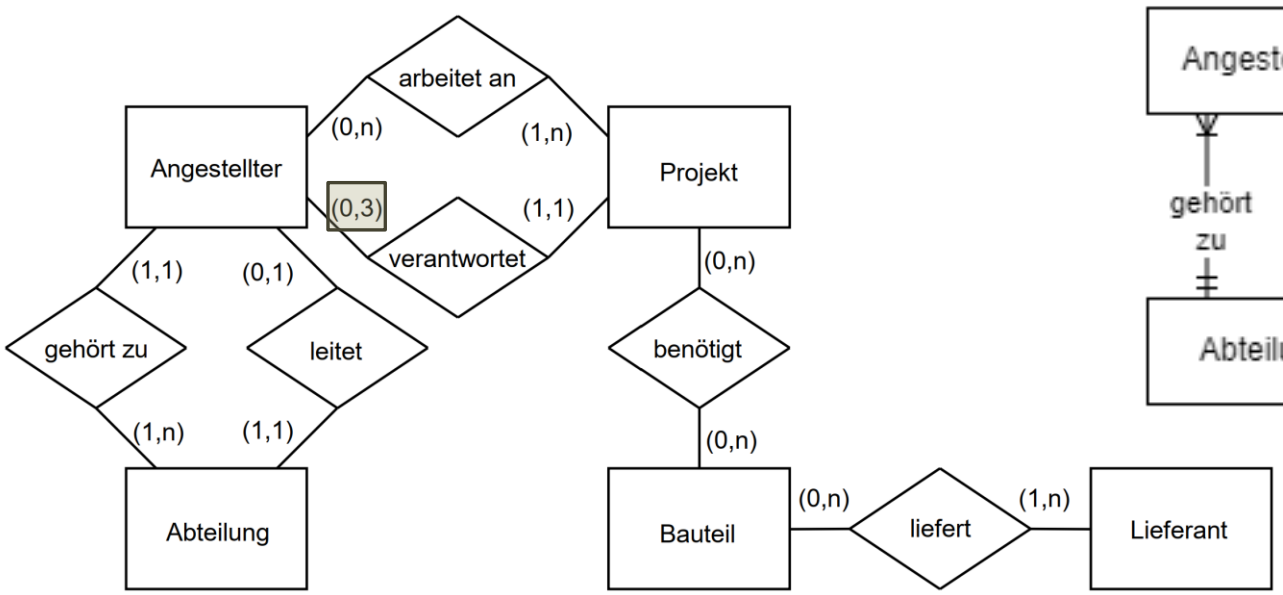


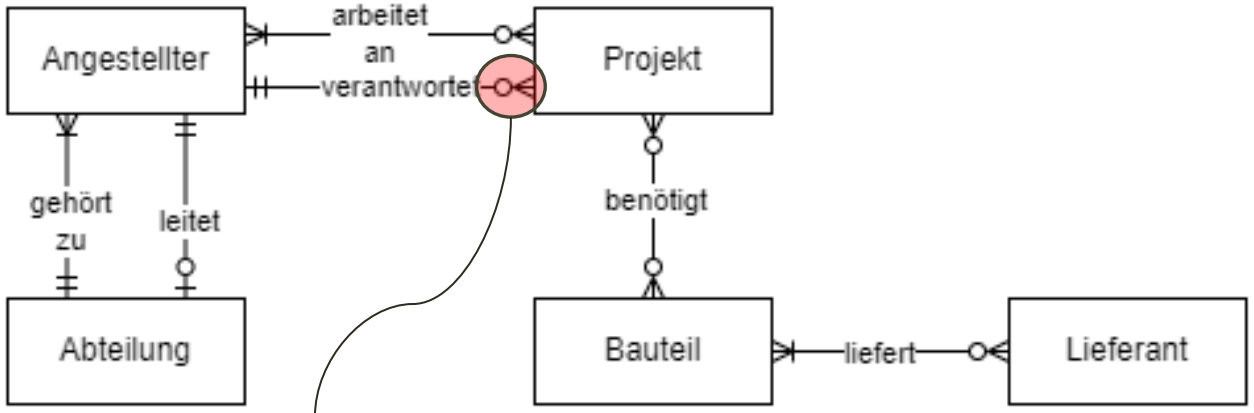
Keine Beziehungs-Typen mit Attributen möglich

ÜBUNG 4: CHEN → BACHMAN

Transformieren Sie folgendes ERD von Chen nach Bachman

- Muss an dem ERD etwas angepasst werden?



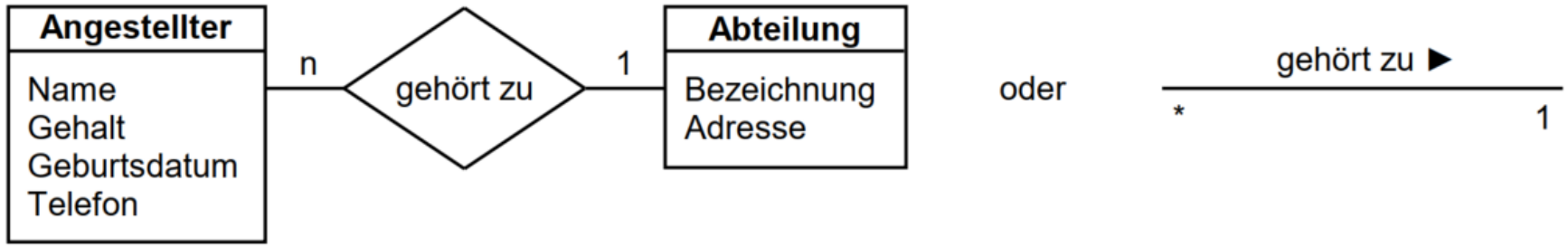


Kann nur als (0,n) dargestellt werden

WEITERE NOTATIONSFORMEN

Attribute im Rechteck:

- Vergleiche mit Klassendiagramme in UML
- Vorteile: Übersichtlicher, weniger Grafikelemente
- Nachteile: Rechtecke werden unterschiedlich groß, keine Beziehungs-Typen mit Attributen möglich



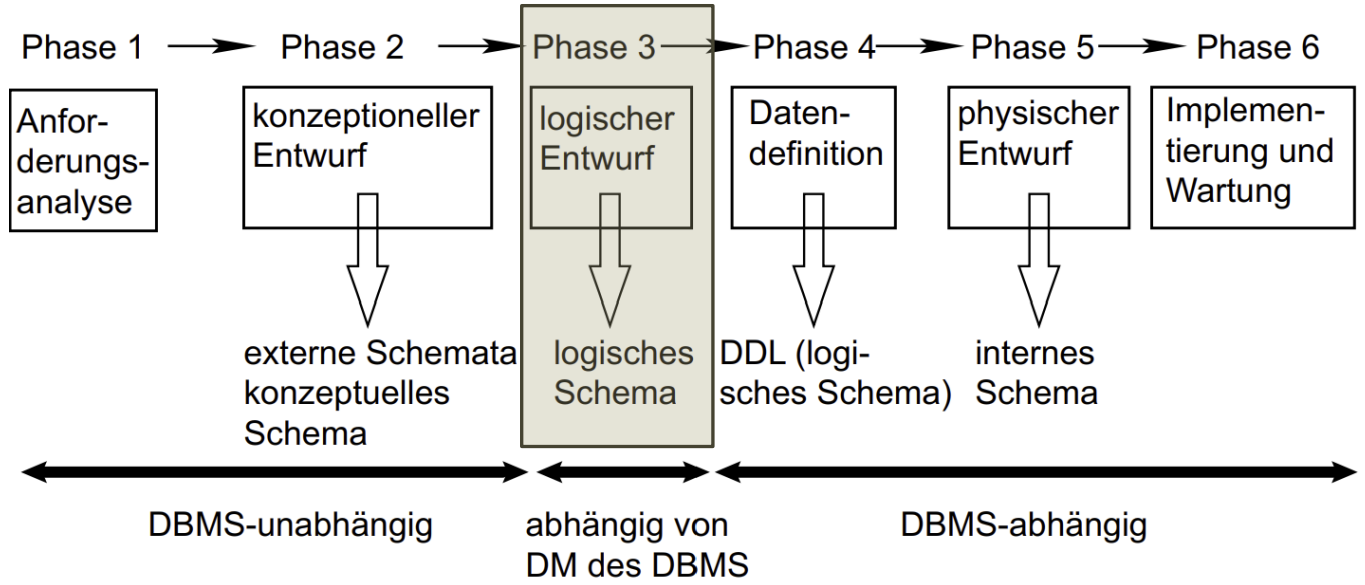
VOM DATENMODELL ZUR IMPLEMENTIERUNG

Kapitel 3

VOM DATENMODELL ZUR IMPLEMENTIERUNG

Wie kann das ERD sinnvoll in Tabellen abgeleitet werden?

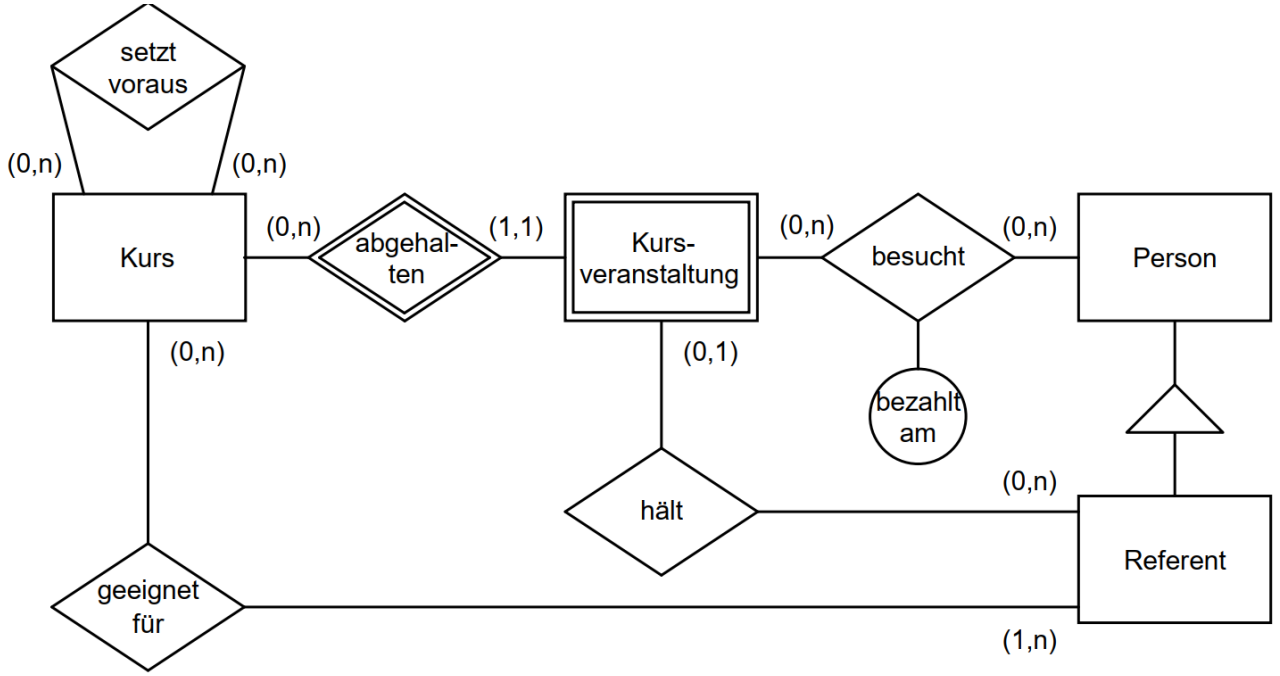
- Welche Entities?
- Welche Schlüssel?
- Welche Kardinalität?
- Welche Attribute und deren Wertebereiche?



KONZEPTIONELLE MODELL IN DAS RELATIONALE MODELL ÜBERFÜHREN

Um vom erstellten Datenmodell nun alle relevanten Daten in einem relationalen Datenmodell darstellen zu können, müssen folgende Informationen transformiert werden:

- Entity-Typ mit Attributen
- Beziehungstypen (1:1, 1:n, n:m)
- Definition von Primary Key und Foreign Key



DARSTELLUNG PK UND FK

Auf die Reihenfolge der Tabellen achten

- Zuerst Tabellen ohne FK erst dann Tabellen mit FK anlegen

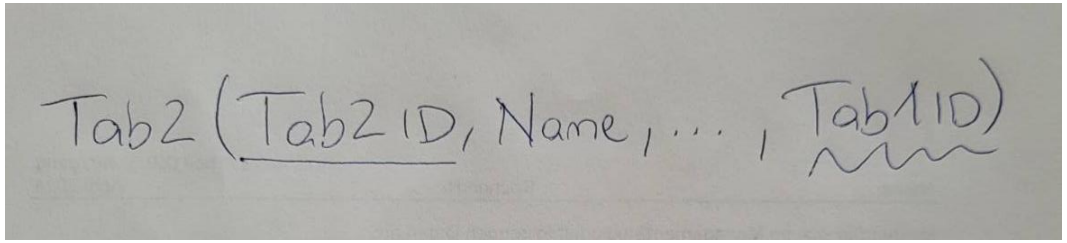
Primärschlüssel werden unterstrichen dargestellt

- `Tab1(TabID, Name)`

Foreign Keys werden kursiv dargestellt

- `Tab2(Tab2ID, Name, …, *Tab1ID*)`

Zeichnen Sie Ihr ERD händisch, dann unterwellen Sie den Foreign Key und unterstreichen den Primary Key

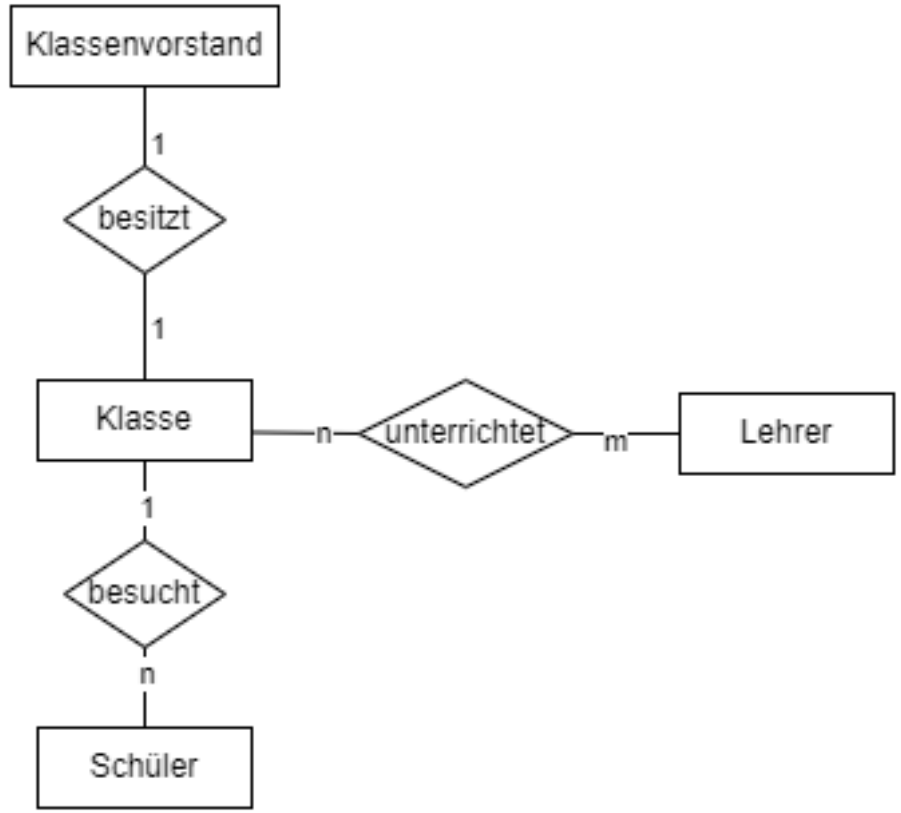


ENTITY-TYP

Für jeden Entity-Typ E wird eine Tabelle T angelegt

- Für jedes Attribut wird eine entsprechende Spalte definiert
- Jedes Entity wird durch eine Zeile repräsentiert
- Die Attributwerte werden in den entsprechenden Spalten der Zeilen (Zellen) als Inhalt abgelegt

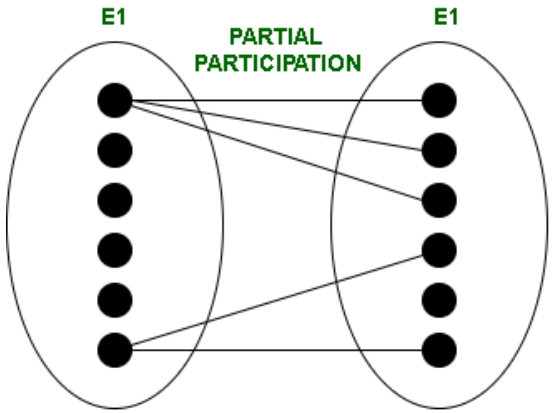
Übung: Leiten Sie nur die beiden Entity-Typen ‚Schüler' und ‚Klasse' in zwei Tabellen um. Welche Attribute werden benötigt?

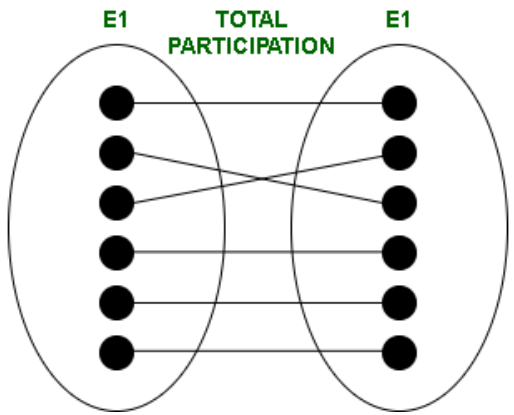


BEZIEHUNGS-TYP – PARTIZIPATION

Beziehungs-Typ

- **Partial oder Optional Participation:** Wenn das repräsentierte Entity an keiner Beziehung teilnimmt, dann erhält der Fremdschlüssel den NULL-Wert
- **Total oder Mandatory Participation:** Wenn ein Entity-Typ vollständig am Beziehungs-Typ teilnimmt, soll die entsprechende Tabelle als T1 gewählt werden → FK auf NOT NULL setzen





1:1 BEZIEHUNG

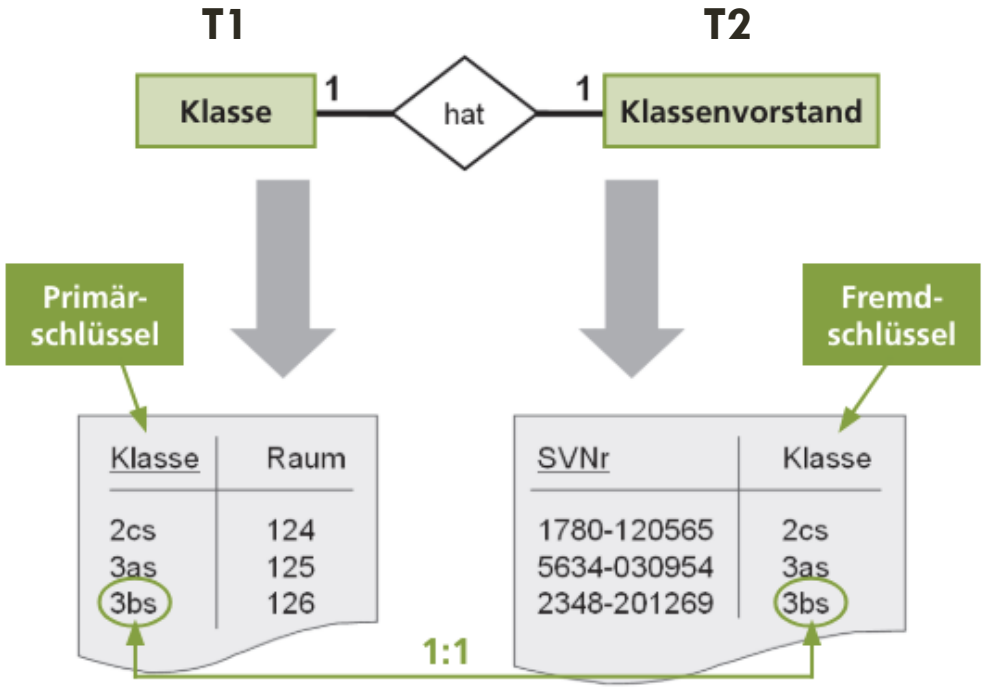
T1 und T2 sind die beiden Tabellen, die die Entity-Typen darstellen, die am Beziehungs-Typ beteiligt sind

PK und FK

- Tabelle T1 erhält einen Foreign Key, bei dem der Primärschlüssel der Tabelle T2 geführt wird

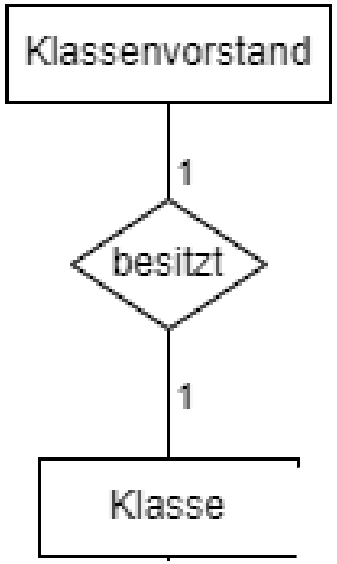
Bei einer 1:1-Beziehungs-Typ muss sichergestellt werden, dass im Fremdschlüssel jeder Primärschlüsselwert **höchstens einmal** auftritt

- NULL-Werte dürfen mehrmals auftreten



1:1 BEZIEHUNG – ABLEITUNG

Für jeden 1:1-Beziehungs-Typen kommt der Fremdschlüssel zu einer der beiden Entity-Typ-Tabellen oder es wird nur 1 Tabelle genutzt



Variante 1:

- `Klassenvorstand (KlassenvorstandID, Name)`
- `Klasse (KlasseID, Klasse, *KlassenvorstandID*)`

Variante 2:

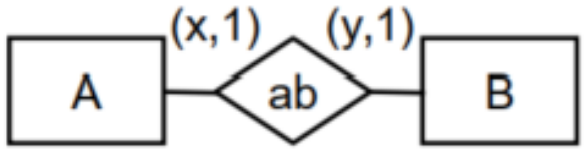
- `Klasse (KlasseID, Klasse, Klassenvorstand)`
- Beide Entity-Typen nehmen **alleine** (ohne anderen Beziehungs-Typen) **vollständig** teil → Darstellung in einer gemeinsamen Tabelle möglich
- Die beiden Entities, die in Beziehung stehen, kommen jeweils in eine Zeile
- Dieser Fall tritt jedoch sehr selten auf

ÜBUNG: (MIN, MAX) NOTATION

Gegeben ist folgendes ERD wie könnte die passende (min, max) Notation aussehen?

(min, max) Notation:

- x=(0,1) & y=(0,1)
  - Tabellenstruktur siehe vorige Folie
  - Wo kommen weniger NULL-Werte zustande?
- x=(0,1) & y=(1,1)
  - A (AID, …)
  - B (BID, …, *AID*) A# NOT NULL und UNIQUE
- x=(1,1) & y=(0,1)
  - B (BID, …)
  - A (AID, …, *BID*) B# NOT NULL und UNIQUE
- x=(1,1) & y=(1,1)
  - Sehr selten!
  - Designfehler?

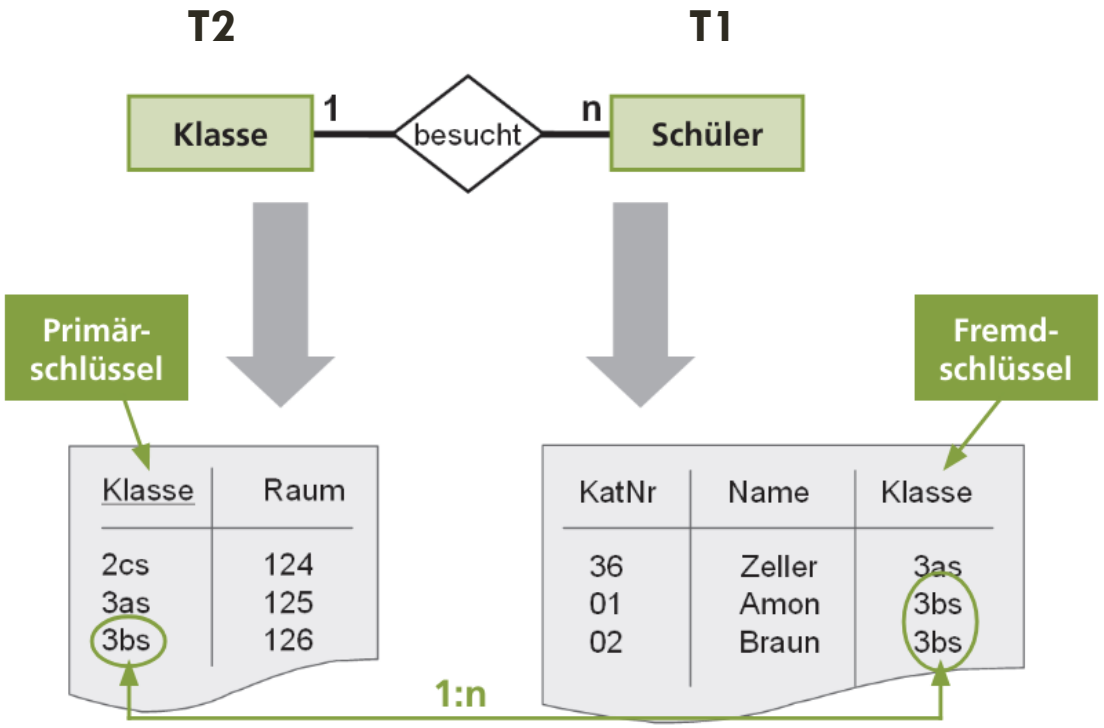


1:N BEZIEHUNG

T1 sei die Tabelle, die den Entity-Typ auf der n-Seite, T2 die Tabelle, die den Entity-Typ auf der 1-Seite des Beziehungs-Typs, darstellt

- Tabelle T1 wird um den Foreign Key Spalten erweitert, in denen der Primärschlüssel der Tabelle T2 geführt wird

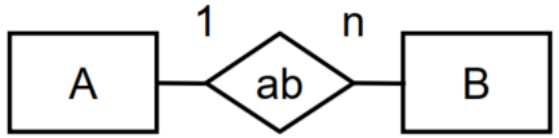
Wenn das Entity, das durch die Zeile repräsentiert wird, an **keiner** Beziehung des Beziehungs-Typs beteiligt ist, enthält der Fremdschlüssel den **NULL-Wert**



1:N BEZIEHUNGSTYP – ABLEITUNG

Tabellenstruktur:

- A (AID, …)
- B (BID, …, *AID*)



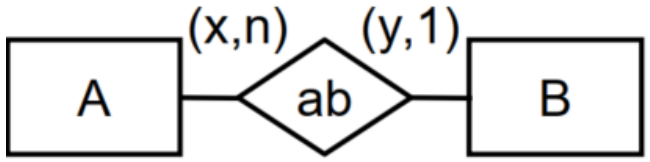
Beispiele:

- Angestellter (PersonalID, ..., *AbteilungsID zu der der Angestellte gehört*)
- Projekt (ProjektID, ..., *PersonalID des Leiters*)
- Angestellter (PersonalID, ..., *PersonalID des Vorgesetzten*)

ÜBUNG: (MIN, MAX) NOTATION MIT 1:N

Gegeben ist folgendes ERD wie könnte die passende (min, max) Notation aussehen?

- Welche Werte können x und y annehmen?
- Inwieweit beeinflusst die Notation die Verbindung?
- Angenommen Tabelle A hat den PK und in Tabelle B liegt der dementsprechende FK



(min, max) Notation:

- x: (0,n) Der PK-Wert AID aus Tabelle A **kann** als FK-Wert in Tabelle B vorkommen
- x: (1,n) Jeder PK-Wert AID aus Tabelle A **muss** als FK-Wert in Tabelle B mind. einmal vorkommen
- y: (0,1) FK-Wert darf NULL sein
- y: (1,1) FK-Wert darf nicht NULL sein

ÜBUNG: (MIN, MAX) NOTATION – UMGANG MIT INSERT, UPDATE, DELETE

Was bedeutet x=1 „Jeder PK-Wert AID aus Tabelle A muss als FK-Wert in Tabelle B mind. einmal vorkommen“ für das INSERT, DELETE und UPDATE?

INSERT in Tabelle A

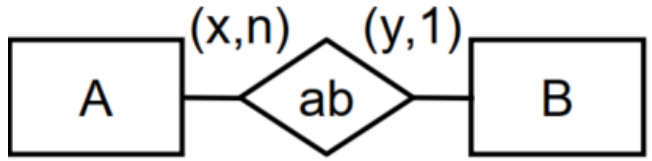
- Es muss ein neuer Datensatz mit dem neuen PK eingefügt werden oder ein bestehender Datensatz aktualisiert, damit „Jeder PK-Wert als FK-Wert mind. einmal vorkommt“

DELETE aus Tabelle B

- Überprüfen ob FK-Wert noch mind. Einmal vorkommt.
  - JA: kein Handlungsbedarf
  - NEIN: Zeile mit PK-Wert ebenfalls löschen

UPDATE aus Tabelle B

- Selbes Verhalten wie bei DELETE

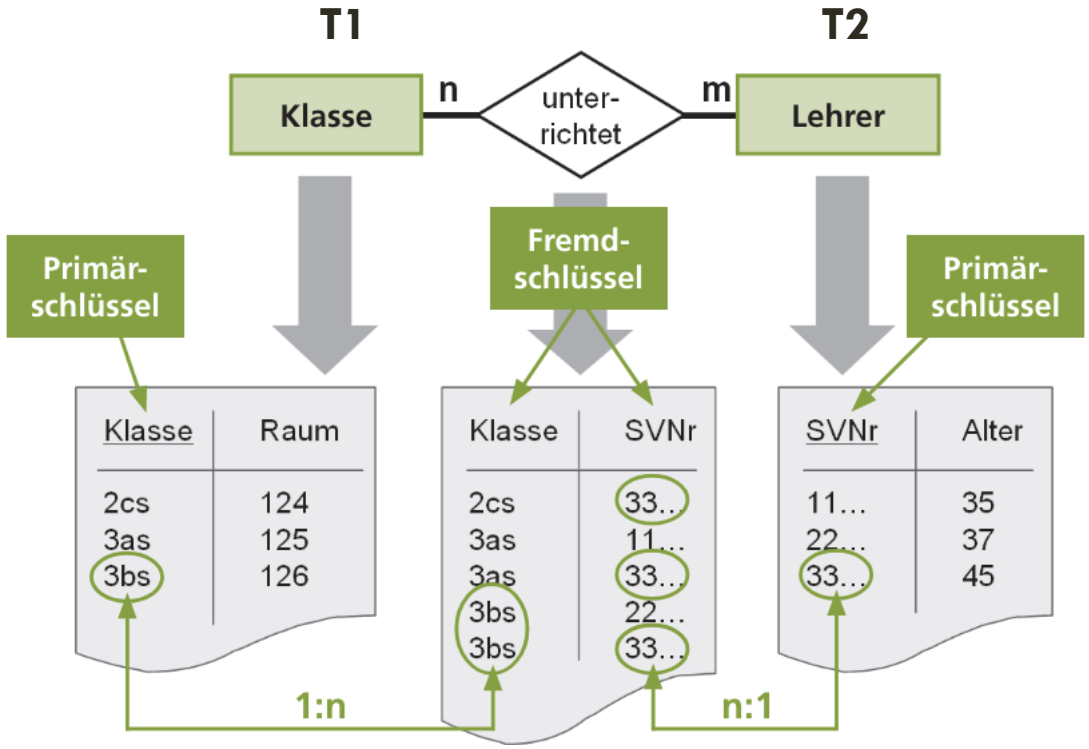


[Schüler] (1,n) <geht> (1,1) [Klasse]

N:M BEZIEHUNG

T1 und T2 seien die beiden Tabellen, die die Entity-Typen darstellen, die am Beziehungs-Typ beteiligt sind

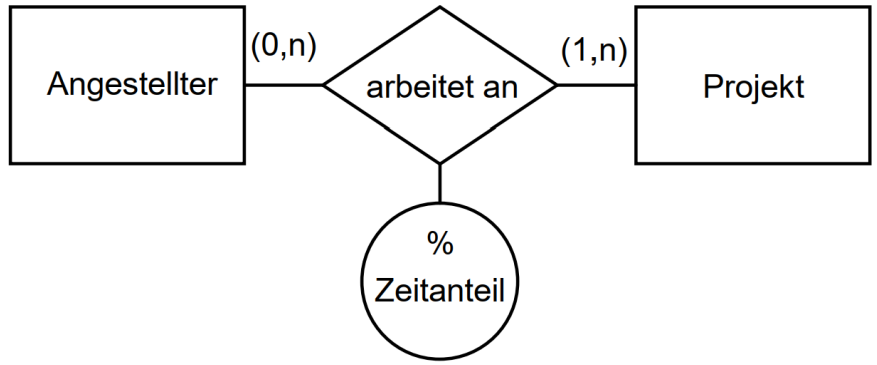
- Assoziative Table (Junction Table): Erstellung einer eigenen Tabelle T, in der die Primärschlüssel der Tabellen T1 und T2 als Fremdschlüssel geführt werden
- Der Primärschlüssel der Tabelle T ist ein zusammengesetzter Schlüssel, der aus den Spalten beider Fremdschlüssel besteht



N:M BEZIEHUNG – ATTRIBUTE

Attribute des Beziehungs-Typs wird die Tabelle T um eine Spalte erweitert

- Bei mehreren Beziehungen zwischen demselben Entity-Paar der Primärschlüssel muss um **ein oder mehrere Attribute** des Beziehungs-Typs erweitert werden, um **Eindeutigkeit zu erreichen**
- Erstellung eines **künstlichen Schlüssels** möglich (eher bei eigenem Entity-Typ)



N:M BEZIEHUNG – WAHL PRIMARY KEY

PK **müssen** eindeutig sein. Bei n:m Beziehungen müssen daher entweder

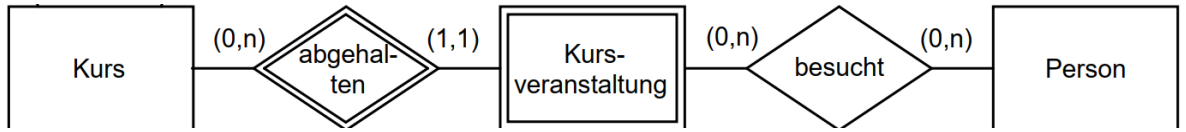
- Ein zusammengesetzter PK aus den Foreign Keys (inkl. Attribute wenn diese notwendig sind, damit PK eindeutig wird)
- Ein künstlicher Schlüssel → laufende ID

Bei n:m Beziehungen ist in den meisten Fällen ein zusammengesetzter PK vorzuziehen

- Außer wenn ein anderer Entity-Typ mit mehreren Schlüsseln angesprochen werden kann



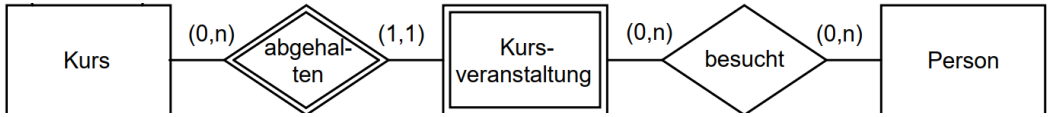
vs



N:M BEZIEHUNG – WAHL PRIMARY KEY



vs



Ableitung (ohne Klasse und Lehrer)

- Unterricht(*KID, LID*)

Künstlicher Schlüssel?

- Beide FK werden nicht gemeinsam benötigt, um ein anderen Entity-Typ zu verbinden
- Bei join reicht meistens **eine** Einschränkung im on
- gute Wahl für PK

Ableitung (ohne Klasse und Lehrer)

- Kursveranstaltung(*KNr, KNrLfnd*,…)
- Person (PNr, …)
- Besuch (*KNr, KNrLfnd, PNr*)

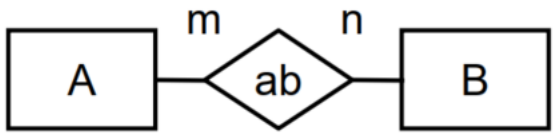
Künstlicher Schlüssel?

- Für n:m Beziehung grundlegend nicht notwendig, aber join von Besuch auf Kursveranstaltung benötigt **zwei** Einschränkungen im on
- Für Kursveranstaltung überlegenswert, damit bei join eine Einschränkung im on ausreichend ist

N:M BEZIEHUNGSTYP – ABLEITUNG

Tabellenstruktur:

- A (AID, …)
- B (BID, …)
- AB (*AID, BID*)
- Beziehungsattribute zusätzlich in AB



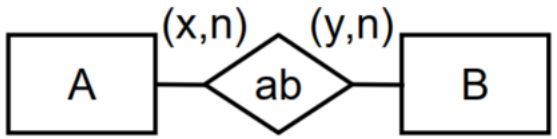
Beispiele:

- arbeitet an (*PersonalID, ProjektID,* %-Zeitanteil)
- hat besucht (*PersonID, SchulID*, Monate, Kosten)
- Ist Komponente von (*BauteilID der Komponente, BauteilID des Zusammenbaus*, Stück)
- kaufte (*KundenID, ArtikelID,* Datum, Stück, Rabatt) Annahme: An einem Tag bei gleichem Kunden und Artikel nur eine Bestellung mit Stück und Rabatt

ÜBUNG: (MIN, MAX) NOTATION

Gegeben ist folgendes ERD wie könnte die passende (min, max) Notation aussehen?

Tabellenstruktur siehe vorige Folie



(min, max) Notation:

- x/y=0 -
- x/y=1 Jeder PK-Wert von A bzw. B muss als FK-Wert in AB mindestens einmal vorkommen

ÜBUNG: (MIN, MAX) NOTATION – UMGANG MIT INSERT, UPDATE, DELETE

Was bedeutet x= 1 bzw. y=1 „Jeder PK-Wert von A bzw. B muss als FK-Wert in AB mindestens einmal vorkommen“ für das INSERT, DELETE und UPDATE?

INSERT in Tabelle A oder in Tabelle B

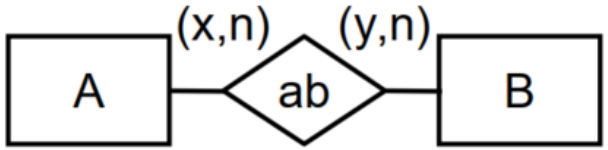
- Es muss ein neuer Datensatz mit dem neuen PK in die Tabelle AB eingefügt werden, damit mindestens ein Datensatz vorhanden ist.

DELETE aus Tabelle A oder aus Tabelle B

- Überprüfen ob FK-Wert noch mind. Einmal vorkommt.
  - JA: kein Handlungsbedarf
  - NEIN: Zeile mit PK-Wert ebenfalls löschen

UPDATE Tabelle AB

- PK → sollten im Regelfall nicht aktualisiert werden



[Klasse] (1,n) <unterrichtet> (1,n) [Lehrer]

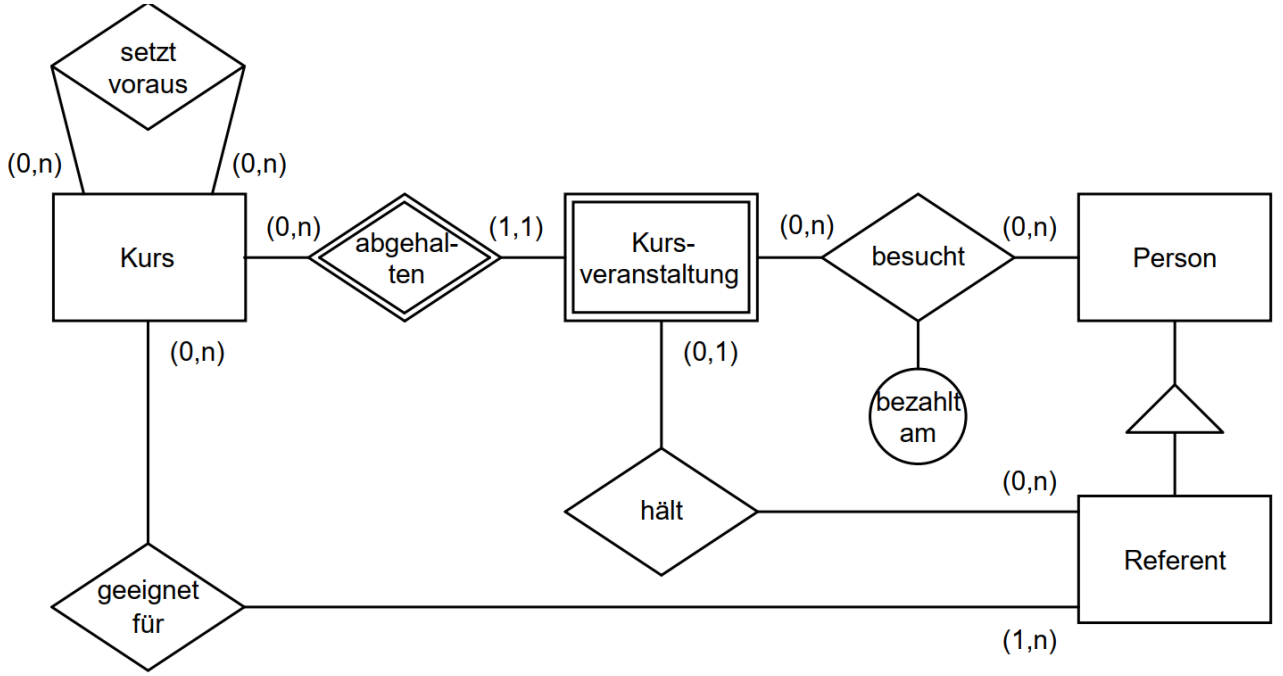
REKURSION

Der Beziehungs-Typ bei einer Rekursion wird als eigene Tabelle abgeleitet

- Es gibt auch Fälle, wo keine eigene Tabelle zwingend notwendig ist

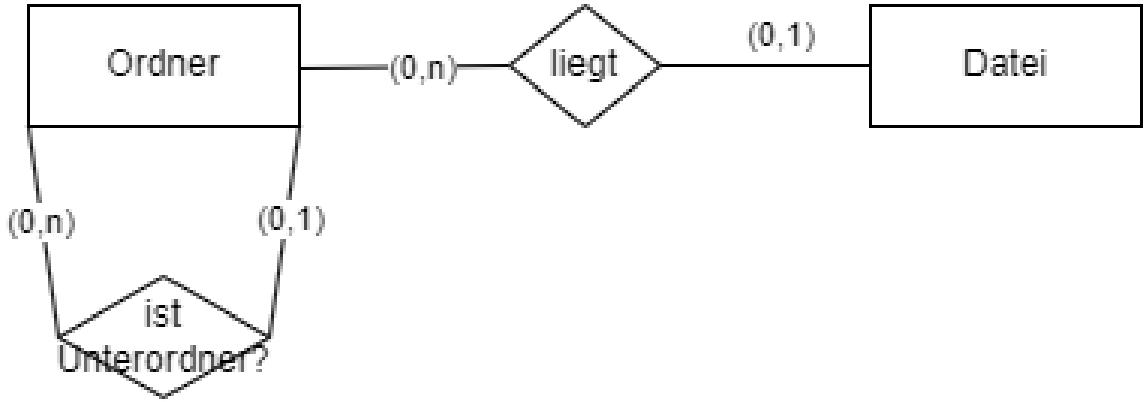
Ableitung:

- `Kurs (KNr, …)`
- `Setztvor (KNr, KNrVor)`



ÜBUNG: REKURSION ABLEITEN

Leiten Sie zu dem nachfolgenden ERD die Tabellen ab:



Tabellen:

- Ordner (<u>OrdnerID</u>, Name, *ParentOrdnerID*)
- Datei(<u>DateiID</u>, Name, *OID*)

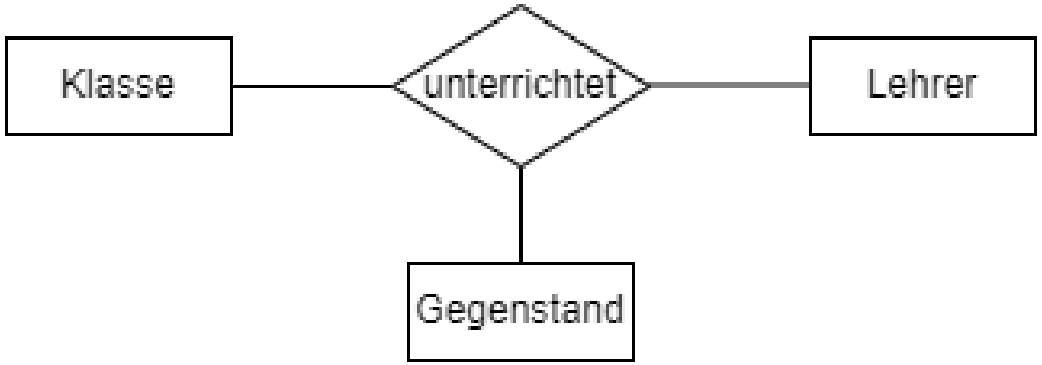
MEHRWERTIGE BEZIEHUNGS-TYP

Mehrwertiger Beziehungs-Typ: Beziehung an der mehr als 2 Entity-Typen beteiligt sind

- Analog wie bei der m:n-Beziehungs-Typ wird eine eigene Tabelle T erstellt,
- Enthält alle PK der Tabellen der beteiligten Entity-Typen darstellen als FK
- PK der Tabelle T ist ein zusammengesetzter Schlüssel aller Fremdschlüssel

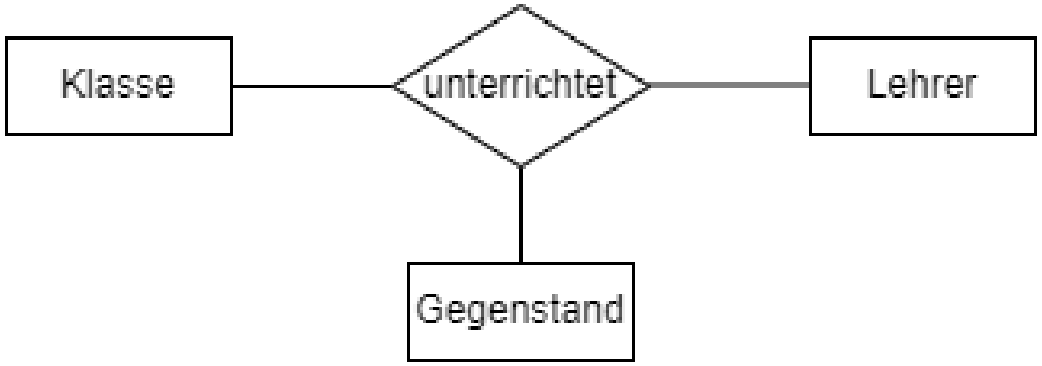
Tabelle T wird für jedes Attribut des Beziehungs-Typs um eine Spalte erweitert

- Eindeutigkeit der PK muss gewährleistet werden → künstlicher Schlüssel auch eine Option



ÜBUNG: MEHRWERTIGER BEZIEHUNGSTYP

Leiten Sie zu dem nachfolgenden ERD die Tabellen ab:



Tabellenstruktur:

- Klasse (KID, …)
- Lehrer(LID, …)
- Gegenstand(GID,…)
- Unterricht(*KID*, *LID*, *GID*)

REGELN FÜR DIE IMPLEMENTIERUNG EINES ERDS IN TABELLEN

Regeln, die zu beachten sind:

- Aus jedem Entity-Typ wird eine gleichnamige Tabelle mit den jeweiligen Attributen.
- Für jeden 1:1-Beziehungs-Typen kommt der Fremdschlüssel zu einer der beiden Entity-Typ-Tabellen.
- Für jeden 1:n-Beziehungs-Typen kommt der Fremdschlüssel zu jener Entity-Typ-Tabelle, bei deren (min, max)-Paar das max den Wert 1 hat.
- Aus jedem m:n-Beziehungs-Typen wird eine gleichnamige Beziehungs-Tabelle, die auf die beteiligten Entity-Typ-Tabellen referenziert. Aus jedem Beziehungsattribut wird ein Attribut der Beziehungstabelle.

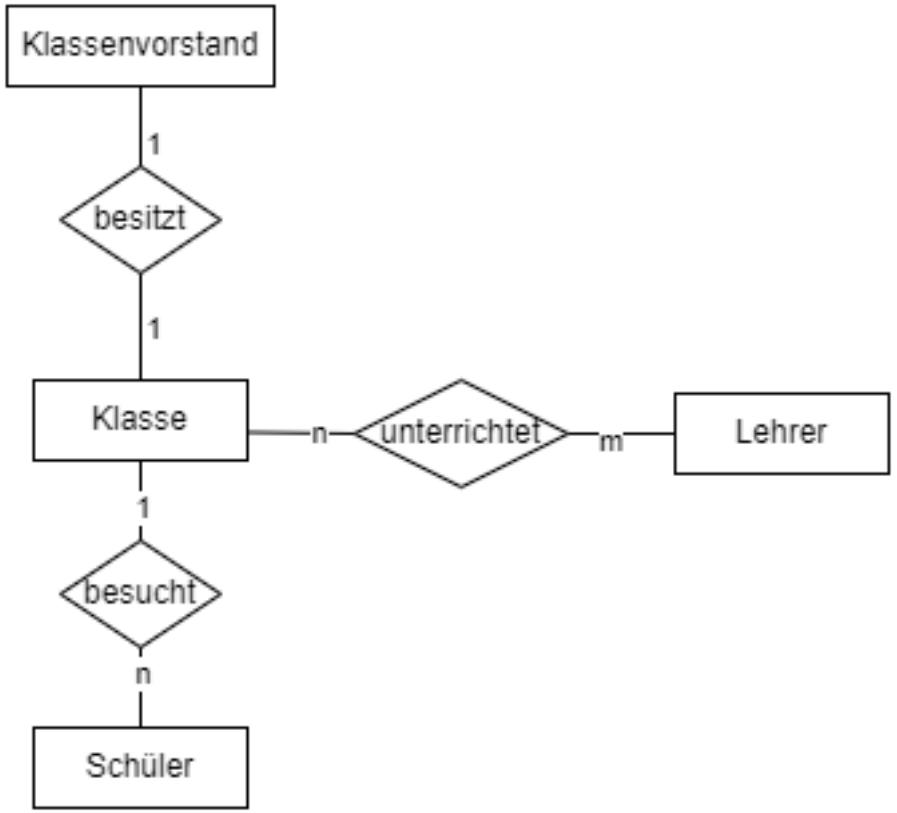
ÜBUNG: ABLEITEN EINES ERD

Leiten Sie folgendes ERD in Tabellen ab:

- Achten Sie auf die Reihenfolge, damit alle Tabellen ohne Fehlermeldung angelegt werden
- PK unterstreichen
- FK kursiv oder unterwellt

Ableitung:

- `Klassenvorstand (KlavoID, Name)`
- `Lehrer (LID, Name, GebDatum)`
- `Klasse (KID, Name, *KlavoID*)`
- `Schüler (SID, Name, GebDatum, *KID*)`
- `Unterricht(*KID, LID*)`



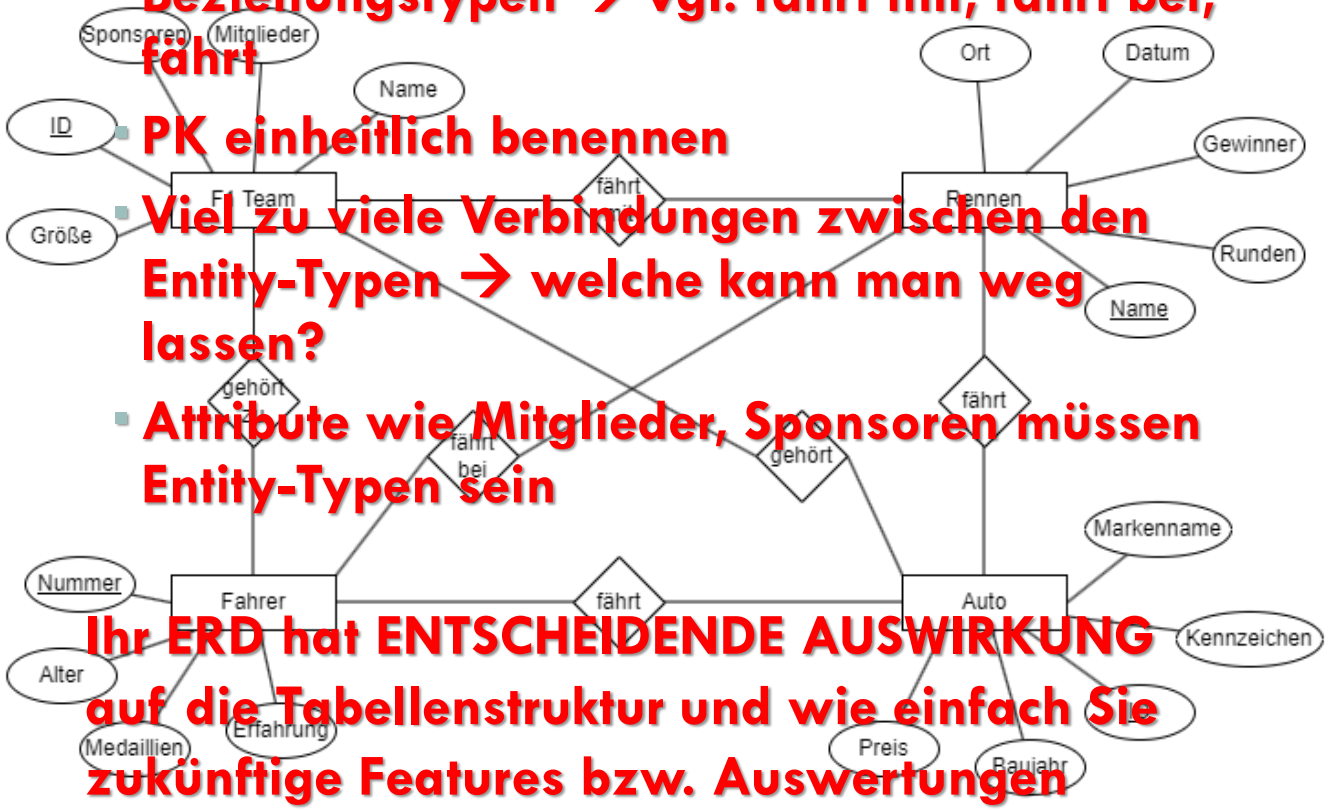
ÜBUNG: ERD ABLEITEN

Leiten Sie folgendes überbestimmtes ERD ohne Adaptierungen ab. Was ist Ihre Meinung zu den abgeleiteten Tabellen?

- F1 Team(ID, Größe, Name, Mitglieder, Sponsoren)
- Rennen(Name, Runden, Gewinner, Datum, Ort)
- Auto(ID, Markenname, Kennzeichen, Baujahr, Preis, *F1TeamID*)
- Fahrer(Nummer, Alter, Medaillen, Erfahrung, *AID, F1Team*)
- fährt bei(*FNummer*, *RName*)
- fährt mit(*F1TeamID*, *RName*)
- Fährt (*FName, AID*)

Hinweise:

- **Sinnvoll unterscheidbare Namen für Beziehungstypen → vgl. fährt mit, fährt bei, fährt**
- **PK einheitlich benennen**
- **Viel zu viele Verbindungen zwischen den Entity-Typen → welche kann man weg lassen?**
- **Attribute wie Mitglieder, Sponsoren müssen Entity-Typen sein**



Ihr ERD hat ENTSCHEIDENDE AUSWIRKUNG auf die Tabellenstruktur und wie einfach Sie zukünftige Features bzw. Auswertungen umsetzen können

ER MODELLIERUNG – DETAILS

Kapitel 4

ÜBERSICHT

Abhängige Entity-Typen

Überlagerte Entity-Typen

Attribute

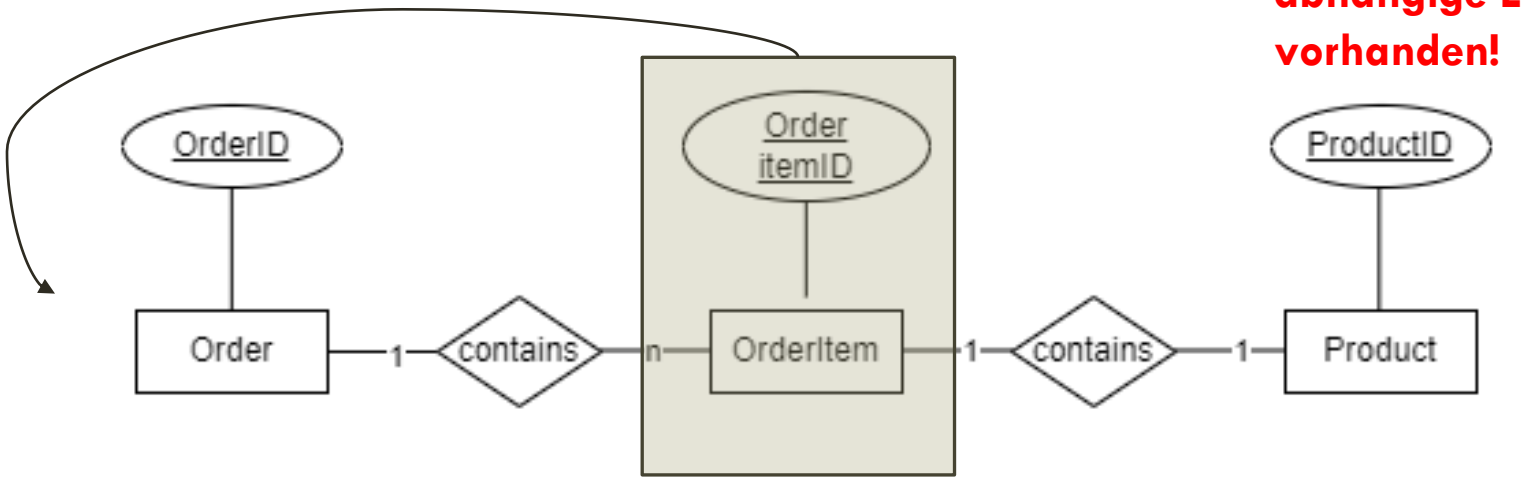
ABHÄNGIGE ENTITY-TYPEN

Gegeben ist eine Bestellung, die 1 bis n Bestellungseinträge enthalten kann. Jedem Bestellungseintrag ist ein Produkt zugewiesen:

- Frage 1: Zeichnen Sie das ERD welches den Sachverhalt darstellen kann
- Frage 2: Gibt es hier Abhängigkeiten, kann ein Entity-Typ ohne dem anderen sinnvoll existieren oder gibt es Entity-Typen die von anderen abhängig sind?

OrderItem kann ohne Order nicht sinnvoll existieren

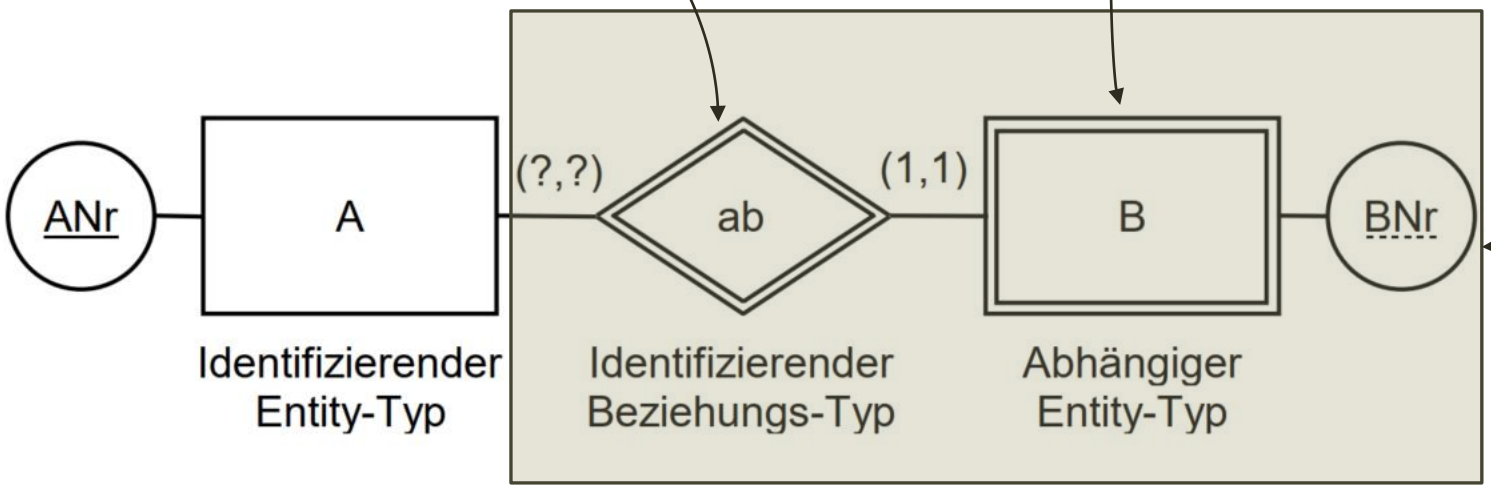
Eigene Notation für abhängige Entity-Typen vorhanden!



ABHÄNGIGER ENTITY-TYP – GRAPHISCHE DARSTELLUNG

Durch „doppelte“ Rauten und „doppelte“ Rechtecke werden die abhängigen Beziehungs-Typen bzw. Entity-Typen dargestellt.

Strichliert unterstrichene PK können **nicht alleine identifiziert werden** → sie sind abhängig vom identifizierenden Entity-Typ



ABHÄNGIGE ENTITY-TYPEN – EIGENSCHAFTEN

Identifizierender Entity-Typ / Starker Entity-Typ

- Jedes Entity existiert **unabhängig** von der Existenz anderer Entities
- Bsp.: Werkzeug

Abhängiger Entity-Typ / Schwacher Entity-Typ

- **Existence Dependency:** Kein Entity kann ohne einem übergeordneten Entity eines bestimmten Entity-Typs existieren
- Abhängige Entity-Typen können von mehreren Entity-Typen abhängen → alle entsprechenden Fremdschlüssel müssen Teil des Primärschlüssels werden
- **Identifizierender Beziehungs-Typ:** Zwischen dem identifizierenden und dem abhängigen Entity-Typ besteht ein Beziehungs-Typ bei dem der (min,max)-Wert beim abhängigen Entity-Typ immer (1,1) ist → muss **vollständig** am Beziehungs-Typ teilnehmen
- (1,1) als (min,max)-Wert bedeutet nicht automatisch Vorliegen eines abhängigen Entity-Typs
- Bsp.: OrderItem muss einer Order zugewiesen sein

ABHÄNGIGER ENTITY-TYP – PK UND FK

PK/FK eines abhängigen Entity-Typs / schwachen Entity-Typs

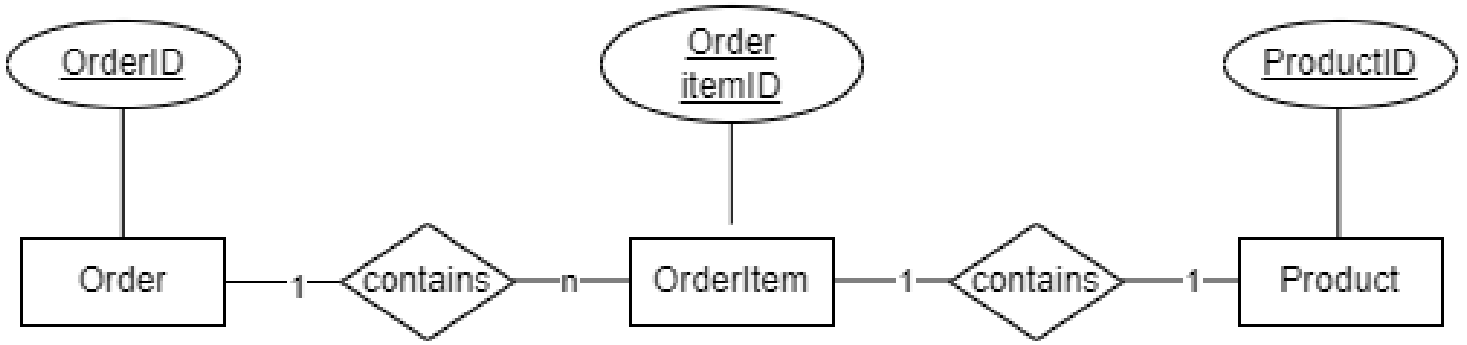
- **Non-transferable Relationships:** Der Fremdschlüssel ändert sich nicht, d.h. die Beziehung vom abhängigen Entity zum identifizierenden Entity ändert sich während der Existenz des abhängigen Entities nicht
- Der Primärschlüssel des Identifizierenden Entity-Typs ist im Primärschlüssel des Abhängigen Entity-Typs als Fremdschlüssel enthalten → ist Teilschlüssel im Abhängigen Entity-Typ
- Bsp.: `OrderItem (OrderItemID, OrderID, …)`

Löschen abhängiger Entity-Typen

- Mit dem identifizierenden Entity werden alle von ihm abhängigen Entities mitgelöscht
- siehe SQL: ON DELETE CASCADE
- Bsp.: Löschen aus Tabelle Order bedeutet, dass die dazu gehörigen OrderItems auch gelöscht werden müssen. OrderItem alleine macht keinen Sinn, dass diese weiter abgespeichert werden.

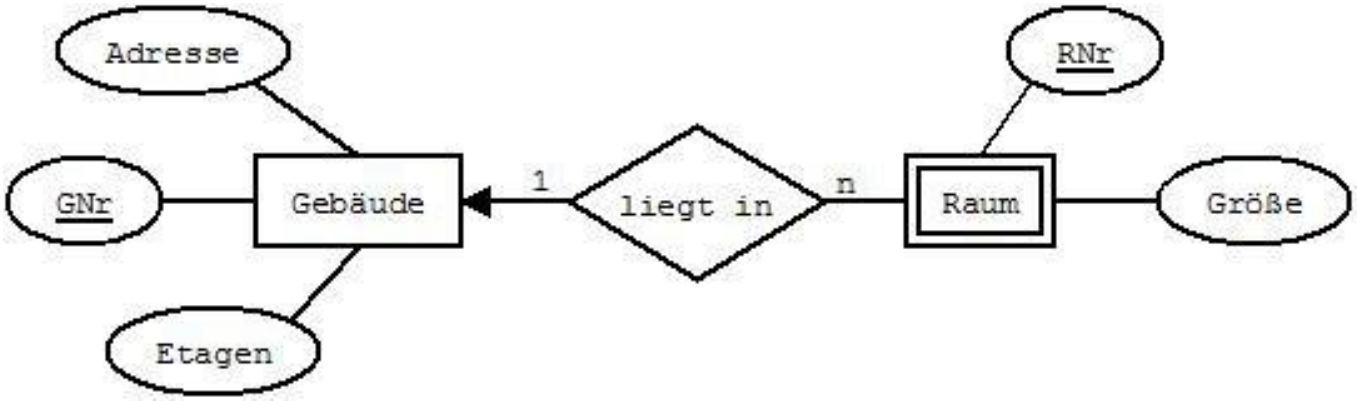
ÜBUNG: ABHÄNGIGER ENTITY-TYP

Zeichnen Sie für das nachfolgende ERD die Starken und Schwachen Entity-Typen ein

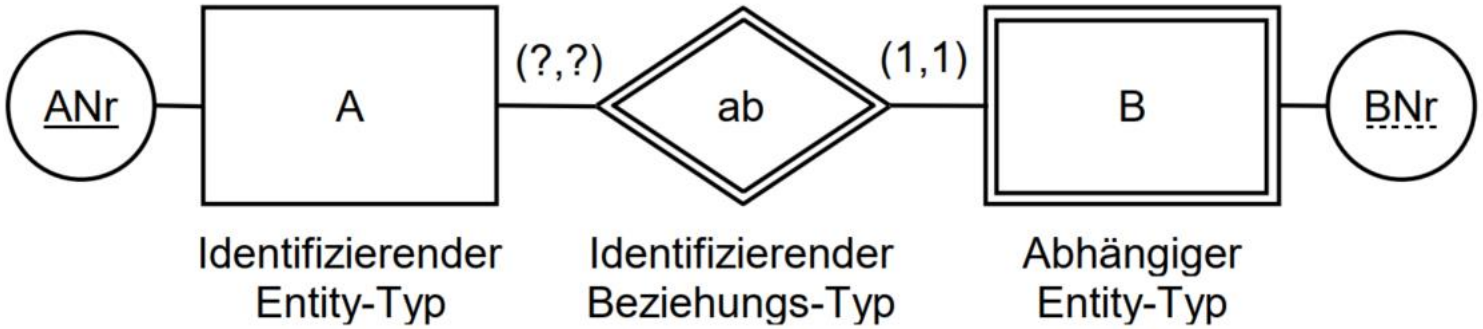


ÜBUNG: ABHÄNGIGER ENTITY-TYP

Zeichnen Sie ein ERD, welches Wohnungen in einem Wohnhaus beschreiben soll.



ABHÄNGIGER ENTITY-TYP IN TABELLEN ABLEITEN



Tabellen:

- A (ANr, ...)
- B (*ANr*, BNr, ...)

ÜBUNG: ABHÄNGIGER ENTITY-TYP IN TABELLEN ABLEITEN

Übung: Leiten Sie das ERD der Wohnungen in Wohngebäuden in die dementsprechenden Tabellen um.

EXKURS: GENERALISIERUNG AUS POS

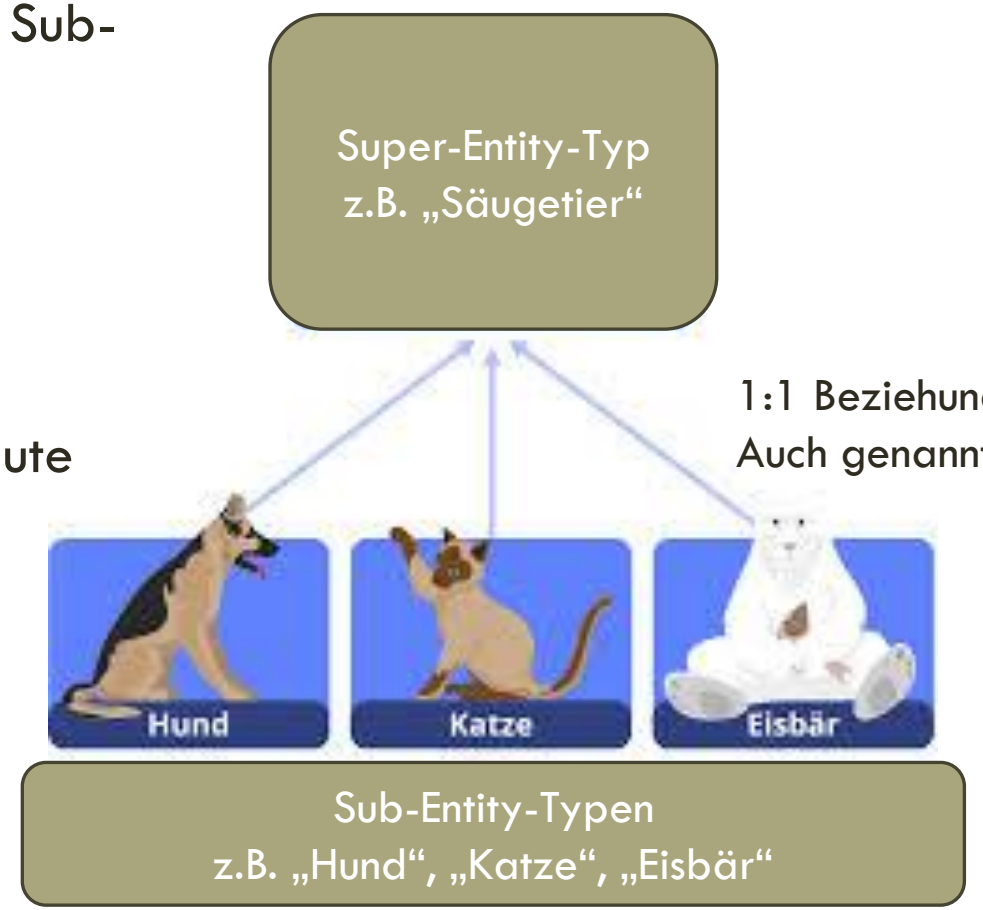
Besitzt alle Eigenschaften,
die für alle Unterklassen gelten

Enthalten alle individuelle
Eigenschaften



ÜBERLAGERTE ENTITY-TYPEN

Besitzt alle Attribute, die für alle Sub-Entity-Typen gelten



Enthalten alle individuellen Attribute

- Mind. 1 Sub-Entity-Typ
- Haben in der Regel selben PK wie Sub-Entity-Typ
- Muss als Entity im Super-Entity-Typ enthalten sein (echte Teilmenge)

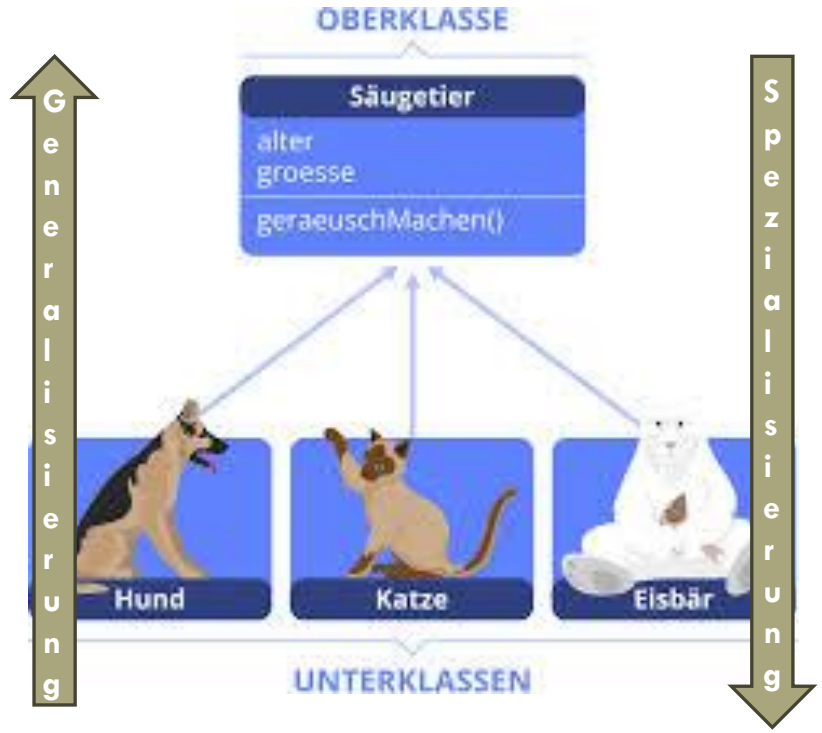
SPEZIALISIERUNG & GENERALISIERUNG

Spezialisierung: Bezeichnet die Aufteilung der Super-Entity-Typen in Sub-Entity-Typen oder die Sub-Entity-Typen selbst

- Beispiel: Mann und Frau sind Spezialisierungen von Person
- Beispiel: Hund, Katze, Eisbär sind die Spezialisierungen von Säugetier

Generalisierung: Bezeichnet die Zusammenfassung der einzelnen Sub-Entity-Typen zum Super-Entity-Typ bzw. der der Super-Entity-Typ selbst

- Beispiel: Person ist die Generalisierung von Mann und Frau
- Beispiel: Säugetier ist die Generalisierung von Hund, Katze, Eisbär



TYP-HIERARCHIE MÖGLICH

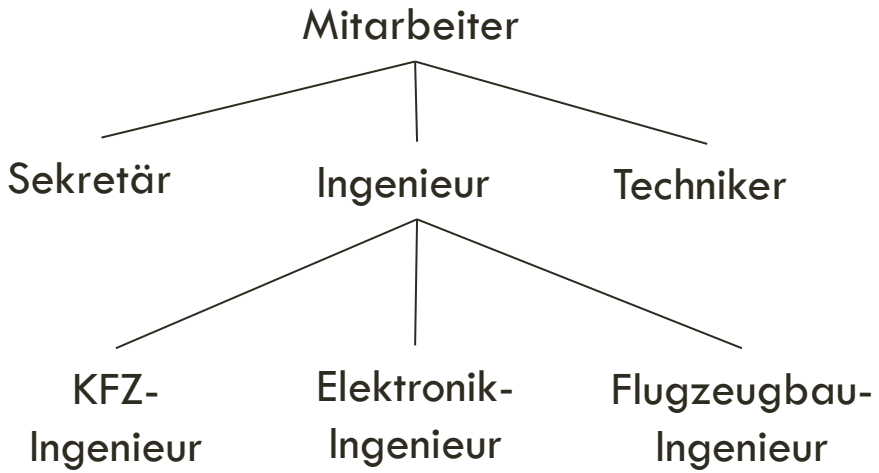
Sub-Entity-Typen können selbst wieder Sub-Entity-Typen haben, sodass eine ganze Typ-Hierarchie entsteht (vergleiche Objektorientierte Konzepte wie Vererbung, etc.)

Beispiel 1: Mitarbeiter

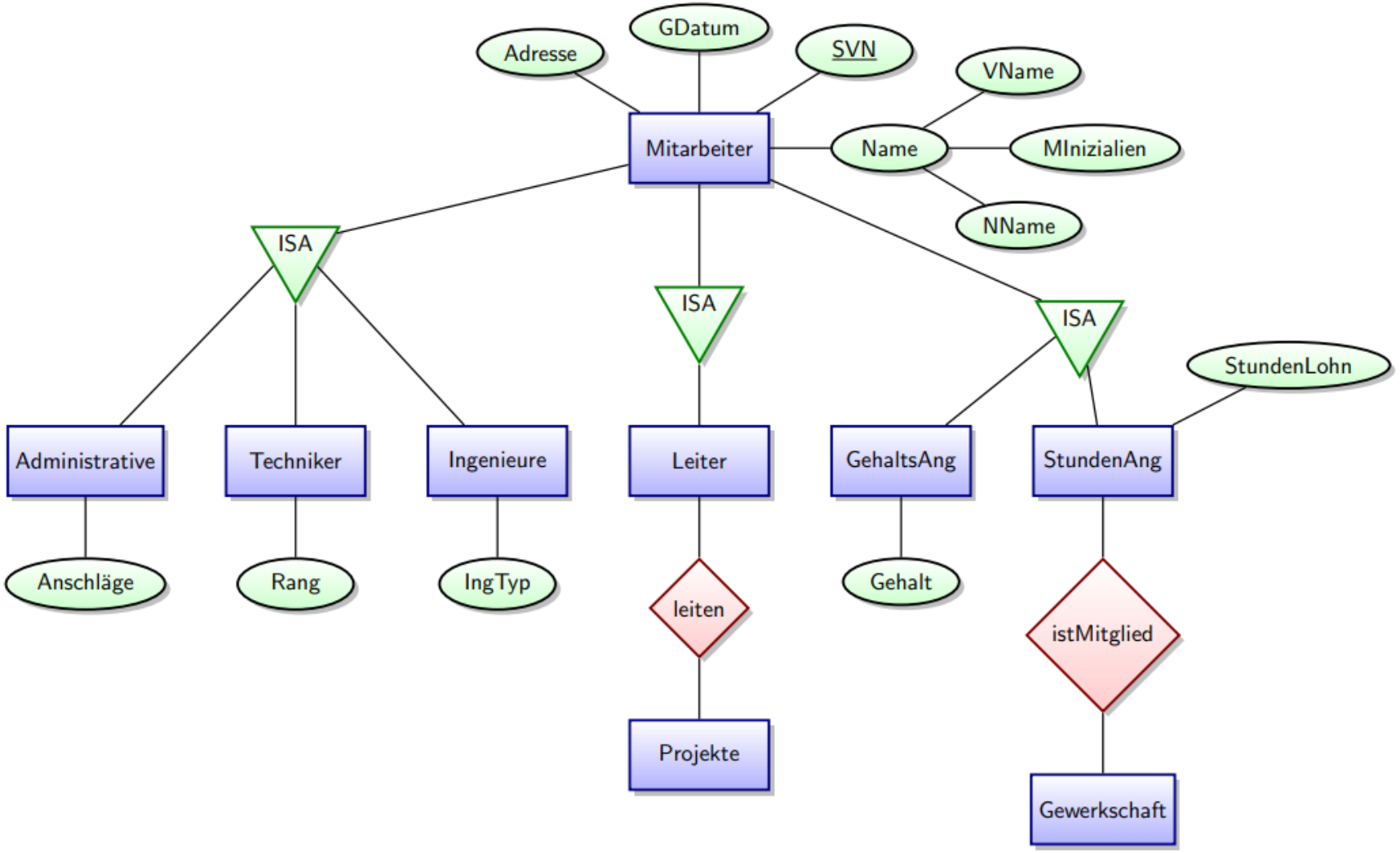
- Mitarbeiter - Sekretär, Ingenieur, Techniker;
- Ingenieur - Kfz-Ingenieur, Elektronik-Ingenieur, Flugzeugbau-Ingenieur;

Beispiel 2: Luftfahrzeug

- Flugzeug - Flugzeug, Hubschrauber, Luftkissenfahrzeug, anderes Luftfahrzeug Flugzeug - Segelflieger, Motorflieger, Düsenflugzeug;



BEISPIEL: ÜBERLAGERTE ENTITY-TYPEN



ARTEN DER SPEZIALISIERUNG

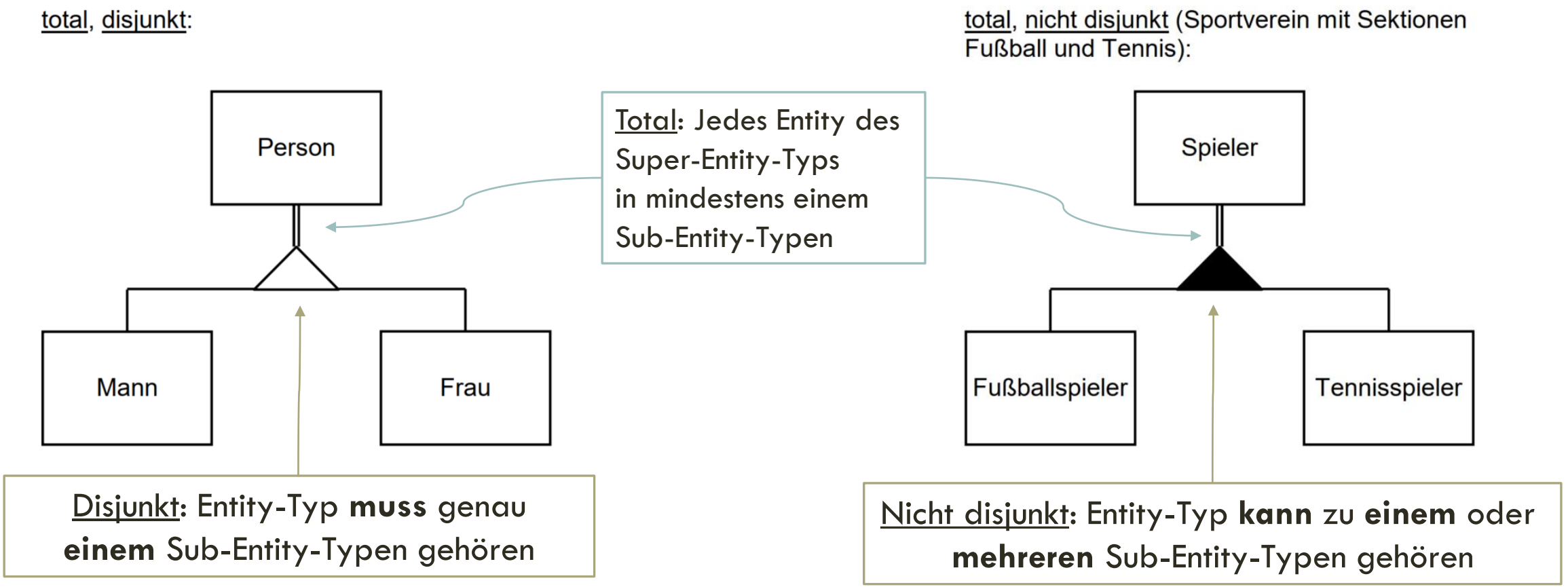
Disjointness Constraint (disjunkt): Gibt es Entities des Super-Entity-Typs, die in mindestens zwei Sub-Entity-Typen enthalten sind?

- nein: disjunkt, disjoint 
- ja: nicht disjunkt, overlapping 

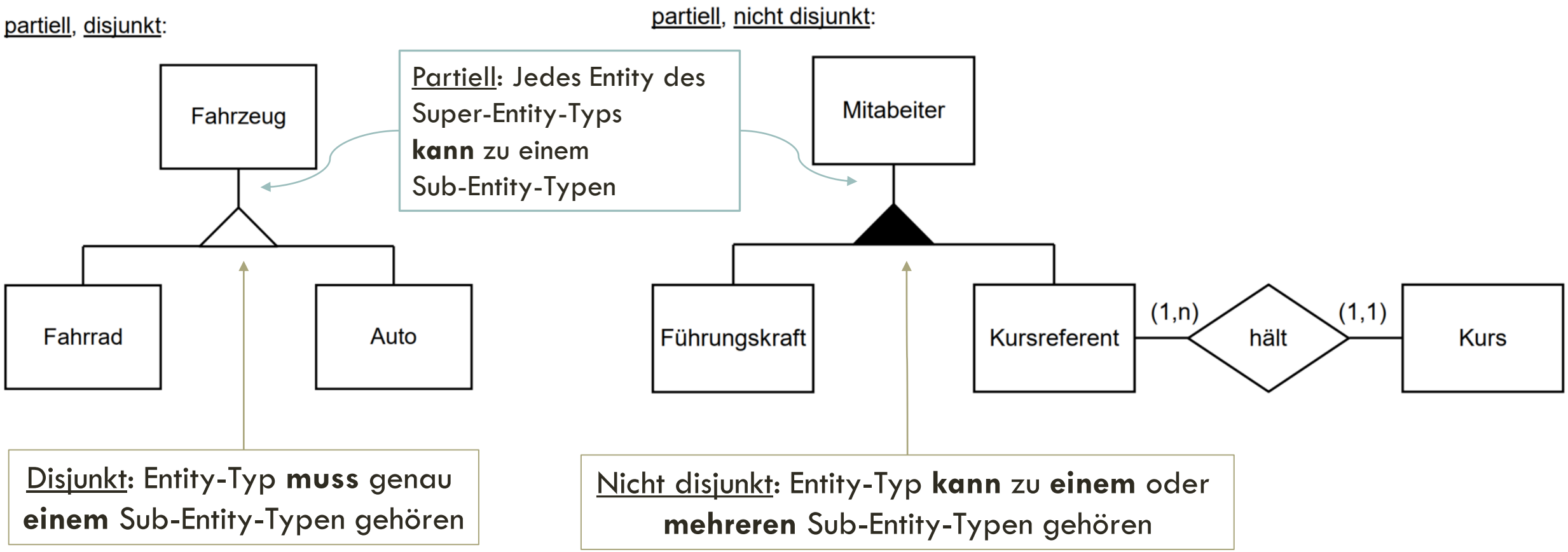
Completeness Constraint (total): Ist jedes Entity des Super-Entity-Typs in mindestens einem Sub-Entity-Typen enthalten?

- Kein Entity-Typ kann nur zum Super-Entity-Typ gehören
- ja: total
- nein: partiell, partial

IS-A-BEZIEHUNGS-TYP – GRAPHISCHE DARSTELLUNG



IS-A-BEZIEHUNGS-TYP – GRAPHISCHE DARSTELLUNG



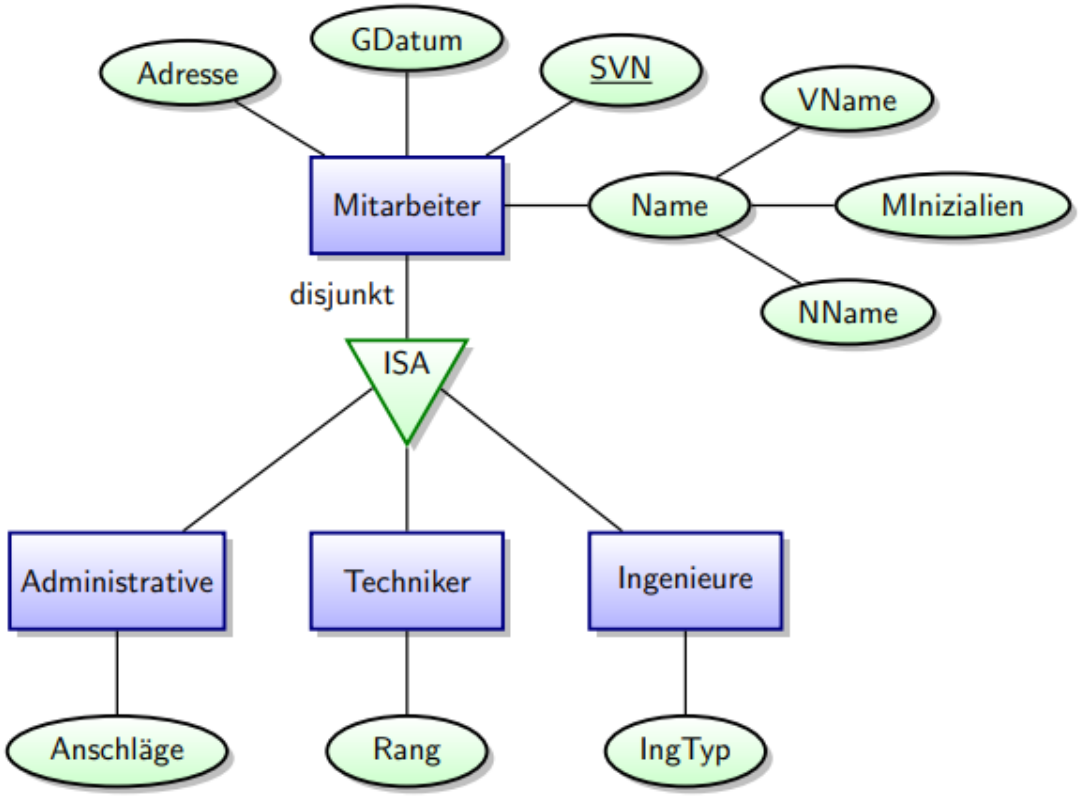
ÜBUNG: ART DER SPEZIALISIERUNG

Welche Constraints werden genutzt?

- Partiell und disjunkt

Wie interpretieren Sie die mögliche Art der Spezialisierung im angeführten Beispiel?

- Partiell: Ein Mitarbeiter muss weder Admin, Techniker noch Ingenieur sein
- Disjunkt: Ein Mitarbeiter kann nicht Admin und Techniker, Admin und Ingenieur oder Techniker und Ingenieur zugleich sein



disjunkt: Gibt es Entities, die in mindestens zwei Sub-Entity-Typen enthalten sind? Nein (disjunkt); Ja (nicht disjunkt)
total: Ist jedes Entity in mindestens einem Sub-Entity-Typen enthalten? Ja (total); Nnein (partiell)

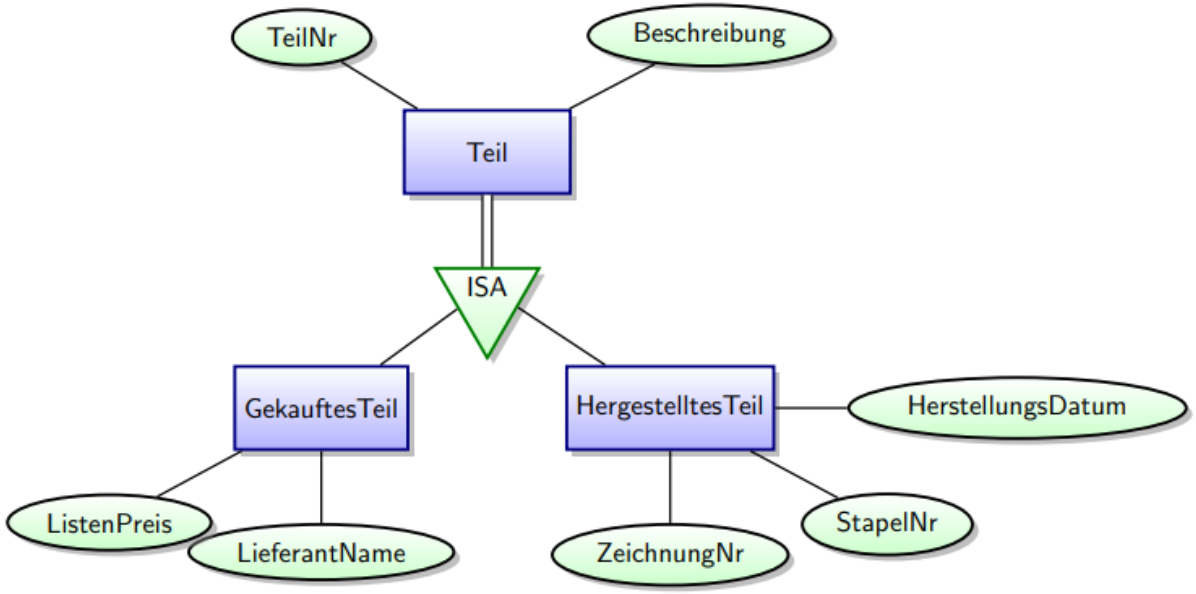
ÜBUNG: ART DER SPEZIALISIERUNG

Welche Constraints werden genutzt?

- Total und nicht disjunkt

Wie interpretieren Sie die mögliche Art der Spezialisierung im angeführten Beispiel?

- Total: Es gibt kein Teil, das weder hergestellt noch gekauft
- Nicht disjunkt: Ein Teil kann zugleich hergestellt und gekauft sein

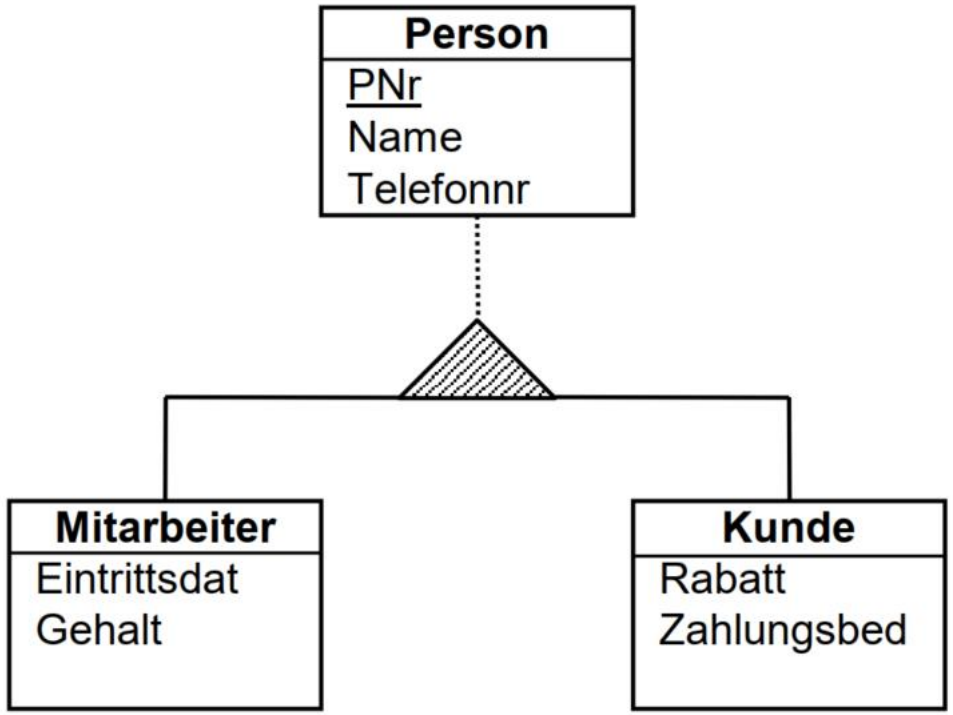


disjunkt: Gibt es Entities, die in mindestens zwei Sub-Entity-Typen enthalten sind? Nein (disjunkt); Ja (nicht disjunkt)
total: Ist jedes Entity in mindestens einem Sub-Entity-Typen enthalten? Ja (total); Nnein (partiell)

IMPLEMENTIERUNG IN TABELLEN

Interpretieren Sie die verschiedenen Möglichkeiten der Disjointness- und Completeness-Constraints

- Wie viele/welche Tabellen sind notwendig?
- Begründen Sie, warum Sie welche Tabellen gewählt haben und die dazugehörigen Vor- und Nachteile



IMPLEMENTIERUNG IN TABELLEN – VARIANTE 1A

Eine Tabelle pro Entity-Typ

- PK der Sub-Entity-Typen sind dieselben wie der Primärschlüssel des Super-Entity-Typs
- PK sind auch die Fremdschlüssel zum Super-Entity-Typ

Vertical Mapping: Tabellen der Sub-Entity-Typen übernehmen PK vom Super-Entity-Typ

- Person (PNr, Name, Telefonnr)
- Mitarbeiter (*PNr* > *Person*, Eintrittsdat, Gehalt)
- Kunde (*PNr* > *Person*, Rabatt, Zahlungsbed)

Person

	PNr	Name	Telefonnr
1	1	Klara Fall	0123456789
2	2	Rob Otter	987654321

Mitarbeiter

	PNr	Eintrittsdatum	Gehalt
1	1	2023-01-01	3999
2	2	2020-01-01	4999

Kunde

	PNr	Rabatt	Zahlungsbed
1	1	2,99	Nachname
2	2	4,99	Kredit

ÜBUNG: ÜBERLAGERTE-ENTITY TYPEN – VARIANTE 1A

Vorteile:

- strukturell sauber,
- keine Redundanzen

Nachteile:

- Vollständige Information aller Attribute über einen Sub-Typ auf zwei Tabellen verteilt → sinnvolle Anwendung einer View
- Information über Sub-Typ-Zuordnung in eigenen Tabellen
- viele Tabellen

bevorzugte Lösung!

IMPLEMENTIERUNG IN TABELLEN – VARIANTE 1B

Eine Tabelle pro Entity-Typ

- PK der Sub-Entity-Typen sind dieselben wie der Primärschlüssel des Super-Entity-Typs
- PK sind auch die Fremdschlüssel zum Super-Entity-Typ

Horizontal Mapping: Tabellen der Sub-Entity-Typen übernehmen alle Attribute vom Super-Entity-Typ

- Person (PNr, Name, Telefonnr)
- Mitarbeiter (*PNr* > *Person*, Name, Telefonnr, Eintrittsdat, Gehalt)
- Kunde (*PNr* > *Person*, Name, Telefonnr, Rabatt, Zahlungsbed)

Person

	PNr	Name	Telefonnr
1	1	Klara Fall	0123456789
2	2	Rob Otter	987654321

Mitarbeiter

	PNr	Name	Telefonnr	Eintrittsdatum	Gehalt
1	1	Klara Fall	0123456789	2023-01-01	3999
2	2	Rob Otter	987654321	2020-01-01	4999

Kunde

	PNr	Name	Telefonnr	Rabatt	Zahlungsbed
1	1	Klara Fall	0123456789	2,99	Nachname
2	2	Rob Otter	987654321	4,99	Kredit

IMPLEMENTIERUNG IN TABELLEN – VARIANTE 1B

Vorteile:

- Vollständige Information aller Attribute über einen Sub-Typ in einer Tabelle

Nachteile:

- Redundanzen (sogar mehrfach, wenn overlapping)
- Information über Sub-Typ-Zuordnung in eigenen Tabellen
- viele Tabellen

IMPLEMENTIERUNG IN TABELLEN – VARIANTE 2

Eine Tabelle pro Sub-Entity-Typ

- Mitarbeiter (PNr, Name, Telefonnr, Eintrittsdat, Gehalt)
- Kunde (PNr, Name, Telefonnr, Rabatt, Zahlungsbed)

Mitarbeiter

	PNr	Name	Telefonnr	Eintrittsdatum	Gehalt
1	1	Klara Fall	0123456789	2023-01-01	3999
2	2	Rob Otter	987654321	2020-01-01	4999

Kunde

	PNr	Name	Telefonnr	Rabatt	Zahlungsbed
1	1	Klara Fall	0123456789	2,99	Nachname
2	2	Rob Otter	987654321	4,99	Kredit

ÜBUNG: ÜBERLAGERTE-ENTITY TYPEN – VARIANTE 2

Vorteile:

- Vollständige Information (alle Attribute) über einen Sub-Typ in einer Tabelle

Nachteile:

- Speicherung der Attribute des Super-Typs bei Completeness Constraint partial
- Redundanzen (sogar mehrfach, wenn overlapping),
- Information über Sub-Typ-Zuordnung in eigenen Tabellen
- Zugriff auf alle Entities des Super-Typs (union)

unübliche Lösung (nur überlegenswert, wenn total und disjoint)

ÜBUNG: ÜBERLAGERTE-ENTITY TYPEN – VARIANTE 3

Eine Tabelle für alle Entity-Typen (Filtered Mapping)

- Person (PNr, Name, Telefonnr, Eintrittsdat, Gehalt, Rabatt, Zahlungsbed)

Zuordnung zu den Sub-Entity-Typen über Abfrage der NULL-Werte aufwendig, daher

- a) zusätzlich ein Attribut: ist von welchem Sub-Typ (wenn **disjoint**)

  Person (PNr, Name, Telefonnr, **PArt**, Eintrittsdat, Gehalt, Rabatt, Zahlungsbed)

- b) zusätzlich pro Sub-Typ ein Attribut (Datentyp Logical): ist von diesem Sub-Typ (wenn **overlapping**)

  Person (PNr, Name, Telefonnr, **IsMitarbeiter**, Eintrittsdat, Gehalt, **IsKunde**, Rabatt, Zahlungsbed)

Ein eigener Entity-Typ PersonTyp bietet sich an → FK PersonTypID statt string

Person

	PNr	Name	Telefonnr	PArt	Eintrittsdatum	Gehalt	Rabatt	Zahlungsbed
1	1	Klara Fall	0123456789	Mitarbeiter	2023-01-01	3999	2,99	Nachname
2	2	Rob Otter	987654321	Kunde	2020-01-01	4999	4,99	Kredit

Bool als Frage formulieren → IsMitarbeiter

Person

	PNr	Name	Telefonnr	IsMitarbeiter	IsKunde	Eintrittsdatum	Gehalt	Rabatt	Zahlungsbed
1	1	Klara Fall	0123456789	T	F	2023-01-01	3999	2,99	Nachname
2	2	Rob Otter	987654321	F	T	2020-01-01	4999	4,99	Kredit

IMPLEMENTIERUNG IN TABELLEN – VARIANTE 3

Vorteile:

- Vollständige Information aller Attribute über einen Sub-Typ in einer Tabelle
- Information über Sub-Typ-Zuordnung direkt beim Super-Typ
- nur eine Tabelle

Nachteile:

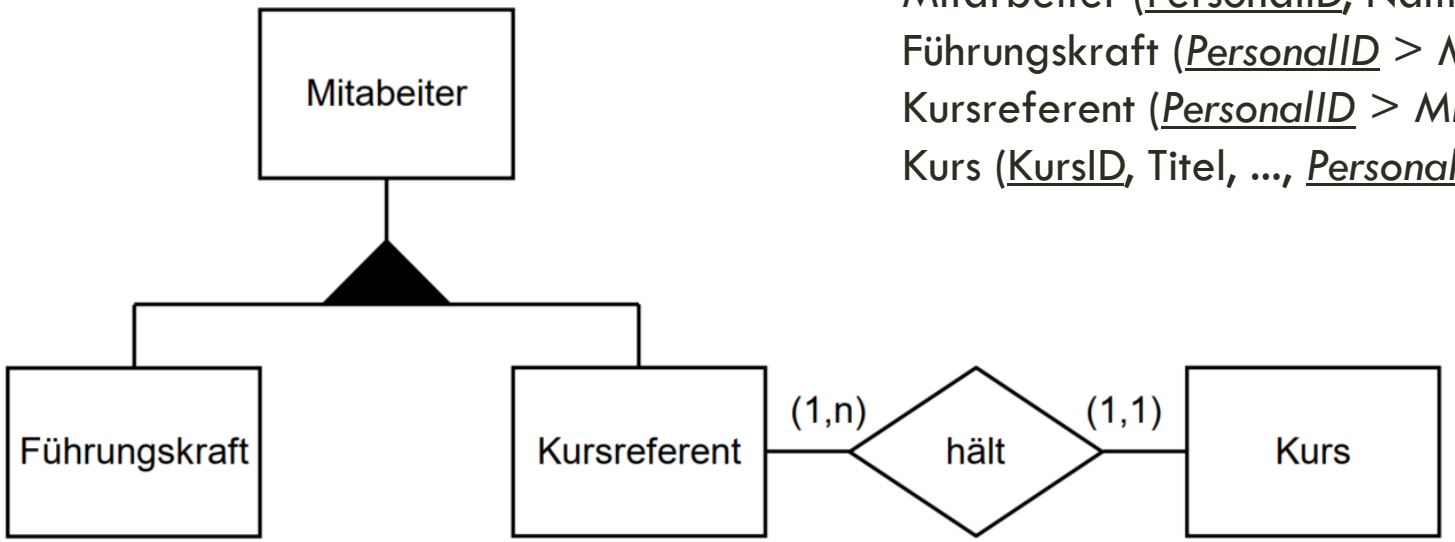
- viele NULL-Werte in den Attributen der Sub-Typen (speziell, wenn partial und disjoint)

Überlegenswert, wenn Sub-Typen wenige zusätzliche Attribute enthalten (und diese überwiegend nicht NULL sind)

ÜBUNG: IMPLEMENTIERUNG IN TABELLEN

Untersuchen Sie folgendes Beispiel um partiell und nicht disjunkt sinnvoll zu implementieren.

partiell, nicht disjunkt:



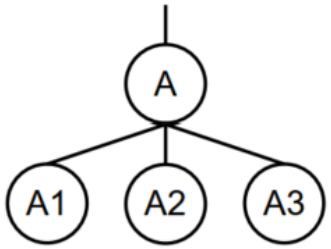
Mitarbeiter (PersonalID, Name, Adresse, ...)
Führungskraft (*PersonalID* > *Mitarbeiter*, Geleitete Abteilung, ...)
Kursreferent (*PersonalID* > *Mitarbeiter*, Stundenkosten, Kursstunden, ...)
Kurs (KursID, Titel, ..., *PersonalID* > *Kursreferent*)

ARTEN VON ATTRIBUTEN 1/3

Einfaches Attribut: Ein Attributwert ist nicht weiter unterteilt.

Zusammengesetztes Attribut: Der Attributwert besteht aus mehreren Teilen.

- Beispiel: Adresse ist unterteilt in Straße, Hausnummer, Stadt, Land
  Darstellung: Adresse(Straße, Hausnummer, Stadt, Land)
- Beispiel: Name ist unterteilt in Vorname, Nachname
  Darstellung: Name(Vorname, Nachname)



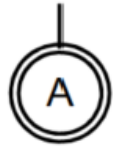
ARTEN VON ATTRIBUTEN 2/3

Einwertiges Attribut: Bei einem Entity hat das Attribut einen Wert.

Mehrwertiges Attribut: Bei einem Entity treten mehrere Werte in diesem Attribut auf.

- Beispiel: Farbe eines Autos, Titel einer Person, Vornamen einer Person, Telefonnummern eines Angestellten Qualifikation eines Angestellten

  Darstellung: In geschwungenen Klammern Graphisch:



ARTEN VON ATTRIBUTEN 3/3

Basis-Attribut: Der Attributwert eines Entities kann von keinem (keinen) anderen Wert eines Attributs (Werten von Attributen) abgeleitet werden.

Abgeleitetes Attribut: Der Attributwert eines Entities kann von einem Wert (mehreren Werten) eines Attributs (mehrerer Attribute) abgeleitet werden.

- Führt zu Redundanzen in der Datenverwaltung und ist im allgemeinen zu vermeiden.
- Beispiele: Alter - Geburtsdatum, Auftragswert - Summe Positionswerte des Auftrags

